

Hardware-Software Co-Design for Accelerating Sparse Tensor Algebra

Marco Siracusa

Barcelona, 2026

Advisors:

Adrià Armejach

Miquel Moretó Planas



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH



**Barcelona
Supercomputing
Center**
Centro Nacional de Supercomputación

A thesis submitted in fulfillment of the requirements for the degree of
Doctor of Philosophy

in the Departament d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

To my family, friends, and colleagues whose support made this work possible.

Acknowledgements

ACKNOWLEDGMENT

Abstract

EN—Many modern scientific and machine learning workloads exhibit inherently sparse and irregular structure. However, traditional supercomputers and datacenters are optimized for dense, regular computation. As a result, sparse workloads use only a fraction of available system resources, leaving significant performance untapped. Although prior solutions have been proposed, they typically target narrow classes of sparse tensor algebra operations, do not fully leverage modern vector compute cores, or leave much of the programmability burden to the user, limiting widespread adoption. To address this gap, this thesis presents a hardware–software co-design for the efficient execution of general sparse tensor algebra operations. We begin by analyzing the architectural implications of sparsity across a broad range of scientific and machine learning applications. Our analysis reveals that traditional compute cores are fundamentally ill-suited to the irregular tensor traversal and coordinate merging required to operate on sparse tensor formats. Building on these insights, we design a Decoupled Access–Execute (DAE) architecture that offloads these operations to a specialized unit, the Tensor Marshaling Unit (TMU), while leaving computation on the core. We demonstrate that the resulting DAE architecture outperforms traditional CPUs and GPUs by up to an order of magnitude. To deliver these gains without exposing the complexity of DAE programming to end users, we also present Ember, a compiler that lowers sparse machine learning operations to DAE code for architectures such as TMU–CPU systems. Ember is implemented in MLIR, enabling natural integration with PyTorch and TensorFlow. By automating this mapping, Ember unlocks the full performance potential of the TMU without imposing an additional programmability burden.

Abstract

CA—Moltes càrregues de treball científiques i d'aprenentatge automàtic modernes presenten una estructura inherentment dispersa i irregular. Tanmateix, els superordinadors i centres de dades tradicionals estan optimitzats per a computació densa i regular. Com a resultat, les càrregues de treball disperses només utilitzen una fracció dels recursos disponibles del sistema, deixant sense aprofitar un rendiment significatiu. Tot i que s'han proposat solucions prèvies, aquestes solen adreçar-se a classes estretes d'operacions d'àlgebra de tensors dispersos, no aprofiten plenament els nuclis vectorials moderns, o deixen gran part de la càrrega de programabilitat a l'usuari, cosa que en limita l'adopció generalitzada. Per abordar aquesta mancança, aquesta tesi presenta un codisseny hardware–software per a l'execució eficient d'operacions generals d'àlgebra de tensors dispersos. Comencem analitzant les implicacions arquitectòniques de la dispersió en una àmplia gamma d'aplicacions científiques i d'aprenentatge automàtic. La nostra anàlisi revela que els nuclis de còmput tradicionals són fonamentalment poc adequats per al recorregut irregular de tensors i la fusió de coordenades necessaris per operar sobre formats de tensors dispersos. A partir d'aquestes observacions, dissenyem una arquitectura Decoupled Access–Execute (DAE) que descarrega aquestes operacions a una unitat especialitzada, la Tensor Marshaling Unit (TMU), mentre manté la computació al nucli. Demostrem que l'arquitectura DAE resultant supera les CPU i GPU tradicionals fins a un ordre de magnitud. Per oferir aquests guanys sense exposar la complexitat de la programació DAE als usuaris finals, també presentem Ember, un compilador que tradueix operacions disperses d'aprenentatge automàtic a codi DAE per a arquitectures com ara els sistemes TMU–CPU. Ember està implementat en MLIR, fet que permet una integració natural amb PyTorch i TensorFlow. Mitjançant l'automatització d'aquesta transformació, Ember desbloqueja tot el potencial de rendiment de la TMU sense imposar cap càrrega addicional de programabilitat.

Abstract

ES—Muchas cargas de trabajo científicas y de aprendizaje automático modernas presentan una estructura inherentemente dispersa e irregular. Sin embargo, los superordenadores y centros de datos tradicionales están optimizados para computación densa y regular. Como resultado, las cargas de trabajo dispersas utilizan solo una fracción de los recursos disponibles del sistema, dejando sin explotar un rendimiento significativo. Aunque se han propuesto soluciones previas, estas suelen dirigirse a clases estrechas de operaciones de álgebra de tensores dispersos, no aprovechan plenamente los núcleos vectoriales modernos, o dejan gran parte de la carga de programabilidad al usuario, lo que limita su adopción generalizada. Para abordar esta carencia, esta tesis presenta un codiseño hardware–software para la ejecución eficiente de operaciones generales de álgebra de tensores dispersos. Comenzamos analizando las implicaciones arquitectónicas de la dispersión en una amplia gama de aplicaciones científicas y de aprendizaje automático. Nuestro análisis revela que los núcleos de cómputo tradicionales son fundamentalmente poco adecuados para el recorrido irregular de tensores y la fusión de coordenadas necesarios para operar sobre formatos de tensores dispersos. A partir de estos resultados, diseñamos una arquitectura Decoupled Access–Execute (DAE) que descarga estas operaciones a una unidad especializada, la Tensor Marshaling Unit (TMU), mientras mantiene la computación en el núcleo. Demostramos que la arquitectura DAE resultante supera a las CPU y GPU tradicionales hasta en un orden de magnitud. Para ofrecer estas mejoras sin exponer la complejidad de la programación DAE a los usuarios finales, también presentamos Ember, un compilador que traduce operaciones dispersas de aprendizaje automático a código DAE para arquitecturas como los sistemas TMU–CPU. Ember está implementado en MLIR, lo que permite una integración natural con PyTorch y TensorFlow. Al automatizar esta transformación, Ember desbloquea todo el potencial de rendimiento de la TMU sin imponer una carga adicional de programabilidad.

Table of contents

List of figures	xvii
List of tables	xix
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	3
1.3 Contributions	4
1.3.1 Performance Characterization of Sparse Workloads	4
1.3.2 The Tensor Marshaling Unit	4
1.3.3 The Ember Compiler	5
1.4 Thesis Structure	5
2 Background on Sparse Tensor Workloads	7
2.1 Sparse Tensor Algebra Fundamentals	7
2.1.1 Einsum Expressions	8
2.1.2 Tensor Compression Formats	8
2.1.3 Tensor Traversal	9
2.1.4 Tensor Merging	10
2.1.5 Scan-and-Lookup for Sparse-Dense Operations	12
2.1.6 Nonzero Computation and Reduction	12
2.2 Sparsity in Scientific Computing	13
2.2.1 Sparse Matrix-Vector Multiplication	13
2.2.2 Sparse Matrix-Sparse Matrix Multiplication	13
2.2.3 Sparse Matrix Addition	14
2.2.4 Other Sparse Linear and Tensor Kernels	15
2.3 Sparsity in Machine Learning	16
2.3.1 Graph Learning Models	17

2.3.2	Deep Learning Recommendation Models (DLRMs)	19
2.3.3	Sparse Attention in Language Models	19
2.4	Summary	20
3	Related Work	21
3.1	Sparse Workload Characterization	21
3.1.1	SpMV Characterization	22
3.1.2	Beyond SpMV	22
3.1.3	Embedding Operation Characterization	23
3.1.4	Our Performance Characterization	23
3.2	Architectures Optimized for Sparse Computation	23
3.2.1	CPU Optimizations for Irregular Workloads	24
3.2.2	Fixed-Function Architectures for Sparse Tensor Algebra	24
3.2.3	Flexible Architectures for Sparse Tensor Algebra	24
3.2.4	DAE Architectures and Access Units	25
3.2.5	Production DAE-Like Architectures	25
3.2.6	Our Hardware Architecture: The TMU	25
3.3	DAE Compilation	26
3.3.1	Machine Learning Compiler Stacks	27
3.3.2	Tensor Compilers	27
3.3.3	Compiler Support for DAE Architectures	28
3.3.4	Our Compiler: Ember	28
3.4	Summary	28
4	Architectural Implications of Sparse Tensor Workloads	29
4.1	Sparse Scientific Computing	30
4.1.1	Scientific Workload Methodology	30
4.1.2	CPU Performance Characterization	31
4.2	Machine Learning Embedding Workloads	35
4.2.1	Embedding Workload Methodology	35
4.2.2	Embedding Workload Characterization	37
4.2.3	Embedding Performance on Off-the-Shelf GPUs	41
4.2.4	Embedding Performance on Off-the-Shelf CPUs	43
4.2.5	Ideal Memory-Level Parallelism Requirements	45
4.2.6	The Cost of Scaling Memory-Level Parallelism	53
4.3	Summary	54

5	The Tensor Marshaling Unit for DAE Designs	55
5.1	TMU Programming Interface	56
5.1.1	TMU Overview	56
5.1.2	Programming Tensor Traversal	58
5.1.3	Programming Merging and Vectorization	60
5.1.4	Data Marshaling and Computation	61
5.1.5	Mapping Sparse Tensor Algebra to the TMU	62
5.2	TMU Microarchitecture	64
5.2.1	Traversal Unit Design	65
5.2.2	Traversal Group Design	66
5.2.3	Output Queue Construction	66
5.2.4	Memory Arbiter	69
5.2.5	TU Queue Sizing	69
5.2.6	System Integration	69
5.2.7	TMU Execution Example	71
5.3	Scientific Sparse Tensor Algebra Evaluation	71
5.3.1	Scientific Evaluation Methodology	71
5.3.2	Scientific Performance Analysis	74
5.3.3	System Utilization	76
5.3.4	TMU Sensitivity Analysis	79
5.3.5	Comparison with Prior Sparse Architectures	80
5.3.6	RTL Implementation and Area Analysis	81
5.4	Machine Learning Embedding Evaluation	81
5.4.1	Embedding Evaluation Methodology	81
5.4.2	Architectural Benefits of Decoupling	82
5.4.3	TMU-Enabled DAE Processor vs. CPU	83
5.4.4	TMU-Enabled DAE Processor vs. GPU	84
5.4.5	Specialized TMU-Enabled DAE Architectures	84
5.5	Summary	92
6	The Ember Compiler for DAE Compilation	93
6.1	A DAE Programming Abstraction for Embedding Operations	94
6.2	Ember Compiler Design	98
6.2.1	Compiler Overview	98
6.2.2	From SCF to DLC	100
6.2.3	SLC-Level Optimizations	104
6.3	Ember Evaluation	110

6.3.1	Evaluation Methodology	110
6.3.2	Impact of SLC-Level Optimizations	111
6.3.3	Impact of Model-Specific Optimizations	112
6.3.4	Comparison with Hand-Optimized Code	113
6.4	Summary	114
7	Conclusions	115
7.1	Lessons Learned	115
7.2	Research Outputs, Awards, and Follow-On Work	116
7.2.1	Publications and Patents	116
7.2.2	Awards	117
7.2.3	Follow-On Research Projects	117
7.2.4	Research Appointments and Professional Contributions	118
8	Future Work	119
8.1	Broader Workload and System Characterization	119
8.2	TMU Implementation and System Integration	119
8.3	Extending the TMU Architecture	120
8.4	Extending Ember	121
	Bibliography	125

List of figures

2.1	Sparse tensor compression formats.	8
2.2	Disjunctive and conjunctive merging of sparse tensors.	10
2.3	CSR elementwise addition and multiplication.	11
2.4	Visualization of sparse matrix-vector multiplication.	12
2.5	Pseudocode of sparse matrix-vector multiplication.	14
2.6	Pseudocode of sparse matrix-sparse matrix multiplication.	15
2.7	Visualization of graph convolution.	18
2.8	Pseudocode of graph convolution.	18
2.9	Visualization of embedding lookup operations for language models.	19
4.1	Roofline model of scientific kernels on CPU.	32
4.2	Cycles spent stalling in scientific kernels on CPU.	34
4.3	Performance bottleneck of sparse ML models on GPU.	42
4.4	Cache miss rate for embedding lookups on CPU.	44
4.5	Load-to-use latency of embedding lookups on CPU.	46
4.6	Architectural implications of embedding lookups on CPU.	47
4.7	System abstraction for the analytical memory-level parallelism model.	48
4.8	Cumulative distribution functions of the reuse distance of the selected graphs.	49
4.9	Accuracy of our analytical model compared to gem5.	50
4.10	Ideal memory-level parallelism on CPU.	52
4.11	The cost of scaling memory-level parallelism on CPU.	53
5.1	TMU architecture and placement in a multicore processor.	55
5.2	SpMV decomposition into traversal, lookup, and computation.	57
5.3	SpMV callbacks.	58
5.4	SpKAdd callback.	61
5.5	TMU code for column-parallel SpMV.	64
5.6	Step-by-step TMU execution for SpMV, part 1.	67

5.7	Step-by-step TMU execution for SpMV, part 2.	68
5.8	DAE speedups for scientific kernels.	74
5.9	Performance breakdown of a CPU and DAE processor on scientific kernels.	75
5.10	Roofline models of a CPU and DAE processor on scientific kernels.	77
5.11	Read-to-write ratio of scientific workloads.	78
5.12	Performance of different TMU configurations.	79
5.13	TMU vs. other architectural solutions for accelerating scientific models.	80
5.14	Embedding execution on the evaluated DAE processor.	82
5.15	TMU and CPU memory-level parallelism for graph convolution.	83
5.16	DAE core benefits for graph convolution.	84
5.17	Bandwidth scaling of DAE cores vs traditional cores for graph convolution.	85
5.18	Performance potential of TMU on graph convolution.	85
5.19	Performance potential of TMU on embedding operations.	86
5.20	Performance and efficiency of DAE processors vs. GPUs on GNNs.	87
5.21	Power breakdown of our DAE processor for graph convolution.	88
5.22	Compute and cache utilization of our DAE processor for graph convolution.	89
5.23	Performance and efficiency of specialized DAE configurations.	91
6.1	Logical DAE abstraction used by Ember.	94
6.2	DLC representation of an EB operation.	96
6.3	Overview of the Ember compiler.	99
6.4	SLC IR grammar.	100
6.5	Lowering from SCF to Ember SLC IR.	102
6.6	Impact of SLC optimizations on DLC queues and compute code.	105
6.7	Progressive SLC optimization steps.	106
6.8	Performance speedup of Ember optimizations.	111
6.9	Impact of Ember optimizations on access and compute throughput.	112
6.10	Impact of cache optimizations for sparse LLMs.	113
6.11	Performance of Ember generated code.	114

List of tables

3.1	Qualitative comparison of sparse-acceleration architectures.	26
4.1	Specifications of the CPUs used for sparse scientific workloads.	31
4.2	Summary of CPU resource utilization for sparse scientific kernels.	33
4.3	Summary of CPU stalls on sparse scientific kernels.	34
4.4	Specifications of the CPU baseline for embedding workloads.	36
4.5	Tested Deep Learning Recommendation Model configurations.	36
4.6	Selected inputs for graph-learning models.	37
4.7	Algorithmic characterization of embedding operations.	38
4.8	Memory characterization of embedding operations.	39
5.1	TU traversal primitives.	59
5.2	TU data streams.	59
5.3	TMU inter-layer configurations.	59
5.4	Mapping sparse tensor algebra kernels to TMU hardware.	63
5.5	Specifications of the simulated DAE multicore for scientific workloads. . .	72
5.6	Selected inputs for scientific sparse tensor workloads.	73
5.7	Specifications of DAE big and little cores.	88
5.8	Specialized DAE configurations.	89
6.1	Evaluated code and reference implementations.	110

Chapter 1

Introduction

This thesis proposes a hardware–software co-design for accelerating general sparse tensor algebra while preserving the programmability and flexibility of general-purpose systems. This chapter motivates the problem, defines the research objectives, summarizes the main contributions, and outlines the remainder of the document.

1.1 Motivation

Many of today’s scientific and machine learning models—including billion-dollar recommendation models and large-scale scientific solvers and simulators—are inherently sparse [50, 49, 157]. Yet modern datacenters and supercomputers are optimized for dense computation [103]. This fundamental mismatch calls for architectures and systems that can execute sparse workloads efficiently.

A tensor—a multidimensional array—is said to be *sparse* when most of its entries are zero [70]. Many real-world datasets naturally exhibit that structure [33]. For example, the *follow* relation in a social network can be represented as a matrix A where $A_{i,j} = 1$ if user i follows user j , and zero otherwise [78]. Since users typically follow only a small fraction of all other users, this matrix is overwhelmingly sparse. Similarly, the internal states of scientific solvers and simulations often contain mostly zero-valued elements [15, 157].

Because zero elements neither need to be stored nor computed, sparse tensors are represented using compressed formats that track only nonzero values and their coordinates [15]. Operating on these formats, however, requires irregular traversals and data-dependent control flow [135]. For example, summation requires merging nonzeros based on their coordinates, which entails frequent data-dependent branch operations [4]. Multiplication often requires locating parts of tensors that share a common index, leading to both irregular memory accesses and additional data-dependent control flow.

Traditional datacenters and supercomputers are not designed for such irregular computation. Instead, they are heavily optimized for dense and regular workloads—such as dense matrix multiplication and convolution—where data is laid out contiguously and accessed with predictable patterns [156, 101, 106]. On these architectures, however, data-dependent branches cause frequent misspeculation and costly pipeline flushes [135]. Meanwhile, irregular memory accesses degrade cache utilization and increase memory latency, ultimately clogging the memory subsystem [159, 135].

As sparse workloads become increasingly pervasive, there is growing interest in hardware and software solutions tailored to their unique characteristics [50]. Programmable prefetchers represent an initial attempt to adapt traditional general-purpose architectures to irregular access patterns [163]. Although they require minimal hardware changes, they can only accelerate irregular memory accesses, not control flow, and provide limited speedups [140]. At the other extreme, specialized accelerators for fixed sparse kernels can deliver substantial performance gains [165], but at the cost of limited flexibility. Reconfigurable architectures broaden the range of supported kernels [123, 155], yet still suffer from classic accelerator drawbacks, including kernel launch overheads and constrained memory capacity.

Decoupled Access–Execute (DAE) architectures [141] offer a promising middle ground between the flexibility of general-purpose processors and the efficiency of specialization. They separate memory access from computation into distinct units, typically connected through queues: an access unit optimized for memory operations, and an execute unit—often a general-purpose core—focused on computation. Hyperscalers have already demonstrated the potential of this design by accelerating irregular vector lookups in recommendation models [65, 41], while other authors have shown similar benefits for scalar lookup operations in scientific computing [14, 110]. Prior work has also demonstrated that DAE designs can successfully optimize more complex sparse operations, including sparse-format compression and decompression [66, 161] and irregular sparse merging [129, 130]. However, although current proposals demonstrate the promise of DAE designs, they do not yet realize the full potential of this approach: they either target narrow classes of sparse tensor algebra operations, do not fully leverage modern vector compute cores, or leave much of the programmability burden to the user. This thesis addresses these limitations by proposing a vector-friendly DAE architecture and compiler stack specifically designed to accelerate *general* sparse tensor algebra operations.

1.2 Objectives

The primary goal of this thesis is to overcome the fundamental challenges of general sparse tensor algebra and enable its efficient execution at scale. To achieve this goal, we propose a hardware–software co-design based on a DAE architecture that spans both processor microarchitecture and the compiler stack. Unlike other solutions in the literature, we aim to design a DAE system that can efficiently accelerate a broad range of sparse and dense tensor algebra operations. We identify the following specific research objectives:

- **Characterize the microarchitectural implications of sparsity:** To design efficient hardware, we must first quantify how irregular control flow and memory access impact the performance of sparse tensor operations in scientific and machine learning workloads. Although it is well known that sparse kernels such as sparse matrix-vector multiplication perform poorly on traditional architectures [83, 18, 125, 42, 7, 48, 151, 157, 104], our objective is to generalize this understanding to a broader class of sparse tensor algebra operations in different domains, each with distinct characteristics and performance challenges, and to delve deeper into the fundamental causes of their inefficiency. By characterizing how these workloads stress modern processors, we establish a detailed understanding of why existing architectures fall short and what requirements future designs must satisfy to scale effectively.
- **Design an efficient vector-friendly DAE architecture for sparse tensor algebra:** Building on this analysis, we aim to develop a DAE architecture capable of overcoming all the identified bottlenecks. Our objective is to design a general and scalable solution that accelerates a broad class of sparse tensor algebra workloads while preserving the throughput of modern vector execution and outperforming traditional CPUs and GPUs in both raw performance and performance per watt.
- **Hide DAE programming complexity from end users:** To facilitate widespread adoption of the proposed architecture, we aim to hide the complexity of the DAE programming model from end users. In scientific computing, this complexity can often be hidden behind small hand-written libraries, such as DAE implementations of BLAS-like routines [77]. This approach, however, does not scale to machine learning models. Modern machine learning compilers explore a wide range of formats, precisions, optimizations, and fusion opportunities [74], making it impractical to hand-write DAE kernels for every variant. Therefore, to fully unlock the benefits of the proposed architecture, we aim to design a generic compiler stack that lowers sparse machine learning

operations to DAE code while integrating with frameworks such as PyTorch and TensorFlow [114, 1].

1.3 Contributions

This thesis addresses these research objectives through the following contributions.

1.3.1 Performance Characterization of Sparse Workloads

The first contribution of this thesis is a comprehensive analysis of the architectural requirements imposed by sparsity across scientific and machine learning workloads. In Chapter 4, we first measure the performance impact of the irregular memory accesses and data-dependent control flow introduced by sparse tensor traversal and merging, showing that both are severe bottlenecks. We then estimate the ideal resource requirements of these operations and show that traditional architectures, which execute traversal, merging, and computation on the same compute cores, cannot satisfy them.

This contribution led to the following publications:

- Francesco Sgherzi, **Marco Siracusa**, Ivan Fernandez, Adrià Armejach, Miquel Moretó. 2024. *SpChar: Characterizing the Sparse Puzzle via Decision Trees*. *Journal of Parallel and Distributed Computing* [135].
- **Marco Siracusa**, Miquel Moretó, Adrià Armejach. Accelerating Graph Machine Learning with Decoupled Memory-Access-Execute Architectures. *Under submission*.

1.3.2 The Tensor Marshaling Unit

The second contribution is the *Tensor Marshaling Unit* (TMU), a specialized hardware unit designed to decouple tensor traversal and merging from the compute core, enabling scalable execution. Unlike prior DAE access units, the TMU (1) supports a broad class of sparse and dense tensor algebra expressions, and (2) integrates seamlessly with modern vector units to exploit structured and unstructured parallelism. In Chapter 5, we present the TMU architecture, its integration into a multicore DAE processor composed of TMUs and CPUs, and a detailed evaluation of performance, power, and area. The resulting DAE processor outperforms traditional CPUs by up to an order of magnitude and GPUs by up to 6× in both performance and performance per watt.

This contribution led to the following publications:

- **Marco Siracusa**, Víctor Soria-Pardos, Francesco Sgherzi, Joshua Randall, Douglas J. Joseph, Miquel Moretó Planas, Adrià Armejach. 2023. *A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors*. In Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23) [140].
- **Marco Siracusa**, Miquel Moretó, Adrià Armejach. Accelerating Graph Machine Learning with Decoupled Memory-Access–Execute Architectures. *Under submission*.

and the following patent:

- **Marco Siracusa**, Joshua Randall, Douglas James Joseph, Miquel Moretó Planas, Adrià Armejach Sanosa. March 6, 2025. *Data structure marshalling unit*. US 20250077499A1. Barcelona Supercomputing Center; Arm Ltd. [139].

1.3.3 The Ember Compiler

The third contribution is the *Ember compiler*, a generic compiler that lowers sparse machine learning operations to optimized DAE code. Ember is implemented in MLIR, enabling integration with PyTorch through torch-mlir and with TensorFlow through its native MLIR compiler stack. In Chapter 6, we first show why manually writing and optimizing DAE programs for machine learning workloads is challenging. We then present Ember’s intermediate representations and compilation strategies, which are designed for automated DAE optimization and integrate tightly with modern tensor compilers. Finally, we demonstrate that Ember achieves performance comparable to hand-optimized implementations, unlocking the TMU’s potential without imposing additional programmability burden on users.

This contribution led to the following publication:

- **Marco Siracusa**, Olivia Hsu, Víctor Soria-Pardos, Joshua Randall, Arnaud Grasset, Eric Biscondi, Doug Joseph, Randy Allen, Fredrik Kjolstad, Miquel Moretó Planas, Adrià Armejach. 2026. *Ember: A Compiler for Embedding Operations on Decoupled Access–Execute Architectures*. In Proceedings of the 24th Annual ACM/IEEE International Symposium on Code Generation and Optimization (CGO '26) [138].

1.4 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 provides background on sparse tensor algebra and its role in scientific and machine learning workloads.
- Chapter 3 reviews prior hardware and software approaches for sparse tensor algebra and motivates the need for new DAE systems such as the TMU–CPU architecture proposed in this thesis.
- Chapter 4 presents the first contribution: a quantitative characterization of sparse workloads and the architectural insights that guide our co-design.
- Chapter 5 presents the Tensor Marshaling Unit, its integration into a DAE multicore processor, and a comprehensive evaluation.
- Chapter 6 presents the Ember compiler, the challenges of decoupled execution, and the corresponding compilation techniques and evaluation.
- Chapter 7 summarizes the key findings of this thesis.
- Chapter 8 outlines promising directions for future research.

Chapter 2

Background on Sparse Tensor Workloads

This chapter introduces the sparse tensor algebra concepts that underlie the workloads, analyses, and designs developed throughout this thesis. Section 2.1 explains how sparse tensor computations are expressed, which sparse formats are commonly used, and which irregular operations these formats require, including traversal, merging, and scan-and-lookup. Sections 2.2 and 2.3 then review common scientific and machine learning workloads that leverage sparsity.

2.1 Sparse Tensor Algebra Fundamentals

Sparse tensor algebra provides the abstraction that connects the different workloads studied in this thesis. We first introduce Einsum expressions as a high-level way to describe tensor computations. We then describe sparse tensor formats and the core operations they induce: traversal to recover compressed coordinates and values, merging to co-iterate sparse operands, scan-and-lookup for sparse–dense access patterns, and computation over the resulting nonzero elements.

An *order- n* tensor is an n -dimensional collection of data; for example, A_{ijk} denotes the scalar element at position (i, j, k) of an order-3 tensor A . A widely used way of describing tensor computation, particularly in machine learning frameworks such as PyTorch [114], is through Einsum expressions [70].

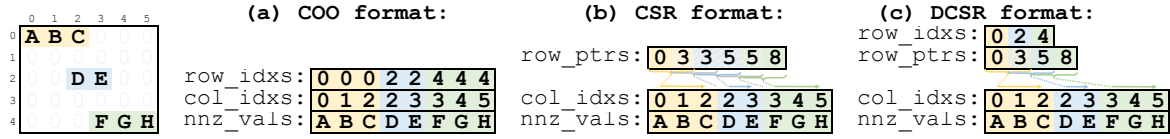


Figure 2.1: Sparse matrix represented in three common compression formats: Coordinate (COO), Compressed Sparse Row (CSR), and Doubly-Compressed Sparse Row (DCSR). COO stores every nonzero coordinate explicitly; CSR replaces repeated row coordinates with row pointers into the column-index and value arrays; DCSR further stores only nonempty rows and their row pointers. In general, the best sparse format depends on the tensor’s sparsity structure.

2.1.1 Einsum Expressions

Einsum expressions specify tensor computations by describing how input and output dimensions relate: (i) input dimensions with repeated indices are combined *element-wise*, (ii) input dimensions whose index is absent from the output are *contracted*, and (iii) all other dimensions are copied to the output. For instance, matrix–vector multiplication is expressed as $Z_i = A_{ij}B_j$. Because the index j does not appear in the output, it is contracted (i.e., multiplied element-wise and summed). Similarly, matrix multiplication is $Z_{ij} = A_{ik}B_{kj}$, contracting along k . By contrast, matrix addition is written as $Z_{ij} = A_{ij} + B_{ij}$, expressing element-wise addition across all coordinates.

These expressions are implemented in software as deep loop nests, where each loop iterates over and combines tensor *fibers*—one-dimensional views of a tensor (e.g., matrix rows or columns) [132]. The loop order is determined by an index *schedule* [70]. For example, an inner-product implementation of matrix multiplication uses the schedule ijk , an outer-product implementation uses kij , and a dataflow implementation uses ikj [44]. However, the iteration bounds of these loops depend on the compression format in which input tensors are stored [29], as discussed next.

2.1.2 Tensor Compression Formats

Sparse formats determine how tensor coordinates and values are stored, which metadata must be traversed, and which memory-access patterns the hardware and compiler must support.

Dense tensors store their fibers contiguously according to a specified data layout (e.g., row- or column-major). Sparse tensors instead rely on compressed formats that store only nonzero values and enough metadata to recover their coordinates during traversal. Figure 2.1 summarizes several widely used sparse matrix formats [29]. Specifically:

- (a) The Coordinate (COO) format (Figure 2.1a) explicitly stores nonzero positions as n -dimensional coordinates—`row_idxs` and `col_idxs` for matrices—typically sorted by a multidimensional ordering (e.g., row-major).
- (b) For matrices with $\text{num_nonzeros} > \text{num_rows} + 1$, the Compressed Sparse Row (CSR) format [15] (Figure 2.1b) improves upon COO by storing row boundaries in a `row_ptr` array, eliminating repeated row indices. Therefore, in the CSR format, the `col_idxs` vector stores the column indices of its nonzero elements. The `row_ptr` vector stores the position at which each row begins.
- (c) When $\text{num_rows} \gg \text{num_nonempty_rows}$, the Doubly-Compressed Sparse Row (DCSR) format [24] (Figure 2.1c) further compresses the `row_ptr` by omitting empty rows.

Higher-dimensional tensors are typically stored either in COO or in Compressed Sparse Fiber (CSF) [144], a generalization of DCSR that compresses fibers at each tensor dimension.

Chou et al. [29] formalize a hierarchical abstraction for expressing these formats using *six level formats*. For example,

- CSR can be represented as a *dense* level followed by a *compressed* level,
- DCSR requires two compressed levels,
- COO is expressed as a set of *singleton* levels, one per dimension,
- CSF is expressed as a hierarchy of compressed levels.

This abstraction enables the construction of arbitrarily complex formats, allowing developers to tune performance and storage efficiency for specific algorithms [70].

2.1.3 Tensor Traversal

Tensor traversal is the mechanism that turns a compressed sparse format into a stream of coordinates and values, often through metadata-dependent and irregular memory accesses.

Each level format defines a specific *traversal* (i.e., iteration and loading) method, such as:

1. Dense traversal:

```
for(idx = 0; idx < fiber_size; idx++)
    val = vals[idx];
```

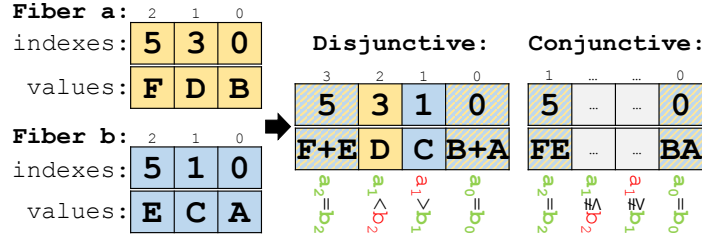


Figure 2.2: Disjunctive and conjunctive merging of two sparse tensor fibers. Disjunctive merging emits every coordinate that appears in either input fiber, while conjunctive merging emits only coordinates that appear in both input fibers. This distinction determines whether a sparse-sparse operation computes over the union or the intersection of input coordinates.

2. Compressed traversal:

```
for(p = ptr[i]; p < ptr[i+1]; p++) {
    idx = idxs[p];
    val = vals[p];
}
```

3. Coordinate singleton traversal:

```
for(p = 0; p < num_nonzeros; p++) {
    idx0 = idxs0[p];
    idx1 = idxs1[p];
    val = vals[p];
}
```

Tensor traversal is implemented by nesting these level-specific functions into deep loop hierarchies. To perform computation, multiple tensors are often co-iterated and combined, typically at the fiber level. For example, summing two tensors requires co-iterating their fibers and adding elements with matching coordinates.

2.1.4 Tensor Merging

When an operation combines multiple sparse operands, traversal alone is not enough: the program must also decide which sparse coordinates align across operands, introducing data-dependent control flow.

When both tensors are sparse, fibers must be *merged*. As illustrated in Figure 2.2, element-wise addition corresponds to *disjunctive* merging, while element-wise multiplication corresponds to *conjunctive* merging. Because $0 + x = x$, disjunctive merging co-iterates fibers

```

CSR csr_add(const CSR& A,
            const CSR& B) {
  for (int i = 0; i < m; ++i) {
    int p = A.rowptr[i];
    int q = B.rowptr[i];
    int pe = A.rowptr[i + 1];
    int qe = B.rowptr[i + 1];

    while (p < pe || q < qe) {
      int ca = (p < pe) ? A.col[p] : INF;
      int cb = (q < qe) ? B.col[q] : INF;
      if (ca < cb) {
        emit(ca, A.val[p]);
        ++p;
      } else if (cb < ca) {
        emit(cb, B.val[q]);
        ++q;
      } else {
        emit(ca, A.val[p] + B.val[q]);
        ++p; ++q;
      }
    }
    close_row();
  }
}

```

(a) Elementwise CSR matrix addition.

```

CSR csr_mul(const CSR& A,
            const CSR& B) {
  for (int i = 0; i < m; ++i) {
    int p = A.rowptr[i];
    int q = B.rowptr[i];
    int pe = A.rowptr[i + 1];
    int qe = B.rowptr[i + 1];

    while (p < pe && q < qe) {
      int ca = A.col[p];
      int cb = B.col[q];
      if (ca < cb) {
        ++p;
      } else if (cb < ca) {
        ++q;
      } else {
        emit(ca, A.val[p] * B.val[q]);
        ++p; ++q;
      }
    }
    close_row();
  }
}

```

(b) Elementwise CSR matrix multiplication.

Figure 2.3: Pseudocode for elementwise addition and multiplication of two CSR matrices. The helper `emit` appends one coordinate–value pair to the output row, and `close_row` updates the output row pointer. Addition and multiplication differ in their merge semantics, but both rely on data-dependent coordinate comparisons over compressed fibers to decide which inputs advance and which nonzeros are emitted.

and, at each step, advances the fiber with the smallest coordinate, summing values when coordinates match [59]. Conjunctive merging similarly advances fibers but outputs a value only when all fibers share the same coordinate, exploiting the fact that $0 \cdot x = 0$ [53].

These operations are typically implemented using while loops with if-then-else logic [70]. Figure 2.3 shows how CSR matrix addition and multiplication implement these two merge semantics: addition advances and emits the smallest coordinate from either input, whereas multiplication advances until both inputs expose the same coordinate.

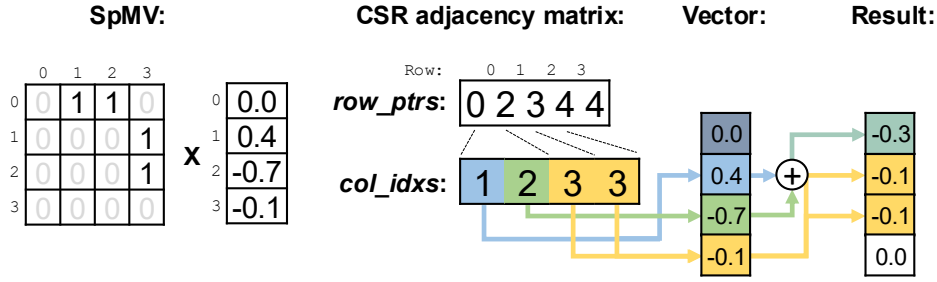


Figure 2.4: Sparse Matrix–Vector Multiplication (SpMV) on a matrix stored in Compressed Sparse Row (CSR) format [15]. Each row scans its nonzero entries and uses their column indices to look up elements of the dense input vector; the fetched vector elements are then scaled by the corresponding nonzero value and accumulated into the output, as shown in Figure 2.5. The scaling step is omitted here for visual clarity. As a result, while SpMV does not require coordinate merging, the scan-and-lookup pattern still introduces irregular memory accesses.

2.1.5 Scan-and-Lookup for Sparse–Dense Operations

Sparse–dense operations—such as multiplying a sparse matrix by a dense vector or a sparse matrix by a dense matrix—could, in principle, be expressed through merging. In practice, however, they are usually implemented with an equivalent yet more efficient *scan-and-lookup* operation. This pattern scans a compressed sparse fiber and uses each sparse coordinate to look up the corresponding value in a dense operand.

Scan-and-lookup is central to several workloads studied in this thesis. In Sparse Matrix–Vector Multiplication (SpMV) using CSR, for example, the program scans the nonzeros of each sparse matrix row and uses their column indices to look up elements of the dense input vector, as shown in Figure 2.4. A similar pattern appears in machine learning embedding operations, where sparse IDs are scanned and used to fetch dense embedding vectors rather than scalar elements.

2.1.6 Nonzero Computation and Reduction

After merging, fiber elements are computed and written back to memory. If the output is dense, results are stored directly. For compressed outputs, the algorithm typically proceeds in two phases: a *symbolic* phase that computes or estimates the size of the output data structure, and a *numeric* phase that performs floating-point computation and writes results [44]. A related fundamental operation is tensor *reduction*, which accumulates multiple fibers into a single one [100, 85]. Given a stream of coordinate–value pairs with possible coordinate

repetitions, reduction produces a stream with unique coordinates by accumulating values of pairs sharing the same coordinate.

2.2 Sparsity in Scientific Computing

Many scientific simulations model systems in which each unknown interacts with only a small subset of the full problem. This structure appears in partial differential equation (PDE) discretizations based on finite difference (FD) [80], finite volume (FV) [79], and finite element (FE) [170] methods, as well as in graph- and mesh-based computations. The resulting operators are large but sparse, so scientific codes store them in compressed formats and process them through sparse linear-algebra kernels. This section reviews the main sparse kernels that appear throughout modern scientific computing and later in this thesis.

2.2.1 Sparse Matrix–Vector Multiplication

Sparse Matrix–Vector Multiplication (SpMV) is the most common example and often the dominant kernel in sparse scientific solvers [15]. Krylov methods such as Conjugate Gradient (CG) [56], Generalized Minimal Residual (GMRES) [124], and Bi-Conjugate Gradient Stabilized (BiCGStab) [152] repeatedly apply a sparse operator to a dense vector. Preconditioners and multigrid methods also rely on SpMV-like sparse operator applications during smoothing, residual computation, and coarse-grid correction [15]. Figures 2.4 and 2.5 show the CSR implementation of SpMV: as mentioned in Section 2.1.5, the program scans the nonzeros in each sparse row, uses their column indices to perform indirect lookups into the dense vector, and accumulates the scaled values into one output element. This makes SpMV a canonical scan-and-lookup kernel with low arithmetic intensity and irregular memory accesses.

2.2.2 Sparse Matrix–Sparse Matrix Multiplication

Sparse Matrix–Sparse Matrix Multiplication (SpMSPM), also known as sparse general matrix–matrix multiplication (SpGEMM), is important when scientific codes construct, compose, or transform sparse operators [44]. For example, algebraic multigrid uses sparse matrix products to build coarse-grid operators, and graph or mesh algorithms use matrix products to compose sparse connectivity. Gustavson-style SpMSPM follows the same scan-and-lookup principle as SpMV, but each nonzero in the left matrix looks up an entire sparse row of the right matrix rather than a single dense-vector element. As shown in Figure 2.6, these looked-up rows are accumulated into a temporary output row before the final sparse row is

```

1 void spmv(row_ptrs: mref<? x idx>, // adj matrix CSR pointers
2          col_idxs: mref<? x idx>, // adj matrix CSR indices
3          nnz_vals: mref<? x f32>, // matrix nonzero values
4          vec_vals: mref<? x f32>, // dense vector values
5          out_vals: mref<? x f32>){ // output tensor
6     // Go through all rows
7     for (idx row_idx=0; row_idx<n_rows; row_idx++){
8         idx ptr_beg = row_ptrs[row_idx];
9         idx ptr_end = row_ptrs[row_idx+1];
10        // Go through all nonzeros in a row
11        for (idx ptr=ptr_beg; ptr<ptr_end; ptr++){
12            f32 nnz_val = nnz_vals[ptr];
13            idx col_idx = col_idxs[ptr];
14            f32 vec_val = vec_vals[col_idx];
15            out_vals[row_idx] += nnz_val * vec_val; }}}

```

Figure 2.5: Pseudocode for the SpMV operation in Figure 2.4. The outer loop iterates over CSR rows using `row_ptrs`; the inner loop scans each row’s nonzeros, loads the corresponding nonzero value and column index, and performs an indirect lookup into `vec_vals`. The final statement accumulates the scaled dense-vector value into the output row. In the end, SpMV exposes data-dependent loop bounds and indirect load operations, which general-purpose CPUs can vectorize using gather operations.

emitted. SpMSpM therefore combines irregular row lookups with intermediate accumulation and higher reuse than SpMV.

2.2.3 Sparse Matrix Addition

Sparse Matrix Addition (SpAdd) is important whenever scientific applications assemble, update, or combine sparse operators. Examples include accumulating terms from different physical processes, updating matrices during nonlinear solves, and combining sparse operators in preconditioners or multigrid setup. Unlike SpMV and SpMSpM, SpAdd does not look up dense operands. Instead, it co-iterates the compressed rows of two sparse matrices and emits the union of their coordinates. This is the disjunctive merge pattern introduced in Section 2.1.4; the corresponding CSR pseudocode is shown in Figure 2.3a. As a result, SpAdd is representative of sparse coordinate merging and the data-dependent control flow induced by sparse-sparse operations. A common generalization of SpAdd is SpKAdd, which merges and adds K sparse matrices simultaneously.

```

1 void spmspm(row_ptrs_A: mref<? x idx>,
2             col_idxes_A: mref<? x idx>,
3             vals_A: mref<? x f32>,
4             row_ptrs_B: mref<? x idx>,
5             col_idxes_B: mref<? x idx>,
6             vals_B: mref<? x f32>,
7             out: CSR) {
8     for (idx i=0; i<n_rows; i++) {
9         clear(touched);
10        for (idx p=row_ptrs_A[i]; p<row_ptrs_A[i+1]; p++) {
11            idx k = col_idxes_A[p];
12            f32 a = vals_A[p];
13            for (idx q=row_ptrs_B[k]; q<row_ptrs_B[k+1]; q++) {
14                idx j = col_idxes_B[q];
15                if (!mark[j]) {
16                    mark[j] = true;
17                    touched.push(j);
18                    accum[j] = 0;
19                }
20                accum[j] += a * vals_B[q];
21            }
22        }
23        sort(touched); // Optional: keep CSR columns sorted
24        for (idx j : touched) {
25            emit(out, i, j, accum[j]);
26            mark[j] = false;
27        }
28    }
29 }

```

Figure 2.6: Pseudocode for Gustavson-style SpMSpM. The outer loop builds one output row at a time. For each nonzero A_{ik} , the middle loop uses k to look up row k of B , and the inner loop scans that row. Products are accumulated in a row-local sparse accumulator: `mark` identifies whether a column has already been touched, and `touched` records the nonzero columns of the output row. The output row can therefore be emitted by iterating only over `touched`, avoiding a scan of the full dense accumulator. Compared with SpMV, SpMSpM performs scan-and-lookup over whole sparse rows and introduces accumulation of intermediate results.

2.2.4 Other Sparse Linear and Tensor Kernels

Several other sparse linear- and tensor-algebra kernels are central to scientific computing and graph analytics, and they build on the same traversal, lookup, merging, and accumulation patterns introduced above.

PageRank (PR) is a classic graph-analysis kernel used to estimate the importance of vertices in a graph [16]. It appears in web ranking, graph mining, and network-science pipelines where the input is naturally represented as a sparse adjacency matrix. Each iteration prop-

agates rank values through the graph structure, repeatedly applying a sparse graph operator to a dense rank vector until convergence. As a result, PR has a structure similar to SpMV, but with application-specific vertex updates and repeated fixed-point iterations.

Triangle Counting (TC) measures the number of triangles in a graph, a key primitive for understanding clustering, transitivity, and community structure. It is widely used in graph analytics to characterize social, biological, and communication networks, where triangles capture local connectivity patterns. In GraphBLAS-style formulations, TC is commonly implemented with masked SpMSpM or with sparse row intersections [23, 95]. TC is relevant here because it stresses merge-heavy graph traversal and sparse-sparse matching rather than only sparse-dense lookup.

Matricized Tensor Times Khatri–Rao Product (MTTKRP) is a core kernel in sparse tensor decomposition. Sparse tensors arise in multiway scientific data, such as time-varying measurements, network events, text corpora, and spatial observations. MTTKRP scans sparse tensor nonzeros and combines them with dense factor matrices, making it one of the dominant costs in many tensor-factorization algorithms [118]. MTTKRP is relevant here because it extends scan-and-lookup from sparse matrices to higher-dimensional sparse tensors while introducing dense computation for each visited nonzero.

Canonical Polyadic Alternating Least Squares (CP-ALS) is a widely used method for decomposing sparse tensors into low-rank factors [118]. These decompositions are used to compress sparse multiway data, extract latent structure, and support downstream analysis. CP-ALS repeatedly invokes MTTKRP-like operations and updates dense factor matrices, making it representative of end-to-end sparse tensor applications rather than a single isolated kernel. It is relevant here because it combines sparse tensor traversal with dense linear-algebra updates and repeated optimization steps.

Sparse Tensor Contraction (SpTC) generalizes matrix multiplication to higher-dimensional sparse tensors [85]. Tensor contractions appear in multilinear algebra, tensor-network methods, and high-dimensional data analysis, where they combine tensor modes according to index relationships. SpTC combines compressed sparse fibers across multiple tensor modes and may require matching coordinates across several dimensions. It is relevant here because it stresses hierarchical traversal and sparse-sparse matching in compressed tensor formats such as CSF.

2.3 Sparsity in Machine Learning

Sparsity also appears in important classes of modern machine learning workloads, especially those that operate over categorical features, graph neighborhoods, or sparse attention pat-

terns. These workloads often use dense embedding vectors stored in large embedding tables, but the computation is sparse because each input activates only a small subset of possible entries [102, 117, 67, 164, 58]. This section focuses on three workload families—graph learning models, recommendation models, and sparse language models—that all rely on sparse indexing and lookup into dense embeddings, but differ in their aggregation patterns and locality properties.

2.3.1 Graph Learning Models

A graph is formally defined as a set of nodes (vertices) and a set of edges that encode relationships between pairs of nodes. For example, in a financial network, nodes may represent economic entities such as individuals or institutions, while edges denote transactions between them. Nodes can include features such as account age, geographic region, and risk score, while edges may include features such as transaction amount, timestamp, device ID, or location.

A bank can detect fraudulent activities by analyzing the features of the nodes or edges of such a graph. Graph Neural Networks (GNNs), for instance, can infer the probability of fraud for each node by aggregating information from neighboring nodes—an operation known as *graph convolution*—and processing it through neural network layers [52, 153].

However, categorical features such as location or device ID cannot be directly processed by neural networks. These categories are represented as integer IDs. Since these IDs are merely indices without intrinsic numerical or relational meaning, they cannot be effectively handled using traditional machine learning techniques, which are designed for continuous numerical inputs. Graph machine learning models address this limitation by embedding structural and relational information into continuous *embedding vectors* [117].

As shown in Figures 2.7 and 2.8, embedding vectors are stored in embedding tables indexed by the IDs of their corresponding vertices or edges. To perform embedding operations such as graph convolution, GNNs scan the nonzeros that identify a node's neighbors, look up the corresponding embedding vectors, and aggregate them with, for instance, an element-wise sum. This is the machine-learning analogue of the SpMV scan-and-lookup pattern, except that each sparse coordinate fetches a dense vector rather than a scalar.

Several other graph learning models, including Message Passing (MP) models [120] and Knowledge Graphs (KGs) [19], leverage the same principles. MP models are a Sampled-Dense-Dense Matrix Multiplication (SDDMM) fused with an SpMM kernel [120]. KGs are SpMM functions that use semirings—general algebraic structures with addition and multiplication—with only one nonzero per row, requiring no segment pointers or lengths and no rescaling.

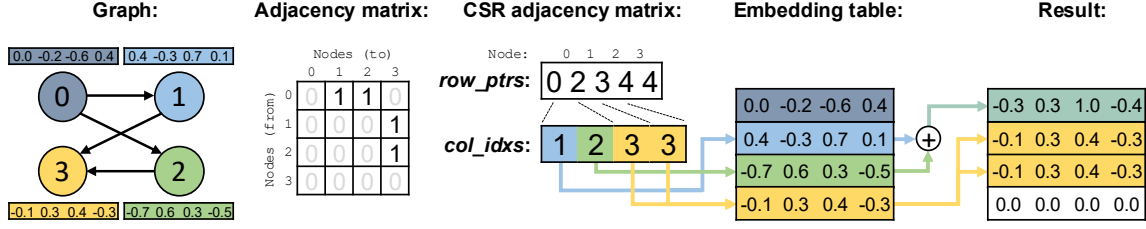


Figure 2.7: Graph convolution in GNNs implemented as Sparse Matrix–Dense Matrix Multiplication (SpMM) [6, 131]. The graph adjacency matrix is stored in CSR format [15]; each nonzero identifies a neighboring node whose embedding vector is looked up, optionally scaled by the nonzero value, and accumulated into the output embedding of the current node. Figure 2.8 shows the corresponding pseudocode. The scaling step is omitted here for visual clarity. In the end, graph convolution follows the same scan-and-lookup structure as SpMV, but each sparse coordinate triggers an indirect dense-vector lookup rather than an indirect scalar lookup.

```

1 void spmm(row_ptrs: mref<? x idx>, // adj matrix CSR pointers
2          col_idxs: mref<? x idx>, // adj matrix CSR indices
3          nnz_vals: mref<? x f32>, // matrix nonzero values
4          emb_vals: mref<? x ? x f32>, // embedding table
5          out_vals: mref<? x ? x f32>){ // output tensor
6   // Go through all rows
7   for (idx mtz_row_idx=0; mtz_row_idx<n_rows; mtz_row_idx++){
8     idx mtz_ptr_beg = row_ptrs[mtz_row_idx];
9     idx mtz_ptr_end = row_ptrs[mtz_row_idx+1];
10    // Go through all nonzeros in a row
11    for (idx mtz_ptr=mtz_ptr_beg; mtz_ptr<mtz_ptr_end; mtz_ptr++){
12      f32 mtz_nnz_val = nnz_vals[mtz_ptr];
13      idx mtz_col_idx = col_idxs[mtz_ptr];
14      // Go through all elements in embedding vector
15      for (idx emb_col_idx=0; emb_col_idx<emb_len; emb_col_idx++){
16        f32 emb_val = emb_vals[mtz_col_idx,emb_col_idx];
17        out_vals[mtz_row_idx,emb_col_idx] += mtz_nnz_val * emb_val; }}}

```

Figure 2.8: Pseudocode for the SpMM-style graph convolution in Figure 2.7. The outer loop iterates over graph nodes, the middle loop scans the nonzeros that identify each node’s neighbors, and the inner loop traverses the embedding vector fetched for each neighbor. Compared with SpMV, the additional inner loop introduces more control-flow operations and a more regular structure that is generally leveraged for vectorization.

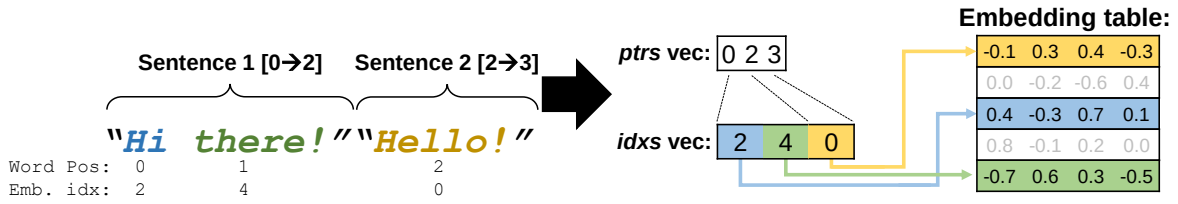


Figure 2.9: Embedding lookup in language models. Words or subwords in a batch of sentences are tokenized into integer IDs, stored in a CSR-like representation for the batch, and used to gather dense vectors from an embedding table. Unlike graph convolution in GNNs, this operation performs traversal and lookup without reducing multiple looked-up vectors into an output vector.

2.3.2 Deep Learning Recommendation Models (DLRMs)

DLRMs recommend products to users by processing a large set of categorical features under strict latency constraints [50, 61, 49]. They currently account for a large fraction of inference cycles in production data centers, with embedding operations consuming a significant portion of runtime [50, 49].

Categorical features in DLRMs are generally multi-hot, meaning they can take on multiple values (e.g., movie genres or cast). When representing these relationships as graphs, DLRMs can be viewed as a special category of graph learning models. At inference time, the embedding vectors corresponding to these values must be looked up and aggregated via operations such as an element-wise sum. High-performance implementations (1) fuse lookup and aggregation to avoid materializing intermediate results and (2) batch the categories of multiple queries to improve throughput [98]. This operation is similar to graph convolution, although the pooling operation may differ from an element-wise sum. The `nn.EmbeddingBag` (EB) in PyTorch [114] is one such example. Unlike graph learning models, DLRMs have higher spatial locality, as discussed in Section 4.2.2.

2.3.3 Sparse Attention in Language Models

Large Language Models (LLMs) embed text into vectors [34]. Figure 2.9 shows how categorical features are tokenized into integer IDs that index into an embedding table. Transformers such as Google's BigBird [164] sparsify the attention mechanism to compare only a selected subset of input tokens, improving memory footprint and performance on long sequences. Although this technique drastically reduces inference latency, it requires an additional step to gather embedding blocks. This gathering step is generally not fused with downstream deep neural network computation [164], requiring only embedding lookups with no additional

compute. This is similar to a `tf.gather` [1], although it replicates blocks of embeddings from the key tensor into the query tensor.

2.4 Summary

Overall, sparse tensor workloads in both scientific computing and machine learning repeatedly exercise the same core mechanisms: compressed tensor traversal, sparse coordinate merging, and scan-and-lookup into dense operands. These mechanisms reduce storage and computation, but they also introduce irregular memory accesses, data-dependent control flow, and limited locality, which become major bottlenecks on traditional architectures. Chapter 3 reviews prior work on analyzing and optimizing these operations, and identifies the limitations that remain. The rest of this thesis then shows how we address them: Chapter 4 quantifies the architectural requirements of sparse tensor algebra, motivating the DAE hardware–software co-design developed in Chapters 5 and 6.

Chapter 3

Related Work

As sparse kernels become pervasive, a large body of work has emerged on their performance analysis and optimization, spanning both hardware and software techniques. This chapter positions the contributions of this thesis with respect to three gaps in the literature. Section 3.1 first reviews characterization studies, which have focused primarily on individual kernels such as SpMV, leaving broader sparse tensor workloads and machine learning embedding operations less thoroughly understood. Section 3.2 then surveys architectural approaches and shows that existing designs tend to trade off performance against flexibility: incremental CPU optimizations preserve programmability and offer the most flexibility, but deliver limited speedups; custom accelerators can provide high performance, but only for narrow kernels, or remain constrained by accelerator limitations such as memory capacity, which is especially important for sparse workloads. Existing DAE-style systems combine the flexibility of general-purpose processors with the performance of specialized access units, but they are not designed around SIMD/vector computation, limiting their potential. Finally, Section 3.3 discusses compiler support for DAE architectures, where the programming model remains challenging and existing techniques do not automatically generate optimized DAE code for the sparse-dense operations that arise in modern machine learning workloads.

3.1 Sparse Workload Characterization

Because of its unique characteristics, sparse computation has been widely studied. However, existing performance analyses mostly focus on SpMV kernels. Less attention has been dedicated to sparse kernels with more complex traversal and merging behavior, or to sparse machine learning workloads with different locality properties.

3.1.1 SpMV Characterization

SpMV is one of the most critical kernels in scientific computing, and its performance has been extensively studied, particularly on CPUs. Mantovani et al. [92] evaluate thread- and node-level scalability on the MareNostrum4 supercomputer and ThunderX2-based systems, comparing both performance and energy consumption across microarchitectural choices such as vector unit width and cache hierarchy design. Armejach et al. [10] analyze a range of HPC processors employing different combinations of in-order and out-of-order cores as well as varying last-level cache capacities per core, concluding that scaling last-level caches yields diminishing returns while substantially increasing area and power. Oliveira et al. [109] compare HPC processors using metrics including GFLOPS per Watt, GFLOPS versus memory bandwidth, and bytes per FLOP.

Other studies focus on the data movement and format choices that shape SpMV performance. Bean et al. [17] characterize data movement in memory-intensive benchmarks using metrics such as MPKI, last-to-first-level cache ratios, and operational intensity. Asgari et al. [11] discuss the performance implications of different sparse matrix representations, while Alappat et al. [7] evaluate SpMV and lattice quantum chromodynamics kernels on the A64FX platform, comparing the impact of multiple formats. As GPUs have become increasingly prominent in HPC, several authors have also investigated SpMV performance on them [18]. More recently, Nguyen et al. [103] analyze SpMV performance on off-the-shelf CPUs, GPUs, and FPGAs, demonstrating that indirect lookups into the dense vector remain a dominant bottleneck across all three architectures.

Together, these studies establish SpMV as a memory-bound sparse-dense kernel whose performance is limited by irregular access to the dense input vector. However, they do not fully characterize the broader set of traversal, merging, and lookup patterns that appear in sparse tensor algebra and sparse machine learning workloads.

3.1.2 Beyond SpMV

Beyond SpMV, mature sparse-library ecosystems and algorithmic studies such as SuiteSparse, GraphBLAS, and high-performance SpGEMM implementations provide optimized support for more complex sparse kernels [33, 23, 100]. However, comparatively fewer studies characterize the microarchitectural behavior of these kernels across broad sparse tensor and machine learning workloads, especially the traversal, merging, and intermediate-data-structure behavior that distinguishes them from SpMV. Gao et al. [44] survey SpGEMM algorithms across applications, sparse formats, accumulation strategies, target architectures, and programming models, and show that performance depends strongly on the interaction

between sparsity structure, intermediate-result management, load balance, and hardware resources. Their main conclusion is that no single SpGEMM strategy dominates across inputs and architectures, motivating more adaptive and architecture-aware sparse-kernel implementations. However, because their work focuses on SpGEMM algorithms and implementations, it does not characterize the underlying microarchitectural bottlenecks across broader sparse tensor and sparse machine learning workloads.

3.1.3 Embedding Operation Characterization

With the rise of sparsity and feature embeddings in machine learning, several authors have begun examining their performance. Gupta et al. [50, 49] show that the embedding bag function in DLRMs suffers from memory bottlenecks analogous to scientific kernels, despite having different spatio-temporal locality characteristics. Zaheer et al. [164] highlight the inefficiency of loading sparse attention blocks in LLMs on conventional architectures. However, none of these studies analyze performance at the microarchitectural level, nor do they characterize these workloads in ways that provide actionable guidance for designing more efficient architectures.

3.1.4 Our Performance Characterization

To overcome these limitations, in Chapter 4 we characterize sparsity across both scientific and machine learning workloads. In our study, we analyze a broad suite of sparse tensor algebra kernels, ranging from traditional scientific computing kernels such as SpMV, SpGEMM, and SpADD to embedding operations from different machine learning workloads. First, we study their performance on off-the-shelf CPU and GPU architectures, measuring the impact of sparse tensor traversal and merging. We then model the resource requirements of these operations to highlight the scaling limitations of conventional solutions.

3.2 Architectures Optimized for Sparse Computation

Because of their importance, several authors have proposed architectures to accelerate sparse workloads. These architectures fall into four broad categories: incremental optimizations to conventional processors, fixed-function sparse accelerators, flexible sparse tensor accelerators, and DAE-style systems that fully decouple access from execution. These designs make different trade-offs among performance, flexibility, vector integration, programmability, and data movement. In contrast, this thesis targets a DAE design that combines efficient sparse traversal and merging with general-purpose vector computation.

3.2.1 CPU Optimizations for Irregular Workloads

Prefetching is widely used to improve memory latency and throughput in dense workloads, where future accesses can often be inferred from recent history. However, simple prefetchers, such as stride prefetchers, are much less effective for sparse workloads dominated by indirect lookups [163, 5, 148]. Programmable prefetchers address this limitation by allowing software to describe specific access patterns to hardware. IMP targets simple indirections, such as the inner loop of SpMV [163], while domain-specific prefetchers support limited forms of multi-level indirection, such as those found in graph algorithms [5, 148]. However, these mechanisms still move data into the cache hierarchy and must be timed carefully to be effective. They also duplicate address-generation work: once to issue the prefetch and again when the core later executes the load.

Stream Semantic Registers reduce this overhead by delivering fetched data directly to the core through streams, avoiding part of the redundant address-generation and cache-management cost [129, 130, 128, 127, 91]. This approach can accelerate scalar scan-and-lookup patterns and sparse merging, but current implementations still require close coordination with the core for more complex sparse kernels, limiting the achievable speedup.

3.2.2 Fixed-Function Architectures for Sparse Tensor Algebra

At the opposite end of the design spectrum lie fully custom architectures for sparse tensor algebra. Several architectures have been proposed to accelerate SpMV-like workloads, including FPGA-based designs and evaluations for sparse linear algebra [42, 48, 151, 113, 134, 103], fixed-function architectures [104, 168, 125], and processing-in-memory systems [45]. Designs such as GAMMA [165] and related systems [145, 111, 168, 83, 119, 112, 167] implement fixed-function units for sparse matrix–matrix multiplication, a fundamental kernel to which many tensor algebra operations can be reduced. Sparse machine learning has similarly motivated specialized GPU kernels and graph-convolution accelerators [43, 81]. These solutions offer high performance, but their specialization limits flexibility, restricts support for kernel fusion, and often requires substantial data movement between the accelerator and the rest of the system [20].

3.2.3 Flexible Architectures for Sparse Tensor Algebra

More flexible architectures, such as Capstan [123], Onyx [72], Opal [28], and others [32, 53, 12, 60, 96, 105], can be programmed to execute a wider range of Einsum expressions [57]. Despite their broader applicability, computation on specialized hardware remains constrained

by design choices such as a restricted set of operations, data types, and memory capacity. Moreover, using such accelerators typically requires moving tensors into and out of accelerator memory before and after every computation. In real-world applications such as CP-ALS tensor decomposition [118], where intermediate results must be evaluated at each iteration, this overhead can outweigh the benefits.

3.2.4 DAE Architectures and Access Units

To address these limitations, DAE architectures combine specialized access units with general-purpose compute cores. Maple [110] uses specialized access units to accelerate indirect memory accesses for a scalar core. Similarly, HATS [99] and SpZip [161] employ dedicated access units to accelerate graph traversals: HATS exploits graph locality to reduce data movement, while SpZip applies compression and decompression techniques to lower memory traffic. Smash uses a custom engine to decompress sparse matrices stored in a custom compressed format [66]. Cambricon-S uses a specialized engine to efficiently load weights of sparse neural networks [169]. Although these systems combine the flexibility of general-purpose processors with the performance of specialized access units, they primarily target specific access patterns and are restricted to scalar cores rather than modern vectorized architectures. They therefore do not provide broad acceleration of sparse traversal and merging while preserving the performance of vector computation.

3.2.5 Production DAE-Like Architectures

DAE-like designs that separate memory access from computation are becoming increasingly prevalent among hyperscalers. Meta MTIAs [41] and Google TPUs [65] both adopt such an approach to accelerate embedding lookups in recommendation models. Recent Nvidia GPUs [107, 106] similarly include specialized access units, Tensor Memory Accelerators (TMAs), for fetching dense tensor tiles. These designs demonstrate the value of separating access from execution for vector lookups, embedding operations, and dense tile movement. However, they do not target the broader combination of sparse traversal, sparse coordinate merging, and vectorized dense computation required by the sparse tensor workloads studied in this thesis.

3.2.6 Our Hardware Architecture: The TMU

We designed the TMU to close these gaps. As discussed in Chapter 5, a multicore DAE processor composed of TMU–CPU cores provides a combination of efficiency and generality

Table 3.1: Qualitative comparison of architectural approaches for sparse tensor acceleration. Rows group prior techniques by their main abstraction, and columns list capabilities needed to accelerate sparse tensor algebra while preserving general-purpose and SIMD-friendly computation. Checkmarks indicate that a class of designs provides the capability in hardware or through its exposed execution model. No prior class combines irregular indirection, sparse merging, high Memory-Level Parallelism (MLP), full SIMD/vector capability, versatility, and the benefits of a CPU memory hierarchy in the same way as the TMU–CPU DAE processor.

Approach	Single-level indirection	Multi-level indirection	Sparse merging	High MLP	Full SIMD	Workload versatility	Cache locality	Large memory
Programmable prefetchers [163]	✓					✓	✓	✓
Domain-specific prefetchers [5, 148]	✓	✓				✓	✓	✓
Stream semantic [129, 130]	✓		✓	✓		✓	✓	✓
Fixed accelerators [165, 145, 111, 168]	✓	✓	✓	✓	✓			
Reconfigurable accelerators [123, 32, 53]	✓	✓	✓	✓	✓	✓		
DAE engines [110, 99, 161]	✓	✓		✓		✓	✓	✓
TMU–CPU DAE processor	✓	✓	✓	✓	✓	✓	✓	✓

not offered by prior designs: the TMU offloads sparse traversal, merging, and marshaling, while the CPU core retains the flexibility and throughput of modern vector execution. Table 3.1 summarizes this positioning by comparing the capabilities provided by the main classes of sparse-acceleration architectures.

3.3 DAE Compilation

One of the primary barriers to widespread adoption of DAE architectures is programmability. In scientific computing, this complexity can often be hidden behind kernel libraries, making hand-written and hand-optimized DAE implementations feasible for a relatively small set of

routines. Machine learning workloads are different: embedding operations appear in many variants, formats, reductions, and fusion contexts, making a library-only approach difficult to scale. Accordingly, this section first reviews how machine learning compiler stacks generate and optimize model code, then discusses tensor compilers as the closest abstraction for sparse tensor programs, and finally explains why prior DAE compiler techniques do not provide the structured access/execute representation needed by Ember.

3.3.1 Machine Learning Compiler Stacks

Traditional machine learning compilers generate machine code from a semantic representation of the model [1, 114, 153, 22, 39, 122, 150]. As in the Meta MTIA ecosystem [41], models are decomposed and lowered to handwritten kernels optimized for the target architecture. While effective, this strategy does not scale to the wide variety of embedding operations, their algorithmic complexity, or the combinatorial explosion of input formats, reduction operators, precisions, and algorithmic optimizations such as operator fusion.

3.3.2 Tensor Compilers

Tensor compilers address this challenge by progressively lowering machine learning models through a sequence of IRs until code generation. For example, through MLIR [76], PyTorch programs can be lowered via IRs such as Linalg (i.e., linear algebra kernels like matrix multiplication), SCF (i.e., structured imperative code composed of loops, if-then-else constructs, and memory/compute instructions), and LLVM (i.e., unstructured low-level imperative code composed of branching instructions and memory/compute operations). This enables optimizations at multiple abstraction levels. High-level IRs (e.g., Linalg [76], index notations [70, 21], or Einsums [1]) describe what to compute, whereas lower-level IRs describe how to compute it. Although high-level tensor IRs support powerful algorithmic optimizations such as operation fusion [1, 114, 153, 22, 39, 122], their level of abstraction makes them unsuitable for architectural optimizations, including those required for DAE systems. Consequently, most architectural optimizations occur in lower-level IRs (e.g., SCF or LLVM IR [75]) after high-level algorithmic transformations have already been applied. While tensor compilers have been successfully used for CPUs [70, 21, 4, 13, 73], GPUs [132, 162, 86], FPGAs [86], and distributed systems [160], compilation for tensor DAE systems remains largely unexplored. Lopoukhine et al. [88] recently showed the potential of multi-level compilation flows for ISA extensions such as streaming registers, but this technique does not directly apply to complex DAE systems like a TMU–CPU core, where both indexing and memory accesses are fully offloaded to the access unit.

3.3.3 Compiler Support for DAE Architectures

Earlier work on DAE compilation [51], predating modern tensor compilers, explored generating DAE code from LLVM IR [75]. However, LLVM IR is unstructured and represents loops using conditional branches, making it ill-suited for analyzing the deep loop hierarchies that arise in embedding operations [137, 88]. For example, identifying loop-offloading candidates (Section 6.2.2) becomes impractical in such representations. High-level synthesis tools address similar challenges using pragmas [30], but these require user intervention and are incompatible with our automated compilation flow. For affine loops, Jimborean et al. [63] use polyhedral analysis to overcome these limitations, but this approach cannot be applied to embedding operations. Moreover, LLVM IR is not designed to represent DAE programs: it cannot express separate access and compute stages, nor can it model data (de)serialization. As a result, optimizations cannot be applied iteratively and must instead be embedded directly into the lowering and code-generation process, which is impractical for sophisticated transformations.

3.3.4 Our Compiler: Ember

To overcome these limitations, in Chapter 6 we introduce Ember, a generic compiler that lowers sparse machine learning operations to optimized DAE code. Ember features custom IRs for DAE-specific optimizations and is implemented in MLIR, allowing it to integrate with PyTorch through torch-mlir and with TensorFlow through its native MLIR compiler stack. This integration enables users to exploit the TMU without manually writing DAE programs.

3.4 Summary

The literature demonstrates that sparse workloads are important, difficult to accelerate, and increasingly relevant across both scientific computing and machine learning. However, three gaps remain. Existing characterization studies do not provide a unified view of the traversal, merging, and lookup bottlenecks that span these domains. Existing sparse architectures either preserve flexibility with limited speedups or achieve high performance through specialization that constrains programmability, vector integration, or data movement. Finally, existing compiler techniques do not automatically generate optimized DAE code for the diverse sparse-dense embedding operations found in modern machine learning models. This thesis addresses these gaps through a cross-domain characterization, the TMU architecture, and the Ember compiler.

Chapter 4

Architectural Implications of Sparse Tensor Workloads

This chapter derives the architectural requirements that motivate the TMU–CPU system developed in the rest of the thesis. We study two complementary classes of sparse tensor workloads. Scientific sparse tensor kernels expose the diversity of sparse tensor algebra: they combine sparse traversal, scan-and-lookup operations, coordinate merging, and computation on partial results. These workloads show that general-purpose processors already provide several capabilities that sparse tensor algebra needs, including large memory capacity, caches that can exploit locality, vector units for dense arithmetic, and flexible general-purpose execution. However, they are inefficient at the irregular traversal, merging, and lookup operations that account for most execution cycles of these workloads. Machine learning embedding workloads isolate the most important of these bottlenecks: irregular lookup and accumulation of dense embedding vectors. Although these workloads are less diverse in their sparse operations, they expose a wide range of spatial and temporal locality because they access vectors of different sizes with different reuse patterns. This makes them useful for quantifying how locality affects cache filtering, memory-level parallelism, and request-tracking requirements. Together, these two studies show that traditional architectures, which execute traversal, merging, lookup, and computation on the same compute cores, cannot scale efficiently to the requirements of sparse tensor algebra. They motivate decoupling sparse traversal and merging from computation, which is the design principle implemented by the TMU.

4.1 Sparse Scientific Computing

Scientific sparse kernels provide the broadest view of the architectural requirements of sparse tensor algebra. They exercise the main sparse operations discussed in Section 2.1: traversal to recover compressed coordinates and values, scan-and-lookup to access dense or sparse operands indirectly, merging to co-iterate sparse operands, and computation over the resulting partial results. This makes them a useful starting point for separating what general-purpose processors already do well from what they handle inefficiently.

4.1.1 Scientific Workload Methodology

To broadly understand the architectural implications of sparsity, we study three representative kernels across a large and diverse set of inputs on two processors with very different architectural characteristics.

We analyze the three main kernels introduced in Section 2.2, each chosen to isolate a distinct architectural stressor:

- *Sparse Matrix–Vector Multiplication* (SpMV) [157], which multiplies a CSR matrix and a dense vector and outputs a dense vector ($Z_i = A_{ij}B_j$). Since co-iteration of $A_{*,j}$ and B_j is generally implemented as a memory-intensive scan-and-lookup operation (Section 2.1.5), we use SpMV as a proxy for sparse traversal and indirect scalar lookup.
- *Gustavson’s Sparse Matrix–Sparse Matrix Multiplication* (SpMSpM) [44], which multiplies and outputs CSR matrices ($Z_{ij} = A_{ik}B_{kj}$ with (ikj) schedule). SpMSpM performs the same scan-and-lookup operation as SpMV but, instead of scalars, it looks up and loads whole (sparse) rows of B , which are then reduced into a single row. For this reason, SpMSpM is a proxy for scan-and-lookup operations with higher spatial locality, as well as for the accumulation of intermediate results.
- *Sparse Matrix Addition* (SpAdd) [59], which adds and outputs CSR matrices ($Z_{ij} = A_{ij} + B_{ij}$) by repeatedly co-iterating and joining (disjunctive merge) matrix fibers with the same row index i . Therefore, SpAdd is representative of coordinate merging and the data-dependent control flow it induces.

We profile high-performance implementations of these kernels from Kokkos version 3.5 [26] on both an HPC processor, the Fujitsu A64FX [126], and a data-center processor, the AWS Graviton 3 [9, 8]. The main architectural characteristics of the two processors are summarized in Table 4.1. While the two processors provide similar peak compute throughput, the A64FX provides more than 3× the memory bandwidth, whereas the Graviton 3 provides

Table 4.1: Specifications of the CPUs used for the sparse scientific workload characterization [135, 126, 9].

	A64FX	Graviton 3
Manufacturer	Fujitsu	Amazon
Architecture	ARM v8.2	ARM v8.4
Core type	Fujitsu superscalar OoO HPC core	Arm Neoverse-V1 OoO server core
Design target	Dense scientific computation	General-purpose server workloads
Sockets	1	1
Cores per socket	48	64
Vector units	2×512 -bit SVE	2×256 -bit SVE
Peak FP64 performance	3.38 TFLOP/s	2.66 TFLOP/s
L1D, L1I per core	64 kB, 64 kB	64 kB, 64 kB
Private L2	N/A	64×1 MB
Shared LLC	4×8 MB	32 MB
Memory technology	HBM2	DDR5
Memory channels	32	8
Peak bandwidth	1024 GB/s	300 GB/s
Operating System	Red Hat Enterprise Linux 8.1	Ubuntu 22.04
Compiler	armclang 22.0.2	armclang 22.0.2

more cores and more aggressive out-of-order resources. To capture widely different input features, we test these kernels on 1000 of the largest SuiteSparse matrices, spanning diverse application domains [33].

4.1.2 CPU Performance Characterization

In this section, we show that neither off-the-shelf HPC processors nor data-center processors efficiently handle the irregular operations that dominate sparse scientific workloads. Although these designs provide different balances of memory bandwidth, vector throughput, cache capacity, and out-of-order execution resources, sparse traversal and merging still leave substantial compute and memory resources underutilized.

Figure 4.1 presents the Roofline models of these kernels and matrices on the two evaluated processors. Overall, all kernels and matrices are heavily memory bound on both processors, and performance generally increases with operational intensity. For both processors, SpMV has the lowest operational intensity and performance across kernels, whereas SpM-SpM has the highest due to partial-result accumulation, and SpAdd falls between the two. Across processors, the A64FX makes better use of caches and achieves higher operational intensity for all kernels, but it does not necessarily achieve higher performance than the Graviton 3.

To better understand the reason, we measure the fraction of frontend and backend stalls on each processor, reported in Figure 4.2 and summarized in Table 4.3. Frontend stalls arise

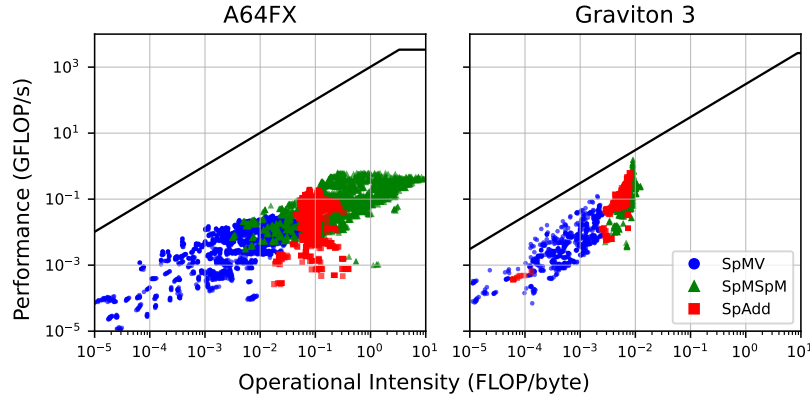


Figure 4.1: Roofline model of sparse scientific kernels on 1000 of the largest SuiteSparse matrices, spanning diverse application domains, evaluated with Kokkos on A64FX (left) and Graviton 3 (right). The black lines are the processor-specific Roofline ceilings from peak compute throughput and peak memory bandwidth. The x-axis reports operational intensity, computed as floating-point operations per byte transferred from main memory by the measured execution (FLOP/byte), and the y-axis reports achieved performance (GFLOP/s); both axes use logarithmic scales. For sparse kernels, the byte count includes accesses to compressed-format metadata, nonzero values, dense operands, intermediate data structures, and output data. Each point is one matrix; blue, green, and red points correspond to SpMV, SpMSpM, and SpAdd, respectively. Overall, performance increases with operational intensity across kernels and inputs. However, despite achieving higher cache utilization, the A64FX does not consistently outperform the Graviton 3.

from inefficiencies in the instruction pipeline, whereas backend stalls originate from inefficiencies in the compute units and memory subsystem. For sparse kernels, frontend stalls are primarily caused by branch mispredictions in data-dependent loop branches. Backend stalls are largely due to irregular memory accesses, which generate long-latency memory operations that block the compute pipeline [55, 54, 148, 149]. This leads to three observations:

- For SpMV, unlike the Graviton 3, the A64FX exhibits substantially more backend stalls than frontend stalls (63% vs. 33% on average). We attribute this behavior to the A64FX being primarily optimized for regular high-throughput computation and therefore being less resilient to irregular memory accesses. Its relatively small cores provide limited instruction-level and memory-level parallelism for tolerating long memory latencies [38]. In contrast, the Graviton 3 exhibits a larger fraction of frontend stalls (58% frontend vs. 28% backend on average), likely because its more aggressive cores sustain higher memory-level parallelism while incurring higher branch misprediction penalties.

Table 4.2: Minimum, average, and maximum resource utilization for the sparse scientific kernels. Values are percentages of each processor’s peak compute throughput and peak memory bandwidth. Each subtable reports one kernel, rows correspond to A64FX and Graviton 3, and columns summarize the distribution across 1000 of the largest SuiteSparse matrices, spanning diverse application domains. Compute utilization is extremely low for all kernels: the maximum is $1.8\text{e-}2\%$ on A64FX and $5.7\text{e-}2\%$ on Graviton 3, both for SpMSpM. Bandwidth utilization is higher and more variable: averages range from 0.046% – 0.28% on A64FX and 2.3% – 9.8% on Graviton 3, with maxima of 1.7% and 56% , respectively. Thus, sparse scientific kernels use little of the available compute throughput, while memory-bandwidth utilization depends strongly on both architecture and input matrix.

(a) SpMV

	Compute			Bandwidth		
	Min	Avg	Max	Min	Avg	Max
A64FX	3.2e-7	9.0e-5	9.5e-4	1.2e-3	2.8e-1	1.7
Graviton 3	1.1e-6	2.5e-4	4.6e-3	1.8e-1	2.3	2.1e1

(b) SpMSpM

	Compute			Bandwidth		
	Min	Avg	Max	Min	Avg	Max
A64FX	2.9e-5	4.9e-3	1.8e-2	7.4e-5	7.5e-2	4.0e-1
Graviton 3	2.4e-10	9.8e-3	5.7e-2	2.1e-2	9.8	5.6e1

(c) SpAdd

	Compute			Bandwidth		
	Min	Avg	Max	Min	Avg	Max
A64FX	5.9e-9	1.5e-3	5.3e-3	1.4e-4	4.6e-2	2.2e-1
Graviton 3	5.0e-8	4.5e-3	2.2e-2	5.8e-1	5.8	2.5e1

- For SpMSpM, the A64FX achieves better backend utilization than in SpMV, with average backend stalls falling from 63% to 21%. Conversely, on the Graviton 3, SpM-SpM exhibits backend utilization comparable to SpMV, with average backend stalls of 38% and 28%, respectively. These results suggest that (i) scan-and-lookup operations with higher spatial locality benefit significantly from high-bandwidth architectures, and (ii) reduction operations do not severely impact backend utilization because partial results typically fit in cache [100]. However, computing these reductions benefits from stronger frontend capabilities and more aggressive speculation, such as those provided by the Graviton 3 cores.
- For SpAdd, the frontend becomes the dominant bottleneck due to highly data-dependent branching behavior, particularly on cores with limited out-of-order execution capabil-

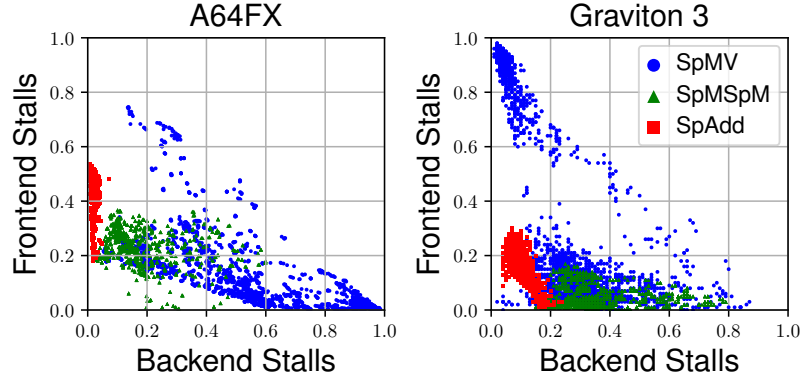


Figure 4.2: Frontend and backend stalls for the sparse scientific kernels on 1000 of the largest SuiteSparse matrices, spanning diverse application domains, evaluated with Kokkos on A64FX (left) and Graviton 3 (right). Stall cycles are normalized to total execution cycles. The x-axis reports the fraction of cycles with backend stalls, which capture delays in compute units and the memory hierarchy, and the y-axis reports the fraction of cycles with frontend stalls, which capture instruction-pipeline inefficiencies such as branch mispredictions. These two quantities are measured independently and are not mutually exclusive: a cycle may contain both frontend and backend stalls, so the two fractions need not sum to one. Each point is one matrix; blue, green, and red points correspond to SpMV, SpMSpM, and SpAdd, respectively. Overall, both data-dependent control and irregular memory accesses generate large stall fractions, limiting performance.

Table 4.3: Minimum, average, and maximum frontend and backend stalls for the sparse scientific kernels in Figure 4.2. Values are percentages of total execution cycles across 1000 of the largest SuiteSparse matrices, spanning diverse application domains. Rows group results by processor and stall source, while columns summarize the distribution for SpMV, SpMSpM, and SpAdd. On A64FX, SpMV is backend-dominated on average (63% backend stalls), whereas SpAdd is frontend-dominated (42% frontend stalls), and SpMSpM is more balanced (28% frontend, 21% backend). On Graviton 3, SpMV shifts toward frontend stalls (58% on average), while SpMSpM has the highest average backend stalls (38%). The extrema are also large: frontend stalls reach 98% for SpMV on Graviton 3, and backend stalls reach 99% for SpMV on A64FX. Thus, sparse-kernel inefficiency is not explained by memory stalls alone; data-dependent control flow and the resulting frontend stalls are also major contributors.

		SpMV			SpMSpM			SpAdd		
		Min	Avg	Max	Min	Avg	Max	Min	Avg	Max
A64FX	Frontend stalls	0%	33%	78%	0%	28%	37%	16%	42%	54%
	Backend stalls	2%	63%	99%	3%	21%	61%	0%	6%	8%
Graviton 3	Frontend stalls	0%	58%	98%	0%	7%	18%	0%	21%	29%
	Backend stalls	1%	28%	86%	14%	38%	79%	4%	14%	26%

ities such as those in the A64FX, where average frontend stalls reach 42% compared with only 6% backend stalls [59].

In conclusion, our study suggests that off-the-shelf CPU cores are not optimized for the irregular control flow and memory operations characteristic of sparse scientific workloads. Data-dependent control flow triggers branch mispredictions that stall the frontend, while irregular memory operations create long-latency fetches that stall the backend. Caches can reduce this cost when locality is available, but they are not sufficient to eliminate the bottleneck. In the next section, we use machine learning workloads—with their wider range of spatial and temporal locality characteristics—to study this limitation further.

4.2 Machine Learning Embedding Workloads

The scientific kernels show that sparse tensor algebra stresses processors through a combination of traversal, scan-and-lookup, merging, and computation. Machine learning embedding workloads let us study the most important of these stressors in isolation: irregular lookup and accumulation of dense embedding vectors. This narrower structure is useful because embedding operations access fixed-size vectors, making spatial locality explicit, and because different models and inputs reuse those vectors at very different rates, making temporal locality explicit. We therefore use embedding workloads to quantify how spatial and temporal locality shape cache behavior, memory-level parallelism, and the request-tracking resources required by conventional architectures. We first characterize these workloads to identify their compute, memory-capacity, and locality requirements. We then show that GPUs, despite being the de facto architecture for machine learning, are inadequate for embedding operations because irregular lookups underutilize their bandwidth and compute resources. Next, we discuss the cost potential of CPUs for these workloads, but show that commodity cores achieve low performance because they cannot sustain enough irregular memory requests. Finally, we build an analytical model to show that this limitation cannot be solved by simply scaling conventional core resources.

4.2.1 Embedding Workload Methodology

Our GPU measurements use an NVIDIA H100 GPU. We measure GPU performance with NVIDIA Nsys [108] and report only device kernel execution time, excluding host-to-device transfers.

To estimate how conventional CPU architectures scale on embedding operations, unlike the previous section, we use gem5 [89] for CPU performance estimation and McPAT [82]

Table 4.4: Specifications of the simulated CPU baseline used for embedding-workload characterization and CPU performance estimation. The system models a high-end Arm-like processor with 96 out-of-order cores, a sliced last-level cache, mesh interconnect, and six HBM3 stacks.

SiP	6 × (16 cores & LLC slices, and 1 HBM stack).
Core	N1-like OoO core @ 2.4 GHz, 224-entry ROB, 2×96-entry LSQ.
Comp	2×64B vector units per core, one 64B×64B matrix unit per core.
L1D	64 KiB/core, 4-way, 2-cycle access, 32 MSHRs, stride/IMP prefetcher.
L2	512 KiB/core, 8-way, 8-cycle access, 64 MSHRs, best-offset prefetcher.
LLC	Mostly exclusive, 1 MiB/slice, 16-way, 12-cycle access, 128 MSHRs.
NoC	2D mesh, 1-cycle routers, 1-cycle links, MOESI-like AMBA 5 CHI.
HBM	HBM3, 560 GB/s per stack, 12 GB per stack.

Table 4.5: Deep Learning Recommendation Model (DLRM) embedding configurations evaluated in this chapter. RM1, RM2, and RM3 vary the number of segments per batch per core, embedding-vector length, and lookups per segment. Larger batches amortize per-batch startup overhead, longer embedding vectors increase spatial locality but also increase cache pressure, and more lookups per segment amortize per-segment loop-control overhead.

Property Description	RM1	RM2	RM3
Segments per batch per core	64	32	16
Elements per embedding vector	32	64	128
Lookups per segment	64	128	256

for power estimation. This simulation-based methodology allows us to vary critical microarchitectural resources such as the reorder buffer (ROB), load-store queue (LSQ), and miss status holding registers (MSHRs), which cannot be controlled on fixed commercial processors. The baseline CPU system, dimensioned as in Table 4.4, models a datacenter-class multicore processor with HBM, HBM bandwidth comparable to the Nvidia H100 GPU [107], and bandwidth per core and per-core performance in the range of commercial HBM-based CPUs [121].

All embedding operations we test are high-performance multicore implementations from the literature. For DLRMs, we use state-of-the-art EB implementations such as those in PyTorch [114] and Glow [122], and test the three configurations in Table 4.5, each with low- (L0), medium- (L1), and high-locality (L2) inputs [50, 49]. For LLMs, we test the original BigBird code and settings [164] while varying the block size. For GNNs and KGs, we use the OGB [58] PyTorch implementations, whereas for MP we adapt the code from

Table 4.6: Graph-learning inputs used in the embedding-workload characterization. The table reports the model class, dataset, graph size, and embedding-vector sizes. For GNNs, layer sizes list the embedding dimension of each layer; for MP and KG workloads, the value is the embedding dimension used throughout the model. Overall, the inputs span different graph sizes and vector dimensions, exposing different locality regimes.

Model	Input	#Nodes	#Edges	Layer sizes
GNN	arxiv [58]	0.2M	1.2M	128×256×256×40
GNN	mag [58]	1.9M	21.1M	128×256×349
GNN	products [58]	2.4M	61.9M	100×256×256×47
GNN	proteins [58]	0.1M	39.6M	8×256×256×112
MP	com-Youtube [78]	1.1M	6.0M	128
MP	roadNet-CA [78]	2.0M	5.5M	128
MP	web-Google [78]	0.9M	5.1M	128
MP	wiki-Talk [78]	2.4M	5.0M	128
KG	biokg [58]	0.1M	5.1M	512
KG	wikikg2 [58]	2.5M	17.1M	512

FusedMM [120]. For these graph-learning models, we use the inputs and feature sizes reported in Table 4.6, drawn from OGB [58] or Snap/SuiteSparse [78, 33].

4.2.2 Embedding Workload Characterization

Before evaluating traditional architectures, we first characterize the properties that determine how embedding operations interact with the memory hierarchy. In particular, we study their loop structure, compute-per-lookup ratio, memory footprint, temporal locality, and embedding-vector size, which together determine whether caches, prefetching, memory bandwidth, or core-side request tracking are likely to become the bottleneck.

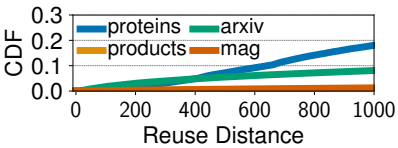
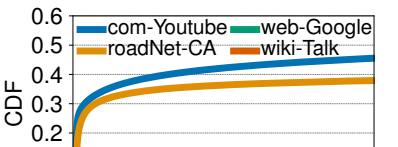
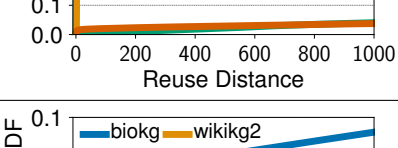
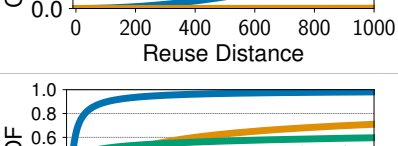
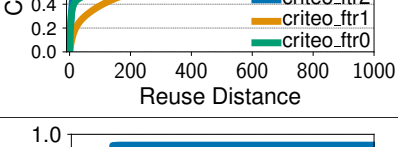
Table 4.7 characterizes the code structure of embedding operations across several machine learning models, whereas Table 4.8 characterizes their memory requirements. For each model, we first analyze its loop hierarchy and compute-per-lookup ratio, which approximate compute requirements. We then compute the memory footprint of the embedding tables to understand their storage demands. Finally, we characterize temporal locality with reuse-distance CDFs and spatial locality with embedding-vector sizes to gauge the effectiveness of caches and prefetchers for embedding lookups.

We characterize temporal locality through the concept of *reuse distance*—the number of other vectors accessed before a vector is accessed again [35]. If a cache can hold x vectors, an access with reuse distance greater than x will likely result in eviction. For a given input,

Table 4.7: Algorithmic characterization of the embedding operations studied in Section 4.2.2. The table summarizes each operation’s loop hierarchy and its compute-per-lookup ratio, using one dense embedding-vector lookup as the normalization baseline. The ratio counts vector arithmetic operations performed for each irregular vector lookup. EB performs one vector accumulation per lookup, KGs perform four vector operations across three lookups, SpMM performs one rescale and one accumulation per lookup, and MP reaches the highest ratio with at most five vector operations across two lookups. SpAttn performs no arithmetic on the looked-up vectors in this characterization because it copies sparse-attention blocks. Thus, the compute-per-lookup ratio ranges from 0 to at most 2.5, showing that these embedding operations expose little arithmetic work per irregular lookup and therefore give compute-oriented architectures limited opportunity to hide lookup latency.

Op	Loop hierarchy	$\frac{N_{\text{compute}}}{N_{\text{lookups}}}$
SpMM (GNNs)	$\forall$ vertex in graph	$\frac{2}{1}$
	$\forall$ neighbor of vertex	
	lookup vector	
	rescale vector	
	accumulate vector	
MP	$\forall$ vertex in graph	$\leq \frac{5}{2}$
	lookup vertex vector	
	$\forall$ neighbor of vertex	
	lookup neighbor vector	
	multiply vert-neigh vecs	
	scale and accumulate vec	
	multiply-accumulate vec	
KGs	$\forall$ sample in batch	$\frac{4}{3}$
	lookup vector	
	compute h-r-l norm	
EB (DLRMs)	$\forall$ segment in batch	$\frac{1}{1}$
	$\forall$ category in segment	
	lookup vector	
	accumulate vector	
SpAttn (LLMs)	$\forall$ block in Q input	0
	$\forall$ block in K input	
	$\forall$ token in K block	
	lookup K vector	
	$\forall$ token in Q block	
	copy K vector	

Table 4.8: Memory characterization of the embedding operations studied in Section 4.2.2. The table reports embedding storage requirements, temporal locality, and spatial locality. Temporal locality is measured with the reuse-distance CDF: the x-axis is cache capacity measured in embedding vectors, and the y-axis is the fraction of accesses whose reuse distance is below that capacity. Thus, the y-value approximates the hit probability of a cache that can hold the corresponding number of embedding vectors. Spatial locality is measured by the number of elements fetched per embedding-vector lookup. Temporal locality varies widely across workloads and inputs. At a reuse distance of 1K vectors, GNN CDFs stay below 20%—meaning that fewer than 20% of accesses have reuse distance below 1K vectors—and KGs below 10%, while MP ranges from below 5% to roughly 45% depending on the input graph. DLRMs exhibit higher but feature-dependent locality, with Criteo features reaching about 63%–99% CDF at 1K vectors. Sparse attention locality depends strongly on block size: larger blocks increase the CDF but also increase copied-vector work. Spatial locality also varies widely, from 8–349 elements for GNNs, 128 for MP, 512 for KGs, 32–256 for EB, and up to $8 \times 8 \times 512$ elements for sparse attention blocks. Together, these ranges show that embedding workloads need architectures that adapt to widely different temporal and spatial locality regimes, exploiting cache reuse when it exists while also sustaining low-reuse vector lookups with different vector sizes.

Op	Memory footprint	Temporal locality (reuse distance CDF)	Spatial locality (vector dim.)
SpMM (GNNs)	Single embedding table GBs large		8-349
MP	Two embedding tables, each GBs large		128
KGs	Single embedding table GBs large		512
EB (DLRMs)	All embedding tables require GBs [50] to TBs [61]		32-256
SpAttn (LLMs)	K-V tensors are MBs Out tensor is MB-GB		$\leq 8 \times 8 \times 512$

we first construct a histogram of all reuse distances, which we then integrate and normalize to obtain the cumulative distribution function (CDF). Hence, $CDF(x)$ approximates the hit probability for a cache that can hold x vectors, as demonstrated in Section 4.2.5. Spatial locality, instead, is characterized by the size of the embedding vector. Beyond the qualitative study in this section, in Section 4.2.5 we use spatial and temporal locality to quantify the ideal memory-level parallelism required to run at peak throughput. We now discuss the characteristics of each class of embedding operations, grouped as graph learning models, deep learning recommendation models, and sparse large language models.

Graph Learning Models

SpMM operations in GNNs perform one multiplication and one elementwise addition per embedding lookup, yielding a compute-per-lookup ratio of 2, as reported in Table 4.7. Message Passing (MP), implemented as an SDDMM operation fused with SpMM, incurs a higher compute-per-lookup ratio. Knowledge Graph workloads (KGs) show the lowest ratio. Overall, across these applications the compute-per-lookup ratio never exceeds 2.5, which is strikingly low for traditional architectures optimized for dense tensor algebra. This limitation is compounded by the wide variation in spatial and temporal locality across workloads and inputs: some embedding lookups can be filtered by caches, whereas others must fetch data directly from memory, making locality a dominant factor in performance.

The reuse-distance CDF reported in Table 4.8 captures temporal locality and cache utilization. For example, at a reuse distance of 1K, GNNs have a CDF below 20%. In other words, more than 80% of embedding lookups touch over 1K distinct vectors before reaccessing the same one. This behavior is especially challenging for large embedding vectors, which consume more cache capacity and reduce the number of vectors that can remain resident. KGs exhibit even lower temporal locality, while MP depends heavily on the input graph structure and can expose substantially different cache behavior across inputs.

Spatial locality is similarly limited, as shown by the embedding-vector sizes reported in Table 4.8. GNN layers access embedding vectors with 8–349 scalar elements per lookup, MP accesses vectors with 128 scalar elements, and KGs reach the highest value at 512 scalar elements per vector. Even these upper bounds remain modest for conventional prefetching mechanisms.

Finally, all three workloads require architectures with memory capacity in the tens of gigabytes to store their embedding tables, as shown by the memory-footprint characterization in Table 4.8. Such capacities make off-chip memory capacity a first-order architectural constraint for current machine learning accelerators [107, 65].

Deep Learning Recommendation Models

EB functions perform one elementwise sum per embedding lookup, yielding a compute-per-lookup ratio of 1, as reported in Table 4.7. Overall, DLRMs invoke the EB function tens to thousands of times to aggregate all categorical features in a batch [61, 49]. Each invocation aggregates tens to hundreds of embedding vectors from a single table with millions of entries [61], each vector storing 32–256 elements, as shown by the embedding-vector sizes in Table 4.8. While this requires a large memory footprint, embedding vectors are accessed with some degree of reuse [133, 46, 64, 2, 154], and modern caches can filter a significant fraction of embedding accesses [61].

If we assume embedding vectors with 256 FP32 elements, a 1 MB cache can fit 1K vectors. Given the reuse-distance CDF of a real-world input such as the Criteo 1TB dataset [31], reported in Table 4.8, $\text{CDF_ftr0}(1\text{K}) = 63\%$, whereas $\text{CDF_ftr2}(1\text{K}) = 99\%$, which approximates the cache hit probability. Similarly, a 2 MB cache would filter 65–99% of the total accesses. Hence, while caches are essential for DLRMs, they provide diminishing returns. Moreover, given the relatively small embedding size, prefetchers yield limited benefits.

Sparse Attention in Language Models

Sparse attention mechanisms replicate blocks of embeddings from the key tensor into the query tensor. As shown in Table 4.8, larger blocks increase spatial and temporal locality over keys (higher horizontal CDF segment). However, smaller blocks reduce the amount of compute. Thus, larger blocks enable better system utilization on compute-oriented machines, while smaller blocks enable lower inference latency on machines with more efficient memory subsystems.

Summary

Taken together, these workloads require architectures that tolerate wide variation in spatial and temporal locality. They must exploit cache reuse when it exists, but also sustain many low-reuse vector lookups without depending on dense arithmetic intensity to hide memory latency. The remainder of this chapter examines the implications of these findings for traditional GPU and CPU architectures.

4.2.3 Embedding Performance on Off-the-Shelf GPUs

GPUs are the de facto accelerator for machine learning, making them the natural first baseline for embedding workloads. However, embedding operations differ from the dense tensor

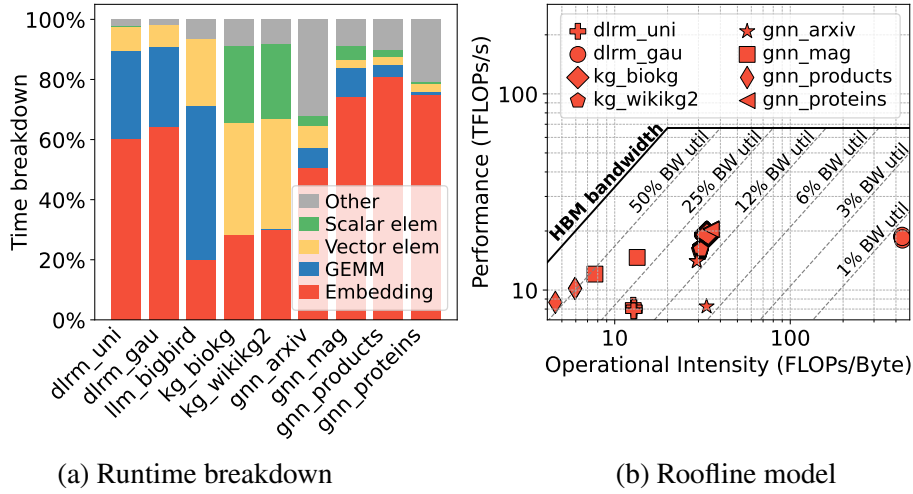


Figure 4.3: Execution time breakdown and architectural utilization of sparse machine-learning models on an Nvidia H100 GPU [107, 106]. Subfigure (a) reports the fraction of end-to-end kernel time spent in embedding lookup/aggregation, GEMM, vector elementwise operations, scalar elementwise operations, and other operations; each stacked bar is normalized to one workload. Subfigure (b) shows a Roofline model for the embedding kernels only. The x-axis reports operational intensity, computed as embedding-kernel floating-point operations per byte transferred to and from GPU HBM during the measured kernel execution, and the y-axis reports achieved performance. Markers identify model/input pairs, and diagonal guide lines indicate fractions of peak HBM bandwidth utilization. In the end, embedding operations dominate the runtime of several sparse ML workloads, yet they do not map efficiently to GPU execution.

kernels that GPUs are optimized for: they perform irregular vector lookups and expose little arithmetic work per lookup. This section shows that, despite their high peak bandwidth and massive parallelism, GPUs cannot efficiently execute these lookup-dominated workloads. Recall from Section 4.2.1 that all GPU results in this section are measured on a real NVIDIA H100 system.

Figure 4.3a shows the runtime breakdown of these models on GPUs, which are the most widely used accelerators for machine learning applications. Overall, embedding operations consume roughly 60%–80% of cycles in GNNs and DLRMs. Because both GNNs and DLRMs pool the embedding vectors of neighboring entities, their embedding runtime scales roughly with the average number of neighbors. In other workloads, such as sparse LLMs and KGs, embedding operations still account for a sizable fraction of execution time, but GEMM and elementwise operations also contribute substantially.

Figure 4.3b shows the Roofline model [158] for embedding operations. Overall, although these operations are critical, they achieve less than 30% of peak compute utilization. As

reported in Table 4.8, GNNs and KGs exhibit the lowest temporal locality and therefore the highest memory bandwidth utilization—yet even in these cases, bandwidth utilization remains below 55%.

The operational intensity of these embedding kernels is higher than that of the sparse linear-algebra kernels in Figure 4.1 because embedding operations generally exhibit higher temporal and spatial locality. Repeated embedding accesses improve cache reuse, while vector-valued lookups expose contiguous accesses within each embedding vector, reducing HBM traffic per operation.

Unlike typical machine learning operations (e.g., dense matrix multiplication), embedding operations look up embedding vectors from scattered memory locations and aggregate them with elementwise operations. GPUs, however, are designed for massively parallel and regular computation where (1) memory accesses can be coalesced and (2) there is sufficient computation to hide memory latency. In embedding workloads, (1) embedding vectors are only hundreds of elements long, limiting coalescing opportunities, and (2) elementwise operations do not provide enough computation to hide memory latency. Therefore, memory latency prevents GPUs from achieving high memory bandwidth utilization, compute utilization, and overall performance on these workloads [115].

Combined with the high cost of GPUs and their limited memory capacity for storing large embedding tables, these inefficiencies make GPUs a poor target for embedding operations. Efficient support for these models must therefore be close to large memories and cannot rely solely on GPU-style coalescing or dense computation to hide lookup latency. To address this issue, several production systems offload embedding operations to CPUs while keeping dense neural network layers on GPUs [27], or execute the entire pipeline on CPUs [98]. Although this solution is more cost-effective, commodity processors remain poorly optimized for the highly irregular memory accesses generated by embedding lookups, as demonstrated next.

4.2.4 Embedding Performance on Off-the-Shelf CPUs

In this section, we show that embedding performance on commodity CPU cores is limited by the inability to sustain enough long-latency memory requests in flight. We demonstrate this with graph learning models, whose spatial and temporal locality span a broad range of the regimes observed across embedding workloads. This analysis shows how cache filtering, long-latency memory accesses, and core-side request tracking interact across inputs with different reuse patterns.

Overall, CPU cores are designed to access data primarily from local caches, which are populated by hardware prefetchers. However, these mechanisms are often ineffective for ir-

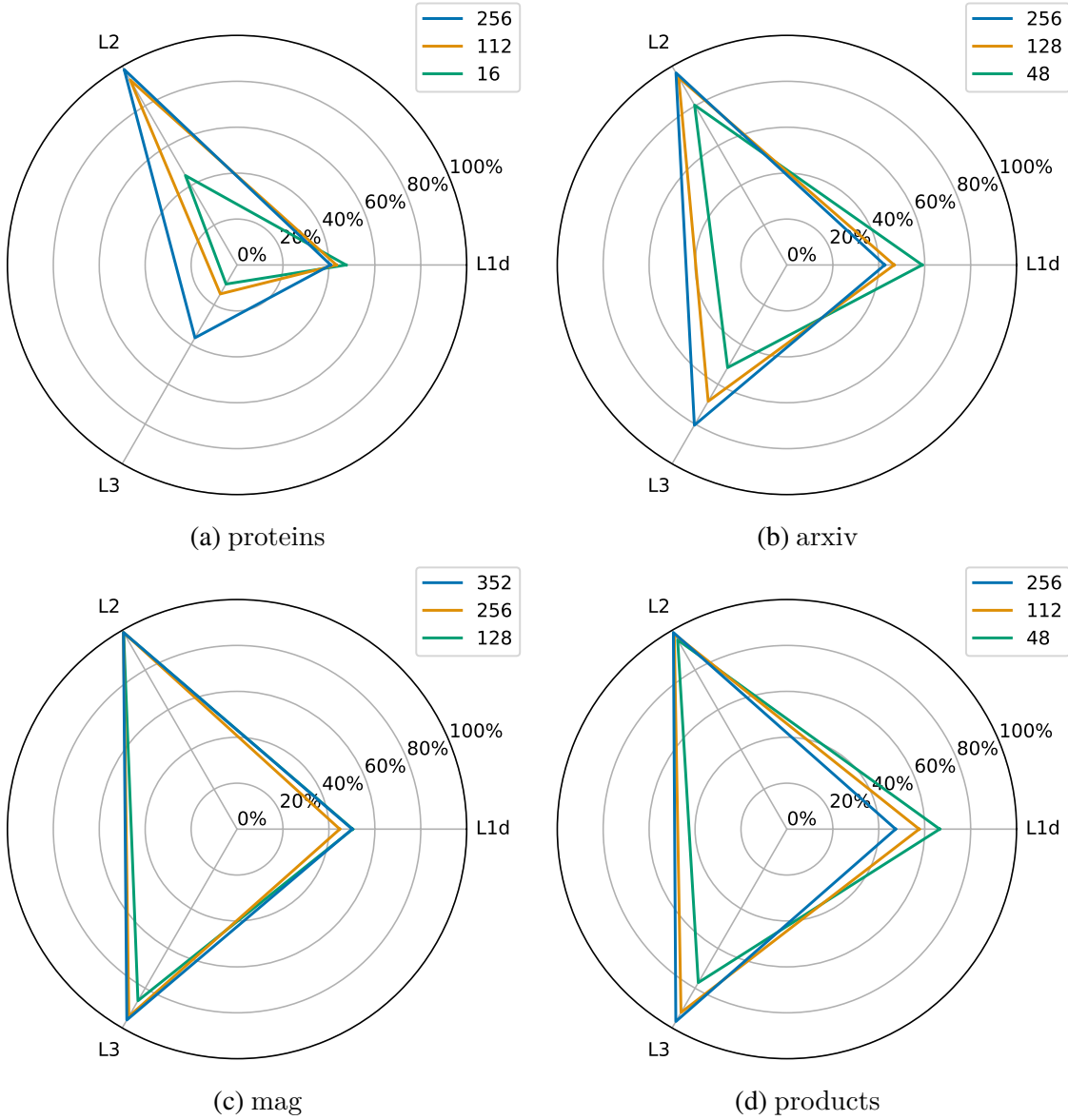


Figure 4.4: Cache miss rate for GNN embedding lookups on the multicore CPU baseline. Each subfigure reports one input graph, and each line corresponds to one embedding-vector size. The radar axes report the local L1D, L2, and L3 miss rates, computed as the percentage of accesses to each cache level that miss at that level. The high-locality graphs, proteins and arxiv, show substantially lower L3 miss rates because repeated accesses to nearby embedding vectors allow the LLC to filter many requests. In contrast, mag and products remain close to the outer rings for L2 and L3, indicating that most vector lookups continue to lower cache levels or HBM. Larger embedding vectors generally reduce L1D miss rates by exposing more spatial locality within each vector, but increase pressure on L2 and L3 because each cached vector occupies more capacity. Thus, L3 miss rates range from 38% to 92% across inputs and embedding sizes, showing that cache effectiveness depends strongly on graph locality and vector size.

regular embedding lookups, especially when reuse is low [50]. Figure 4.4 reports local miss rates for graphs from different domains, which exhibit distinct spatial and temporal locality characteristics. A local miss rate is the fraction of accesses to a given cache level that miss at that level. Overall, different graphs exhibit markedly different cache behaviors. More clustered graphs like proteins and arxiv, which reuse embedding vectors more frequently, achieve significantly better cache utilization. In contrast, sparser graphs like mag and products experience consistently high miss rates across the entire cache hierarchy. Furthermore, larger embedding vectors improve L1 cache behavior due to increased prefetching opportunities, but degrade performance at lower cache levels because of higher cache contention and eviction pressure. Overall, GNN layers experience L3 miss rates ranging from 38% to 92%, resulting in a substantial number of accesses to HBM memory.

As a result, embedding lookups generate a high volume of long-latency memory requests [38]. For instance, as shown in Figure 4.5, up to 86% of the memory requests in GNN embedding operations take more than 10 $\times$ the latency of an L1D cache access, and up to 36% exceed 100 $\times$ that latency because they must reach HBM.

When a core issues a memory request, it must allocate entries in the reorder buffer, load-store queue, and miss-status handling registers at each cache level. The longer the memory access, the more entries are required to sustain multiple outstanding requests, a property generally known as memory-level parallelism. However, because these structures do not scale easily, traditional architectures provision just enough capacity to efficiently load data from local caches, not from HBM. As shown in Figure 4.6, off-the-shelf CPU cores achieve low request throughput across all inputs because long-latency embedding lookups require more memory-level parallelism than the core can sustainably provide [38], and because data-dependent traversal introduces additional control overhead. This results in low HBM utilization even for inputs whose requests mostly miss the caches and reach memory. Therefore, an efficient architecture for embedding operations must expose high memory-level parallelism without tying every in-flight vector lookup to scarce out-of-order core resources.

4.2.5 Ideal Memory-Level Parallelism Requirements

The previous section shows that commodity CPU cores cannot sustain enough irregular memory requests to execute embedding lookups efficiently. This section quantifies why simply scaling conventional cores is not a practical solution. We estimate the memory-level parallelism an ideal core would need to sustain peak embedding-lookup throughput: the model starts from a target request rate at the core, uses the reuse-distance CDF to determine which cache or memory level serves each request, and applies Little’s rule to compute the number of outstanding requests required at the core and at each cache level. This gives a lower-bound

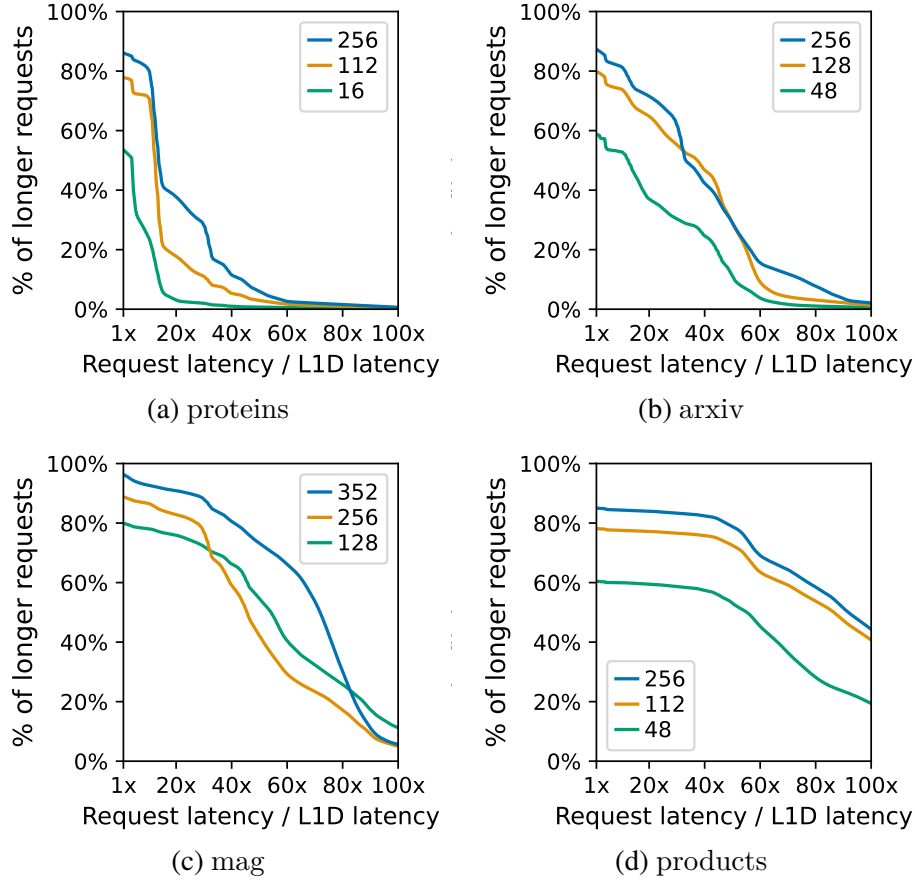


Figure 4.5: Load-to-use latency of GNN embedding lookups on the multicore CPU baseline. Each subfigure reports one input graph, and each line corresponds to one embedding-vector size. The x-axis reports request latency normalized to the latency of an L1D cache access, while the y-axis reports the fraction of requests whose latency is higher than that threshold. High-locality inputs such as proteins drop quickly, indicating that most requests are served close to the core. Lower-locality inputs, especially products and mag, have much longer tails. The embedding-size lines capture spatial locality effects: larger vectors expose more contiguous accesses within each lookup, but also increase the cache footprint of each embedding vector, which can lengthen the tail when temporal locality is low. Across these temporal and spatial locality regimes, up to 86% of requests take more than 10 $\times$ the L1D latency, and up to 36% take more than 100 $\times$ the L1D latency. Thus, many GNN embedding lookups bypass the local cache hierarchy and become long-latency HBM accesses.

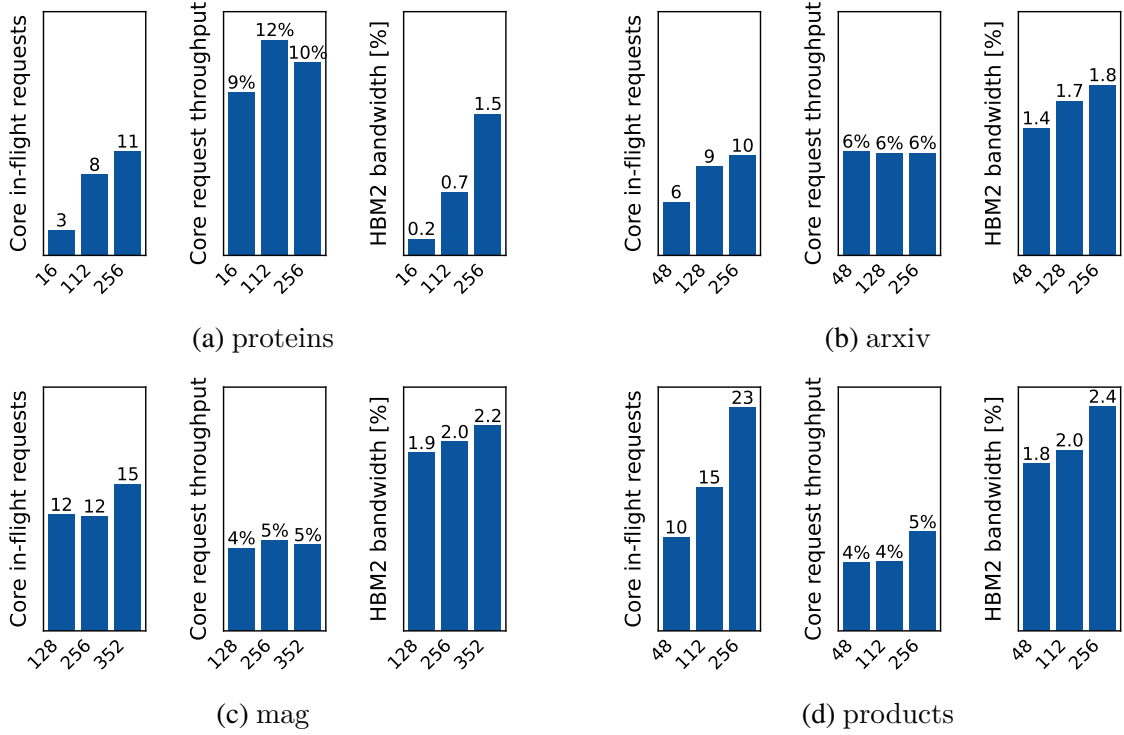


Figure 4.6: Architectural implications of long GNN embedding lookups on the multicore CPU baseline in Table 4.4. Each subfigure reports one input graph, and bars correspond to different embedding-vector sizes. The left plot reports the average number of in-flight memory requests tracked by the core, the middle plot reports request-throughput utilization relative to the ideal rate of one request per cycle, and the right plot reports per-core HBM bandwidth utilization. Across all inputs and embedding sizes, request throughput remains low: the core issues requests at only 4%–12% of peak throughput, with an average of 6.3%. This low throughput comes from memory-level parallelism requirements that exceed what the core can sustain, together with core-side inefficiencies such as data-dependent control overhead. Consequently, HBM bandwidth utilization remains low, ranging from 0.2% to 2.4% per core and averaging 1.6%, even for products and mag, where most requests miss the caches and reach HBM. Thus, off-the-shelf CPU cores cannot generate enough concurrent long-latency requests to use memory bandwidth efficiently, motivating an architecture that tracks many outstanding requests outside scarce core resources.

estimate of the buffering and request-tracking resources that conventional cores would need to keep embedding lookups fully pipelined.

Figure 4.7 illustrates the system abstraction underlying our model. In Figure 4.7a, the system has N logical levels, where L_0 is the core, $L_1 \dots L_{N-2}$ are the caches, and L_{N-1} is HBM. The latency to access level L_b from level L_a is $\lambda_{a \rightarrow b}$ clock cycles. For each level L_x , we define T_x as the request throughput entering that level, measured in requests per cycle.

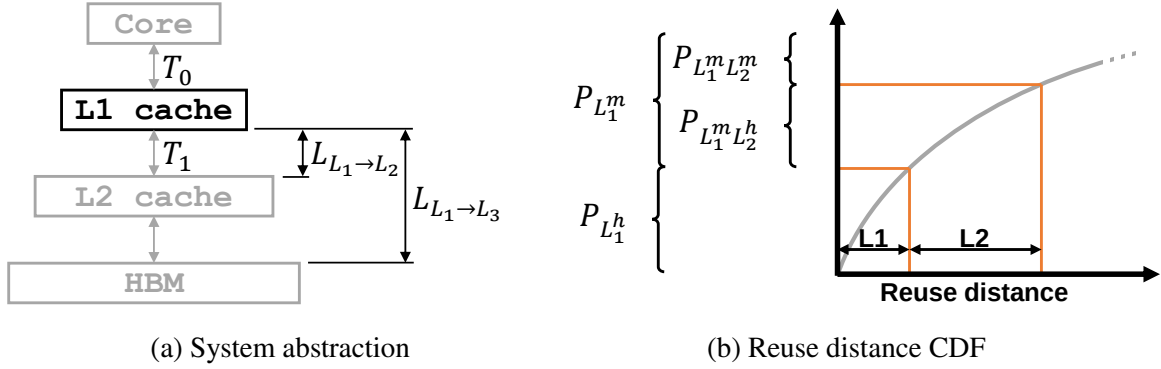


Figure 4.7: System abstraction used by the analytical memory-level parallelism model. Subfigure (a) models the core, cache hierarchy, and HBM as a sequence of levels with per-level latency and request throughput. Subfigure (b) shows how the reuse-distance CDF maps cache capacity, expressed as the number of embedding vectors that fit in each cache level, to hit and miss probabilities. This abstraction connects workload locality to the request rates and outstanding requests needed at each memory level.

Thus, T_0 is the request rate issued by the core, while T_x for $x > 0$ is the rate of requests that miss in previous levels and reach level L_x .

We compute the number of outstanding memory requests using Little’s rule [84], which states that, assuming the core issues memory requests at throughput T_0 , the number of outstanding requests at each level is:

$$R_x = T_x \times \bar{\lambda}_x \quad (4.1)$$

where both terms depend on the cache hit rate per level. Intuitively, the longer the memory requests or the higher the throughput, the higher the number of outstanding memory requests.

We define the cache hit or miss probability at levels x and y as $P_{L_x^{a_x} L_y^{a_y}}$, with $a_x, a_y \in \{h, m\} = \{\text{hit}, \text{miss}\}$. Moreover, we define the (conditional) probability of a hit or miss at level x given a hit or miss at level y as

$$P_{L_x^{a_x} | L_y^{a_y}} = P_{L_x^{a_x} L_y^{a_y}} / P_{L_y^{a_y}}. \quad (4.2)$$

Hence, the *output throughput* T_x at level x is

$$T_x = T_{x-1} \times P_{L_x^m | L_1^m \dots L_{x-1}^m} \text{ cycle}^{-1}, \quad (4.3)$$

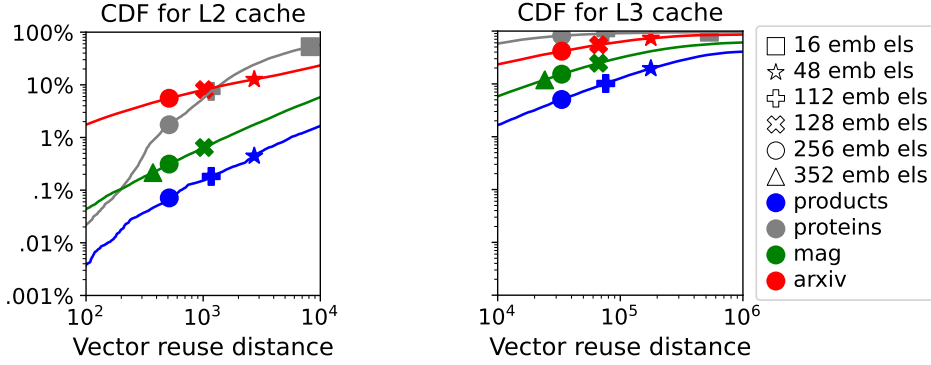


Figure 4.8: Reuse-distance CDFs for the GNN inputs in Table 4.6, mapped to the L2 and L3 capacities of the CPU baseline in Table 4.4. Both subfigures show the same reuse-distance CDFs, but use different x-axis ranges to make the L2 and L3 cache-capacity markers readable. The x-axis reports reuse distance in number of embedding-vector accesses, and the y-axis reports the fraction of accesses with reuse distance below that value; at each cache-capacity marker, this fraction approximates the probability that an access can be served by a cache of that capacity or smaller. Colors identify input graphs, while markers identify embedding-vector sizes, which determine how many vectors fit in L2 (left) or L3 (right). The expected fraction of accesses captured by the L2 capacity is low for most inputs: products ranges from 0.07% to 0.46%, mag from 0.22% to 0.64%, arxiv from 5.6% to 13%, and proteins from 1.7% to 53%. The L3 capacity substantially increases the expected captured fraction, but with large differences across graphs: products ranges from 5.1% to 20%, mag from 12% to 25%, arxiv from 42% to 76%, and proteins from 82% to 96%. Together with the measured miss rates in Figure 4.4, these trends suggest that reuse-distance CDFs can predict cache behavior.

with T_0 the throughput at which the core issues memory requests. Similarly, the *average request latency* $\bar{\lambda}_x$ at level x is

$$\bar{\lambda}_x = \sum_{\ell=x+1}^{N-1} \lambda_{x \rightarrow \ell} \times P_{L_{x+1}^m \dots L_{\ell-1}^m L_{\ell}^h | L_1^m \dots L_x^m} \text{ cycles.} \quad (4.4)$$

We compute these probabilities from the spatial and temporal locality of the input model.

As discussed in Section 4.2.2, we model spatial locality via the embedding vector size, and temporal locality via the CDF of the reuse distance [35]. Recall that the reuse distance is the number of other embedding vectors accessed before a vector is accessed again. If a cache can hold x vectors, an access with reuse distance greater than x will likely result in eviction. Hence, we can model the hit or miss probability of a given cache as the fraction of accesses with reuse distance smaller or greater than x . We model such probabilities by first constructing a histogram of all reuse distances, which we then integrate and normalize to ob-

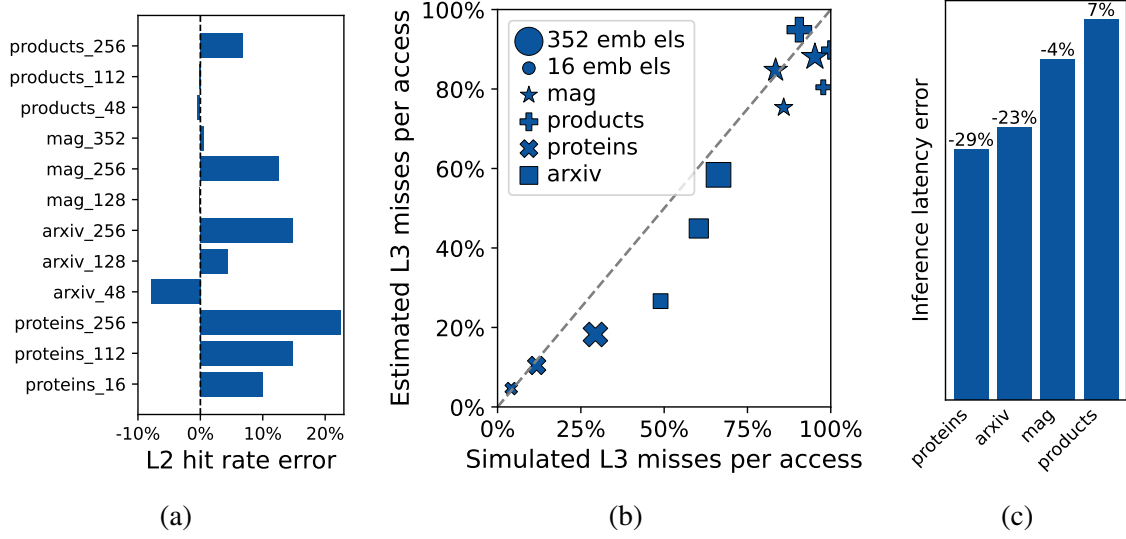


Figure 4.9: Accuracy of the analytical model compared with gem5, using the GNN inputs in Table 4.6 on the CPU baseline in Table 4.4. Subfigure (a) reports L2 hit-rate prediction error; positive values indicate that the model overestimates the gem5 hit rate, and negative values indicate underestimation. Across layers, L2 error ranges from -8 to +23 percentage points, with an average absolute error of roughly 8 percentage points. Subfigure (b) compares simulated and estimated L3 misses per access. The dashed diagonal indicates perfect agreement, marker shape identifies the input graph, and marker size identifies the embedding-vector size. The model preserves the main trend: low-miss cases remain low, high-miss cases remain high, and points with more than 80% simulated L3 misses stay close to the diagonal, although the model underestimates some midrange cases. Subfigure (c) reports end-to-end execution-cycle error per input graph; errors range from -29% for proteins to +7% for products, with roughly 16% average absolute error. Thus, the CDF-based model is accurate enough to capture cache behavior and execution-cycle trends for the memory-level-parallelism analysis, providing reasonable accuracy for an analytical model intended to predict trends rather than exact cycle counts [25].

tain the CDF. Thus, $CDF(x)$ approximates the hit probability of that cache, and Figure 4.7b shows how to derive the above probabilities starting from the CDF. Figure 4.8 characterizes the graphs in Table 4.6; the CDF trends match the measured cache behavior in Figure 4.4, suggesting that reuse-distance CDFs are a promising predictor of cache performance. We now validate this connection directly against gem5.

Figure 4.9a shows that our model estimates L2 accesses within 23% error, whereas Figure 4.9b shows similar behavior for L3 accesses, especially for high miss rates, which are the most relevant for our analysis. We now use these cache-access probabilities to estimate the end-to-end execution cost of embedding inference. At a high level, the model first computes the execution cycles and memory-level parallelism required by ideal execution, then

compares these requirements with the maximum throughput and request-tracking capacity of each memory-system level, and finally rescales the ideal cycle count according to the most restrictive resource. The model represents ideal execution with the throughput T_ℓ^i and number of outstanding requests R_ℓ^i required at each memory level ℓ , where the superscript i indicates *ideal*. We start by computing the *ideal execution cycles* when running at full core throughput (one load per cycle per port):

$$C^i = \left\lceil \frac{\text{numLookups} \times \text{numLoadsPerLookup}}{\text{numReadPorts}} \right\rceil \quad (4.5)$$

where numLookups is the number of loaded embedding vectors, numLoadsPerLookup is the number of vector loads required to fetch an embedding vector, and numReadPorts is the number of read ports out of the core. Given a system where each component at level ℓ has:

- *max throughput* T_ℓ^m (bounded by its ports and frequency),
- *max number of requests* R_ℓ^m (bounded by MSHRs),

where the superscript m indicates *maximum*. Rather than composing these limits across memory levels, we use a bottleneck model: each resource determines the fraction of ideal execution it can sustain, and the most restrictive resource sets the overall scaling factor. We define the fraction of ideal execution that each component at level ℓ can sustain as:

$$S_\ell^t = \min \left(1, \frac{T_\ell^m}{T_\ell^i} \right) \quad (4.6)$$

for throughput, and

$$S_\ell^r = \min \left(1, \frac{R_\ell^m}{R_\ell^i} \right) \quad (4.7)$$

for request tracking capacity. These scaling factors are 1 when the corresponding resource can sustain ideal execution and smaller than 1 when it becomes a bottleneck. We then compute the limiting scaling factor S as:

$$S = \min \left(\min_\ell S_\ell^t, \min_\ell S_\ell^r \right). \quad (4.8)$$

Finally, we compute the *estimated execution cycles* C^e by rescaling the ideal execution cycles C^i according to this bottleneck scaling factor:

$$C^e = \frac{C^i}{S}. \quad (4.9)$$

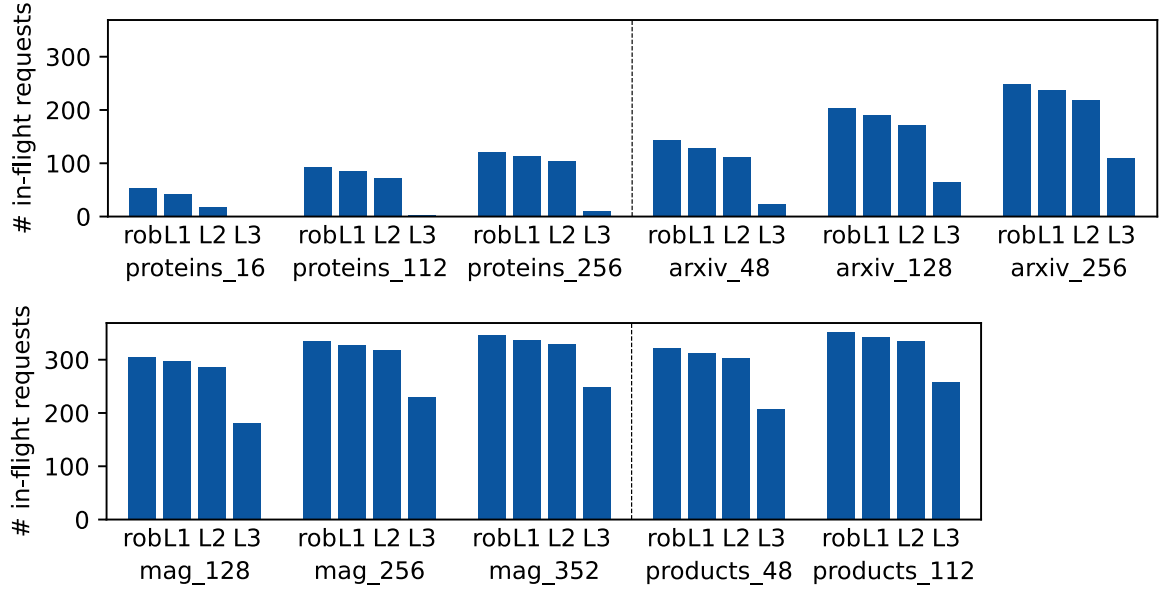


Figure 4.10: Outstanding requests required to sustain ideal GNN embedding-lookup throughput on the CPU baseline in Table 4.4. These values are lower-bound requirements computed with the analytical model described in this section (Equation (4.1)), assuming an ideal core-side request rate of one embedding-vector load per cycle and accounting for the cache hit and miss probabilities derived from the reuse-distance CDFs. Each group of bars corresponds to one evaluated layer, identified by input graph and embedding-vector size. Within each group, the rob bar reports the number of requests that must remain in flight at the core, while the L1, L2, and L3 bars report the corresponding MSHR demand at each cache level after cache filtering. The required memory-level parallelism grows with lower temporal locality and larger vectors: `proteins_16` requires only 55 core-level requests and roughly one L3 request, whereas `products_112` reaches 368 core-level requests and 270 L3 requests. Across all layers, the average demand is about 241, 230, 216, and 127 requests at the core, L1, L2, and L3 levels, respectively. Thus, sustaining ideal embedding throughput would require far more in-flight request tracking than conventional out-of-order cores and cache MSHR structures are designed to provide.

Execution time is then obtained by dividing these cycle counts by the core frequency. Figure 4.9c validates this estimate by showing that the model estimates the end-to-end execution cycles of the entire embedding operation within 29% error, which is reasonable for an analytical model intended to predict trends rather than exact cycle counts [25]. This level of accuracy suggests that the locality and memory-system parameters captured by the model are sufficient to explain the dominant performance behavior of these workloads.

We now leverage Equation (4.1), Little’s rule, to estimate the number of outstanding requests per core required to run at peak throughput (one load per cycle per port). This ideal request rate is not intended to model achieved execution on a conventional core; rather, it asks how much memory-level parallelism the memory system would need to sustain if the

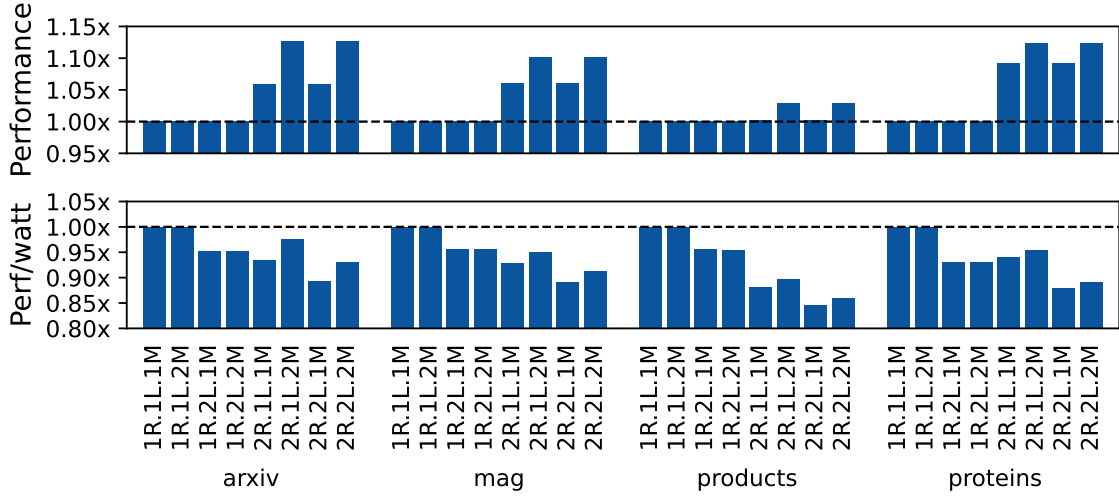


Figure 4.11: Cost of scaling memory-level parallelism in the CPU baseline from Table 4.4 for GNN embedding operations. The top plot reports performance normalized to the baseline core, and the bottom plot reports normalized performance per watt; dashed horizontal lines mark the baseline value of $1\times$. Groups correspond to input graphs, and bars within each group correspond to different resource-scaling configurations. In the configuration names, R denotes reorder buffer and register-file capacity, L denotes load-store queue capacity, and M denotes L1D MSHR capacity; 1R.1L.1M is the baseline core, while 2R.2L.2M doubles all three resources. Scaling only L or M provides essentially no performance improvement, while scaling R provides most of the gains. Even the fully scaled core improves performance by only $1.03\times$ – $1.13\times$ across inputs, but reduces performance per watt to $0.86\times$ – $0.93\times$ because power increases by roughly 20%–26%. Thus, scaling conventional core structures is an inefficient way to provide the memory-level parallelism required by embedding lookups.

core-side pipeline were not the bottleneck. Figure 4.10 shows that this would require up to 368 outstanding requests per core, 241 on average. In the next section, we demonstrate that traditional cores cannot achieve such memory-level parallelism.

4.2.6 The Cost of Scaling Memory-Level Parallelism

Increasing the memory-level parallelism of a CPU core to track more in-flight requests requires scaling up the reorder buffer, load-store queue, and miss-status handling registers [135]. However, these resources scale inefficiently. As shown in Figure 4.11, for datasets with high reuse, doubling them improves performance by only 12% while increasing power consumption by 25%, whereas for datasets with low reuse, it improves performance by 3% while increasing power consumption by 18%. In addition to degrading performance per watt, scaling up these resources introduces timing-closure challenges [136], which would require lowering the core frequency and, consequently, overall performance.

Another option is to scale out the number of cores or hardware threads. This can increase the number of outstanding memory requests, but it does not remove the underlying inefficiency: the system must still provision enough shared MSHRs, interconnect bandwidth, and memory-system queues to track those requests, while also replicating core structures that provide little benefit for memory-dominated embedding lookups. As shown in Figure 4.6, saturating a single HBM stack would require 43–72 traditional CPU cores, depending on input reuse. Thus, while scale-out can expose memory-level parallelism, it does so by scaling general-purpose execution resources along with the memory-request machinery. These results point to the same design principle as the scientific workloads: sparse accesses should be decoupled from the compute pipeline so that memory-level parallelism can scale without proportionally scaling expensive core structures.

4.3 Summary

In this chapter, we have demonstrated that traditional architectures that tightly couple control flow, memory access, and computation are fundamentally inadequate for sparse tensor algebra workloads. Scientific kernels show that traversal, scan-and-lookup operations, and coordinate merging leave both compute units and memory bandwidth underutilized. Machine learning embedding operations confirm this bottleneck across a wide range of spatial and temporal locality regimes. Scaling conventional cores up or out does not provide this memory-level parallelism efficiently. In the next chapter, we demonstrate how decoupling sparse traversal, merging, and marshaling from vector computation can overcome these limitations and substantially improve performance and efficiency.

Chapter 5

The Tensor Marshaling Unit for DAE Designs

The previous chapter showed that sparse tensor algebra needs both the flexibility of general-purpose cores and specialized support for irregular traversal, merging, and lookup. This chapter turns that observation into a concrete architecture: the Tensor Marshaling Unit (TMU), a specialized access unit for Decoupled Access–Execute (DAE) execution of sparse tensor algebra. The TMU efficiently performs sparse tensor traversal and merging, then

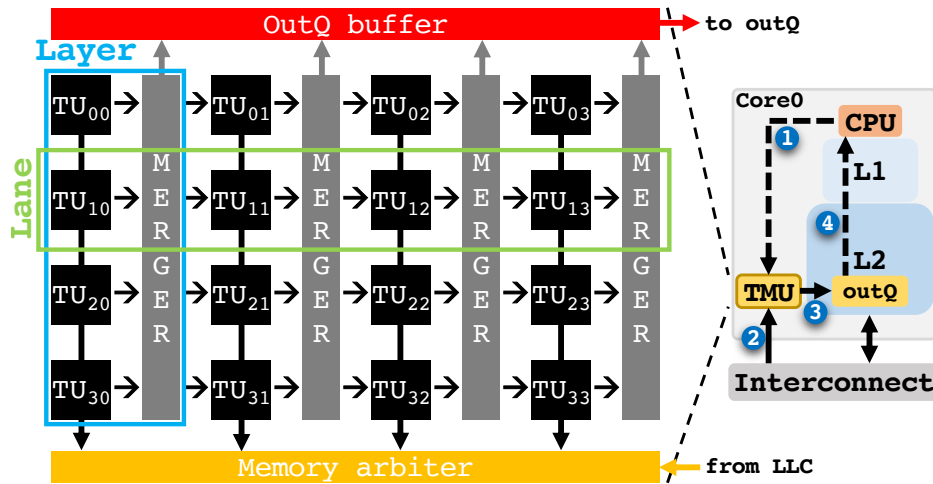


Figure 5.1: TMU architecture and placement in a multicore processor. Each core is paired with a nearby TMU that reads data through the interconnect/last-level cache and marshals operands to the core through an output queue (outQ). Internally, the TMU is organized as a matrix of Traversal Units (TUs): rows form lanes for parallel traversal and vector marshaling, while columns form layers that implement nested traversal and coordinate merging. In this way, the TMU offloads sparse traversal and merging from the core while feeding dense operands to conventional vector computation, improving overall system performance.

marshals vector operands to the CPU vector units. By mapping irregular operations to the TMU and regular computation to the CPU, these two units can be specialized and optimized independently, improving overall system performance. Sections 5.1 and 5.2 introduce the TMU programming model for decoupled sparse tensor algebra, its microarchitecture, and its integration into a DAE processor. Sections 5.3 and 5.4 then evaluate the TMU on scientific sparse tensor algebra and machine learning embedding workloads, including specialized DAE organizations for low-reuse embedding operations.

5.1 TMU Programming Interface

The TMU programming interface exposes the architectural boundary established in this thesis: traversal, merging, and data marshaling are programmed as dataflow operations in the TMU, while arithmetic remains in ordinary callback code executed by the CPU. This section defines that interface and shows how sparse tensor algebra operations are decomposed across the TMU and the compute core. The architecture follows directly from the requirements derived in Chapter 4: Traversal Units (TUs) expose memory-level parallelism for irregular scan-and-lookup operations, Traversal Groups (TGs) perform coordinate merging and loop synchronization, the memory arbiter and TU queues track long-latency requests outside the core [38], and the outQ marshals scalar, vector, and predicate operands into the core so computation can still use standard vector execution. Section 5.1.1 first presents the overall TMU execution model, Sections 5.1.2 and 5.1.3 then describe how traversal, merging, and vectorization are programmed, and Sections 5.1.4 and 5.1.5 explain how marshaled operands and callbacks map complete sparse tensor algebra expressions to the TMU.

5.1.1 TMU Overview

Figure 5.1 shows the proposed DAE processor—a general-purpose processor that features a TMU per core to decouple memory access and computation into two separate units. Overall, the TMU can be ① programmed to ② traverse and merge tensor operands and ③ marshal the aggregated data to the compute core via a memory-mapped output queue (outQ) that the compute core can ④ process. Data reads are managed by the TMU memory arbiter, whereas marshaling is managed by the outQ buffer. TMU traversal, merging, and marshaling are programmed with dataflow primitives, whereas the compute code is wrapped in callback functions that the TMU triggers on traversal/merging events. The programming contract is intentionally narrow: the TMU does not execute arbitrary arithmetic, but instead emits an ordered stream of callback IDs and operands. The compute core consumes this stream

```

1 void spmv(v_ty* x, const csr_ty* a, const v_ty* b) {
2     v_ty sum = 0;
3     for (i_ty i = 0; i < a->num_rows; i++) {
4         p_ty p_beg = a->ptrs[i], p_end = a->ptrs[i+1];
5         for (p_ty p = p_beg; p < p_end; p++) {
6             v_ty vec_val = b[a->idxs[p]];
7             v_ty nnz_val = a->vals[p];
8             sum += vec_val * nnz_val; //inner-loop body (ri)
9         }
10        x[i] = sum; sum = 0; //inner-loop tail (re)
11    }
12 }

```

Figure 5.2: SpMV decomposition into TMU-managed traversal/lookup and core-managed computation. The outer loop, shown in yellow, traverses the dense CSR fiber of row pointers; the inner loop, shown in blue, scans nonzeros and performs the dense-vector lookup; and the body and tail regions, shown in green, perform arithmetic and final writeback. This decomposition exposes the TMU programming boundary: traversal and marshaling move to the TMU, while arithmetic remains on the compute core.

and executes the corresponding callback code, so the TMU preserves the traversal and event order of the original loop nest.

Because of the format abstraction described in Section 2.1.2, we can express tensor operations as *dataflow* programs that stream input tensors through traversal, merging, and compute operations [57]. With this dataflow abstraction, we can map tensor traversal and merging to the TMU, and computation to the compute core. Figure 5.2 shows this decomposition for SpMV, where row-pointer traversal and nonzero traversal become TMU work, while multiplication, accumulation, and final writeback remain compute core callbacks.

Traversal units, lanes and layers: As shown in Figure 5.1, the TMU is a matrix of Traversal Units (TUs), where each TU implements logic and storage to traverse a tensor fiber, that is, each of the for loops in Figure 5.2 (lines 3-4 and 5-7). Each of these for loops can be mapped to the TUs of a TMU column, called a *layer*. The for loop in lines 3-4 would map to the left-most layer, producing indices that flow into the second layer, where the for loop in lines 5-7 would be mapped. Therefore, the TMU has a dataflow design that flows rightward. In addition, loop iterations can be parallelized using multiple rows, called *lanes*, which can also be used to load and merge multiple tensors.

Multi-lane parallelism: The values loaded by multiple lanes, either for parallel traversal or merging, are marshaled into vector operands. For example, each lane of the second layer that is traversing the for loop in lines 5-7 of Figure 5.2 loads one *vec_val* and one *nnz_val*, which are then marshaled into vector operands.

```

1 ri_callback(vfloat nnz_vals, vfloat vec_vals){
2   vfloat mul_vals = vec_mul(nnz_vals, vec_vals);
3   sum += vec_reduce(mul_vals);
4 }
5 re_callback() {
6   x[i++] = sum; sum = 0;
7 }

```

Figure 5.3: SpMV callbacks executed by the host core. The *ri* (row iteration) callback multiplies nonzero values with dense-vector values and accumulates them, whereas the *re* (row end) callback writes back the accumulated value and reinitializes the accumulator. Callbacks isolate arithmetic and output updates from the traversal logic executed by the TMU.

Callbacks and outQ processing: These operands are computed by the host core with the callbacks in Figure 5.3. These callbacks wrap the compute code of the body (*ri*) and tail (*re*) regions of the inner loop shown in Figure 5.2. The *ri* callback accumulates partial results (into the *sum* variable) and is triggered by the TMU at every row iteration (inner-loop body). The *re* callback stores results at the end of every row traversal (inner-loop tail). To decouple TMU and compute core execution, the TMU sends to the compute core, through the outQ, a control/data stream defining the ordered sequence of callbacks and operands that the compute core should compute.

For instance, running a two-lane SpMV on the CSR matrix in Figure 2.1 would push the following control/data sequence into the outQ:

```
ri AB ab ri C0 c0 re re ri DE cd re re ri FG de ri H0 f0 re
```

In this notation, *ri* and *re* are callback IDs, uppercase letters denote nonzero matrix values, lowercase letters denote dense-vector values, and 0 marks an inactive lane introduced by padding or masking. This sequence triggers two accumulations for the first row, none for the second row (since empty), and so on.

Sections 5.1.2 to 5.1.5 illustrate how to program the TMU to run parallel traversal and merging of Einsum expressions.

5.1.2 Programming Tensor Traversal

As described in Section 2.1.3, all sparse and dense fiber traversals have the following for loop structure:

```
for(i=beg; i<end; i+=stride)
```

Table 5.1: TU traversal primitives used to program fiber iteration. In each signature, `int` arguments are constants and stream arguments are values produced by a parent traversal. Dense implements a regular loop with static bounds, Range implements a loop with data-dependent bounds from streams, and Index implements a fixed-size dense lookup from a streamed base position. Overall, these primitives cover the loop-bound patterns needed for dense, compressed, and indirect traversal.

Type	Description	Signature	Traversal
Dense	dense (or single) fiber scan	<code>DnsFbrT(int beg, int end, int stride=1)</code>	<code>for(i=beg; i<end; i+=stride)</code>
Range	compressed fiber lookup and scan	<code>RngFbrT(stream beg, stream end, int offset=0, int stride=1)</code>	<code>for(i=beg.head()+offset; i<end.head(); i+=stride)</code>
Index	dense fiber lookup and scan	<code>IdxFbrT(stream beg, int size, int offset=0, int stride=1)</code>	<code>for(i=beg.head()+offset; i<beg.head()+size; i+=stride)</code>

Table 5.2: Data streams produced or transformed by a TU. Memory streams load data using streamed indices, iteration and arithmetic streams generate addresses and loop positions, and forwarding or mask streams communicate values and predicates to the next layer. In this way, streams provide the dataflow interface between traversal, memory access, merging, and marshaling.

Type	Description	Function
mem	loads p 's elements from memory at index x	$p[x]$
ite	generates TU iteration indices $\in [beg, end)$	$x: b \rightarrow e$
lin	implements a linear transformation on x	$ax+b$
map	selects x -th value from a small TU-local map $a = \{v_1, \dots, v_{16}\}$	$a[x]$
ldr	computes the address of the x -th element in p	$\&p[x]$
fwd	forwards a leftward TU stream to the next layer	
msk	outputs a layer predicate for merging and lockstep operations	

Table 5.3: Inter-layer configurations used by Traversal Groups. Single, BCast, and Keep control how lanes are selected or broadcast, whereas DisjMrg, ConjMrg, and LockStep implement disjunctive merging, conjunctive merging, and lockstep co-iteration. In this way, these modes determine how lanes synchronize and how sparse fibers are merged across layers.

Type	Description
Single	iterates a single lane
BCast	broadcasts a single lane to a parallel group
Keep	keeps one lane out of a parallel group
DisjMrg	joins lanes in the current TMU layer
ConjMrg	intersects lanes in the current TMU layer
LockStep	co-iterates lanes in the current TMU layer

Each TU provides the logic and storage to iterate such a loop, with beg and end defined by the fiber traversal, and load fiber values into streams.

Fiber iteration: Table 5.1 shows the primitives we can use to program TUs to implement the traversals in Section 2.1.3. For instance, we can use a Dense primitive to implement the outer loop of SpMV in Figure 5.2, to iterate through matrix rows and load the row pointers. These pointers, loaded into streams, are then used to implement the inner-loop traversal, with a Range primitive, to load nonzero values and column indices (compressed levels), which, in turn, are used to load dense vector values through scan-and-lookup. SpMM ($Z_{ij} = A_{ik} B_{kj}$), instead, works similarly but requires an additional inner loop, implemented with an Index primitive, to scan entire rows of the right-hand side matrix (B_k).

Data loading: Once the iteration space of a TU is defined, data loading is implemented through the *data streams* listed in Table 5.2. The mem stream loads data from a base address and a stream of indices. To generate the index stream, TUs push their current iteration index (i.e., loop induction variable) into an ite stream, which can also be transformed with lin or map streams. Indirect accesses can be implemented by chaining mem streams. Hence, we can represent a fiber traversal with a *parent-children* dependency tree where the ite stream is the *root*. For instance, the SpMV scan-and-lookup operation in Figure 5.2 is implemented by instantiating two mem streams: a parent stream that loads the column indices, which are then used by its child stream to index into the dense vector.

5.1.3 Programming Merging and Vectorization

We can use multiple TMU lanes to parallelize loop traversal or merge different tensors. Parallelization and merging can happen at any tensor level by setting TMU layers to one of the configurations shown in Table 5.3.

Parallelization is achieved by distributing loop iterations across different lanes. Overall, parallelization enables (1) parallel loading, reducing the control overhead of traversing compressed data structures, and (2) parallel marshaling, enabling efficient vectorized compute in the host core. The two-dimensional TMU design allows loops to be parallelized at different levels. For instance, inner-loop parallelization allows the TMU to marshal adjacent tensor elements into the same vector operand (i.e. the usual vectorization scheme). Outer-loop parallelization, instead, allows the TMU to marshal multiple fibers, slices, and arbitrary tensor dimensions into the same vector operand, enabling higher-dimensional parallelization schemes. In the SpMV example in Figure 5.2, where we parallelize at the inner-loop level, we configure the first layer to load row pointers and BCast them to the next layer. The second layer then loads multiple elements of the same row in LockStep, with each lane starting at a different offset and advancing by the number of active lanes so that lanes cover disjoint

```

1 ri_callback(vfloat nnz_els){
2   *out_ptr++ = vec_reduce(nnz_els);
3 }

```

Figure 5.4: SpKAdd callback executed by the host core. The TMU merges sparse fibers across lanes and produces a predicate identifying active vector elements; the callback reduces those active elements into the output value. In this way, merging remains in the TMU, while arithmetic reduction remains in the vector core.

nonzeros. To parallelize SpMV at the outer-loop level, concurrently loading multiple rows into different lanes, both layers work in LockStep. Both parallelization schemes aggregate data into vector operands that are marshaled into the core for computation. Boundary conditions can be handled by padding or marshaling the `msk` stream along with the operands, indicating the active elements in a multi-hot bit vector (equivalent to the `svwhilelt` operation in Arm SVE [146]).

Fiber merging is achieved by loading different tensors in different lanes and merging them hierarchically with the mergers placed between layers. Merging operations are implemented by sorting the fiber indices produced by the active lanes of a TMU layer, which is achieved by iteratively pulling fibers with minimum indices. The position of these fibers can be identified with a multi-hot predicate with one bit per lane, which is pushed in the `msk` stream of the layer. For the two-lane merging example in Figure 2.2, the `msk` stream is 11, 01, 10, 11, with the first bit representing fiber A and the second bit representing fiber B. These predicates are then used to aggregate vector operands to send to the core. In this way, for instance, we can implement SpKAdd [59], a summation of K sparse matrices, by mapping matrices in K different lanes and merging them with DisjMrg layers. Figure 5.4 shows the compute code of SpKAdd: the TMU traverses all the K matrices row by row and, for each row, sends to the core all nonzero values with the same index, which are reduced with a vector operation. Running SpKAdd on the two rows in Figure 2.2 would produce the following vector operand stream: AB C D EF, with AB and EF being reduced in the core. If matrices are in CSR format, only the second compressed dimension requires merging. In contrast, if matrices are in DCSR format, both dimensions are compressed and both need to be merged. In this latter case, the merge happens hierarchically: only the active lanes from the first dimension, which are identified by the `msk` predicate of the first layer, are merged in the second layer.

5.1.4 Data Marshaling and Computation

Once we map the loop structure of a tensor expression into TMU layers, the TMU streams the aggregated data to the core for computation. Core compute is enabled by wrapping

the compute code within the *head* (H), *body* (B), and *tail* (T) regions of traversal/merging loops into callback functions. These callback functions have a unique callback ID and a list of scalar, vector, or predicate operands produced by the TMU. In the SpMV example in Figure 5.2, we wrap the code in the inner-loop body into a *ri* callback, which multiplies and accumulates the matrix and vector values provided by the TMU, and the code in the inner-loop tail into a *re* callback, to store the accumulated results.

Each TMU layer can be programmed to trigger callbacks upon traversal or merging events, such as the *begin*, *iteration*, and *end* events of the traversal assigned to that layer. In the SpMV example in Figure 5.2, we register the *ri* callback to the *iteration* event of the TMU layer that traverses the nonzeros of a CSR row (the inner loop), and the *re* callback to the *end* event of that same inner-loop traversal. For the *ri* callback, we also register the list of operands consisting of matrix and vector values. Callback registration can be done with the following operation:

```
add_callback(event, callback_id, args_list)
```

While running, the TMU pushes the callback IDs and vector operands of each registered event into the current outQ chunk. When the chunk is full, the compute core starts reading callback IDs and executes the proper HBT callback to compute the data operands. Meanwhile, the TMU populates another outQ chunk, overlapping data loading and computation.

5.1.5 Mapping Sparse Tensor Algebra to the TMU

Figure 5.5 shows the full TMU configuration code for the SpMV example in Figure 5.2. The top block defines the dense traversal (*DnsFbrT*) for the CSR matrix row pointers. The middle block (second layer) co-iterates in *LockStep* two columns in parallel (*RngFbrT*), marshaling vector operands to the compute core (lines 17 and 18). The bottom block registers the *callbacks* that occur at each row iteration (*ri*), specifying the associated data, and at the end of a row (*re*).

Table 5.4 provides an extensive list of tensor algebra algorithms showing how they map to the TMU. The loop schedule encodes the different parallelization schemes by underlining the parallel loops that generate vector operands. Additionally, for each loop we state: (i) traversal type, (ii) instantiated data streams, (iii) layer operation mode, and (iv) associated callbacks. To prove functional completeness of TMU primitives with respect to tensor algebra, we observe that our primitives implement all of the minimal functionalities for tensor algebra formulated by Hsu et al. [57].

Table 5.4: Mapping sparse tensor algebra kernels to TMU hardware. Each row lists an Einsum expression, sparse input formats, a loop schedule, the traversal primitives, data streams, inter-layer modes, and callbacks needed to execute that kernel on the TMU. Underlined schedule indices identify the loop level parallelized across lanes; labels such as P0, P1, and P2 distinguish alternative lane-parallelization choices for the same kernel. Thus, the table shows that the same TMU primitives cover sparse-dense traversal, sparse-sparse merging, vectorization, and callback generation across a broad set of kernels.

Algorithm	Einsum expression	Sparse formats	Schedule (SIMD)	Traversal type	Data streams	Single	BCast	Group Traversal	Callbacks
				Dns Idx Rng	lin map mem ldr fwd msk	Step	Lock	CujMrg DsjMrg Keep	H B T
SpMV P0	$Z_i = A_{ij} B_j$	A=CSR	$\underline{ij}$	i	ij		ij		j
SpMV P1	$Z_i = A_{ij} B_j$	A=CSR	$\underline{ij}$	i	ij		i		j
SpMSPV	$Z_i = A_{ij} B_j$	A,B=CSR	$\underline{ij}$	ij	ij		i	j	j
SpMM P0	$Z_{ij} = A_{ik} B_{kj}$	A=CSR	$\underline{ikj}$	ij	ikj		ikj		j
SpMM P1	$Z_{ij} = A_{ik} B_{kj}$	A=CSR	$\underline{ikj}$	ij	ikj		i		j
SpMM P2	$Z_{ij} = A_{ik} B_{kj}$	A=CSR	$\underline{ikj}$	ij	ikj		k		j
SpMSpM P0	$Z_{ij} = A_{ik} B_{kj}$	A,B,X=CSR	$\underline{ikj}$	i	ikj		ikj		j
SpMSpM P2	$Z_{ij} = A_{ik} B_{kj}$	A,B,X=CSR	$\underline{ikj}$	i	ikj		k		j
SpKAdd	$Z_{ij} = \sum_k A_{ij}^k$	A ^k ,X=DCSR	$\underline{kij}$	i	ij	j		ij	j
PageRank	$Z_i = A_{ij} X_j Y_i$	A=CSR	$\underline{ij}$	i	ij		i		j
TriangleCount	$c = L_{ik}^T L_{kj} L_{ij}$	L=CSR	$\underline{ikj}$	i	ikj	k			j
MTTKRP P1	$Z_{ij} = A_{ikl} B_{kj} C_{lj}$	A=COO	$\underline{ikl}$	ij	ikl		ik	j	j
MTTKRP P2	$Z_{ij} = A_{ikl} B_{kj} C_{lj}$	A=COO	$\underline{ikl}$	ikl	ikl	i	kl		kl
SpTC	$Z_{ij} = A_{ikl} B_{lkj}$	A,B=CSF	$\underline{ikl}$	i	$ijkl$		i	$ijkl$	j
SpTTV	$Z_{ij} = A_{ijk} B_k$	A=CSF	$\underline{ijk}$	ik	ijk		k		k
SpTTM	$Z_{ijk} = A_{ijl} B_{lk}$	A=CSF	$\underline{ijk}$	ik	$ijkl$		l		k

```

1 //Load and broadcast CSR row pointers
2 row_fbrt = DnsFbrt(0, num_rows);
3 row_ptbs = row_fbrt.add_mem_str(a->ptrs);
4 row_ptes = row_fbrt.add_mem_str(a->ptrs+1);
5 row_grpt = BCast(row_fbrt);
6 //Load two row elements (and vec vals) in lockstep
7 //TUx1 have step=2, TU01 has offset=0, TU11 has offset=1
8 col_fbrt0 = RngFbrt(row_ptbs, row_ptes, 0, 2);
9 col_idx0 = col_fbrt0.add_mem_str(a->idxs);
10 nnz_vals0 = col_fbrt0.add_mem_str(a->vals);
11 vec_vals0 = col_fbrt0.add_mem_str(b, col_idx0);
12 col_fbrt1 = RngFbrt(row_ptbs, row_ptes, 1, 2);
13 col_idx1 = col_fbrt1.add_mem_str(a->idxs);
14 nnz_vals1 = col_fbrt1.add_mem_str(a->vals);
15 vec_vals1 = col_fbrt1.add_mem_str(b, col_idx1);
16 col_grpt = LockStep(col_fbrt0, col_fbrt1);
17 nnz_vals = col_grpt.add_vec_str(nnz_vals0, nnz_vals1);
18 vec_vals = col_grpt.add_vec_str(vec_vals0, vec_vals1);
19 //Set the row-end and row-ite callbacks and operands
20 col_grpt.add_callback(GITE, ri, {nnz_vals, vec_vals});
21 col_grpt.add_callback(GEND, re, {});

```

Figure 5.5: TMU configuration code for column-parallel SpMV. Colors match Figure 5.2: the top block programs the row-pointer traversal, the middle block programs the parallel nonzero traversal and dense-vector lookup, and the bottom block registers the row-iteration and row-end callbacks. Together, these blocks show how nested sparse traversal is expressed with TMU primitives while computation is exposed as callbacks.

5.2 TMU Microarchitecture

The TMU microarchitecture implements the programming interface by turning sparse tensor traversal and merging into a pipelined dataflow execution model. Its main purpose is to sustain many irregular memory requests, synchronize traversal across tensor levels and lanes, and produce ordered operand streams for the compute core without consuming out-of-order core resources. We first describe how Traversal Units iterate tensor fibers and produce synchronized data and control streams. We then describe how Traversal Groups merge or co-iterate lanes within each layer, how the memory subsystem supports many outstanding sparse accesses, and how the TMU stores architectural state for context switching. Finally, we walk through the execution of a column-parallel SpMV example to connect these mechanisms end to end.

5.2.1 Traversal Unit Design

Each TU contains a set of data streams and a control stream, all implemented as circular queues. Data streams carry iteration variables and values produced by memory or arithmetic operations, such as tensor values, tensor indices, memory addresses, and transformed indices or addresses. The binary control sequence marks fiber boundaries for the synchronized data streams: a 0 token denotes a regular iteration element, and a 1 token denotes the end of the current fiber.

All data streams within the same TU have the same depth and advance together with the TU control sequence through synchronized push and pull operations. Hence, each queue entry is computed from the corresponding entry at the same position in its parent stream, as defined in Section 5.1.2. This synchronization allows a TU to produce related streams in parallel; for example, in SpMV it can concurrently load column indices and nonzero values for the same CSR row entry.

With this organization, TUs implement the logic to (i) iterate tensor fibers, (ii) generate a binary control sequence to track the iteration status, and (iii) populate data streams with memory or arithmetic operations. To iterate a tensor fiber, each TU implements a finite-state machine (FSM) looping through the fbeg, fite, and fend states. The fbeg state initializes the iteration boundaries, which can either be constant values or read from a leftward TU, stalling the current TU execution if the leftward TU has not produced new valid data yet. If the streams are not full, the fite state pushes (i) a 0 token into the binary control sequence, (ii) the current iteration index into the ite stream, and (iii) a new element into each other stream (which is generated according to the stream type). As discussed below, the value of this element depends on its input queue. Finally, when the fiber traversal has no more elements to iterate, the fend state pushes a 1 token into the binary control sequence and returns to the fbeg state.

If the parent queue implements a mem stream, its value may not be available immediately because of memory latency, leaving some queue elements in a transient state. In particular, queue elements can be in either a valid or waiting state according to whether the corresponding element in the parent queue is valid. Elements of mem queues, instead, can be in a waiting, queued, requested, or valid state according to whether the parent index is not valid, the address has been computed, the request has been submitted to memory, or the data has returned, respectively. This per-entry state tracking is central to the TMU's dataflow design: independent queues can continue generating addresses and issuing memory requests while other queue entries wait for their operands or memory responses, allowing the TMU to sustain many outstanding sparse memory accesses.

5.2.2 Traversal Group Design

A Traversal Group (TG) corresponds to one TMU layer in Figure 5.1: it includes the TUs in that layer, which implement different lanes, and the associated merger logic. The TG implements the logic to merge and co-iterate these TUs. Similarly to TUs, TGs also implement an FSM to loop through *gbeg*, *gite*, and *gend* states, generating predicates and a control sequence. The output predicate is the layer predicate later exposed through the *msk* stream. The control sequence is used later on to trigger callbacks.

TG FSMs compute their control sequence by combining (i) the predicate of the previous layer, if any, and (ii) the control sequences of all the TUs in the layer, implementing a hierarchical evaluation. In particular, we consider a lane to be *active* only if the corresponding bit of the predicate coming out of the previous layer is set to true. TGs only process a *gite* state if all the active lanes have valid data in their queue heads.

In disjunctive merging, the *gite* state (i) computes the output predicate by setting the active lanes with minimum indices to 1, (ii) consumes them, and (iii) pushes a 0 token into the binary control sequence. When all active TUs have no more elements to merge disjunctively, the *gend* state pushes a 1 token into the binary control sequence.

In conjunctive merging, the *gite* state (i) computes the output predicate by setting active lanes with minimum indices to one, (ii) consumes them, and (iii) pushes a 0 token into the binary control sequence only if all active lanes have minimum indices (all-true predicate). When any active TU has no more elements to merge conjunctively, the *gend* state pushes a 1 token into the binary control sequence.

Finally, in lockstep co-iteration, the *gite* state (i) computes the output predicate by setting active lanes that have not finished iterating to one, (ii) consumes their heads, and (iii) pushes a 0 token into the binary control sequence. When all active TUs have no more elements to co-iterate, the *gend* state pushes a 1 token into the binary control sequence.

5.2.3 Output Queue Construction

The predicates and control sequences generated from each TG are then used to push callback IDs and operands into the *outQ*. Similarly to traversal and merging, *outQ* construction is implemented as an FSM running in each TG. However, while traversal and merging phases can be fully decoupled (i.e., each TU/TG iteration can start as soon as it has valid inputs), *outQ* generation needs to be serialized across TGs to preserve the order in which callbacks and operands are processed by the compute core (analogous to out-of-order execution and in-order commit implemented in today's processors). Hence, besides the *obeg*, *oite*, and *oend* states, *outQ* generation also requires *ow4p* and *ow4n* states to signal that a TG is waiting for

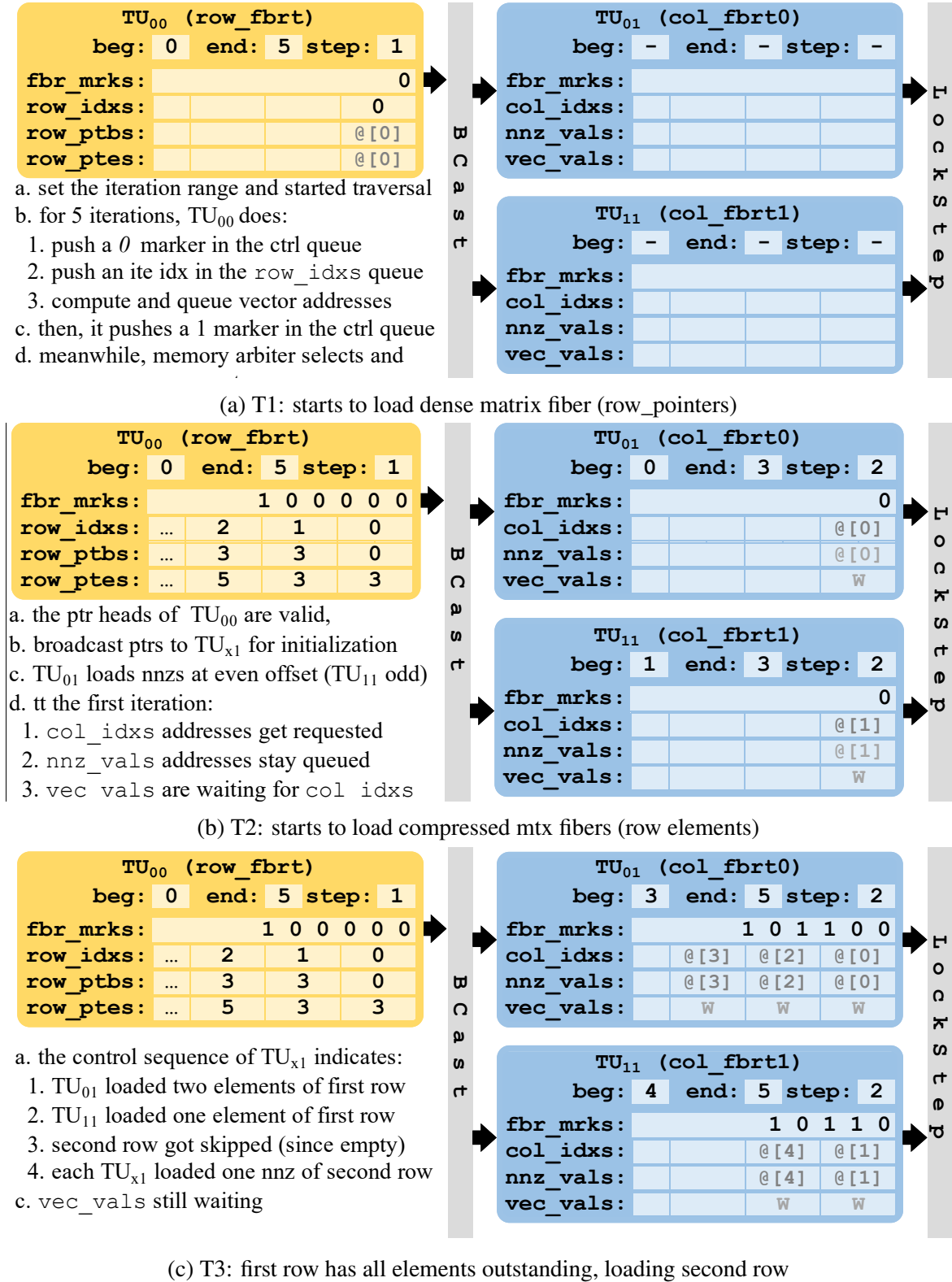
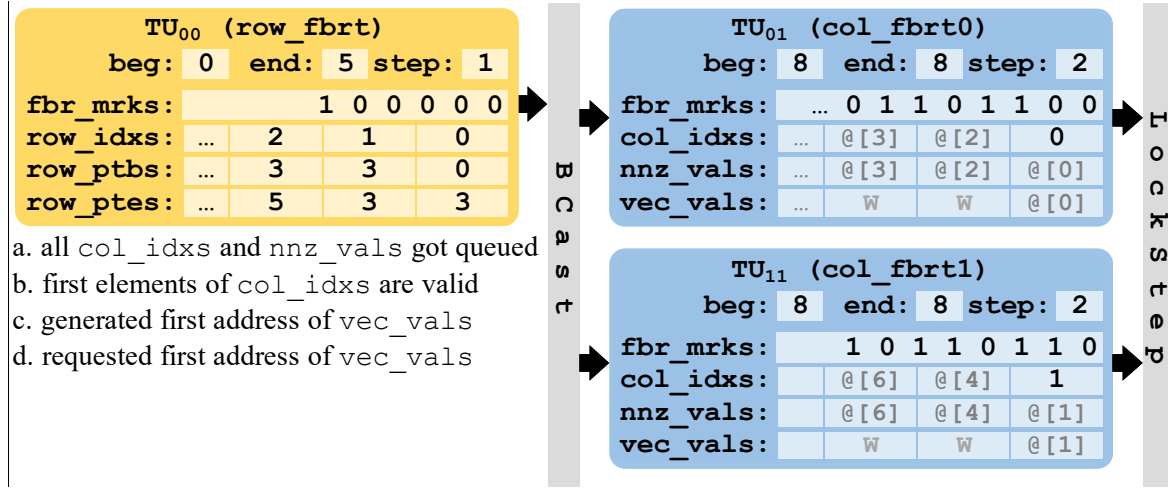
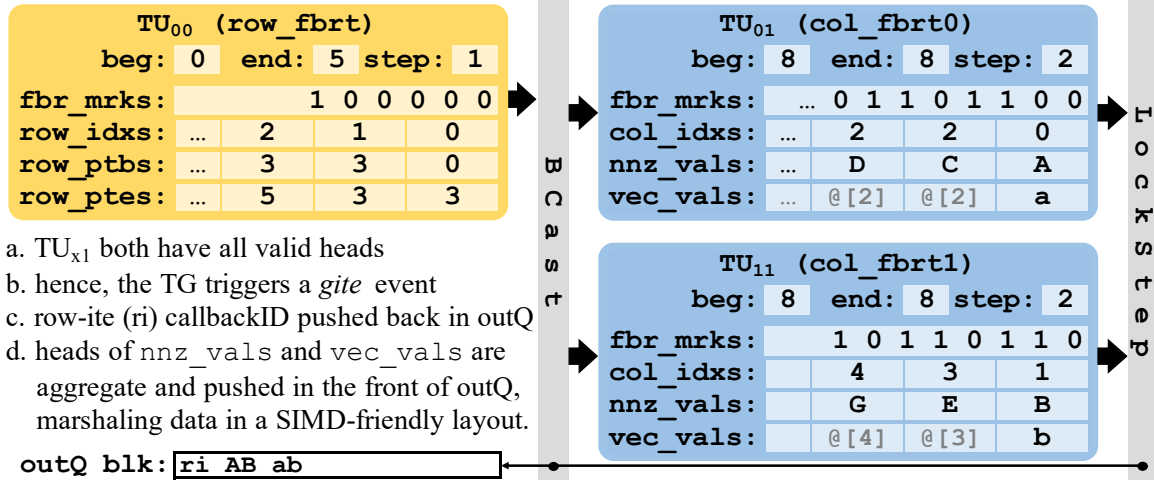


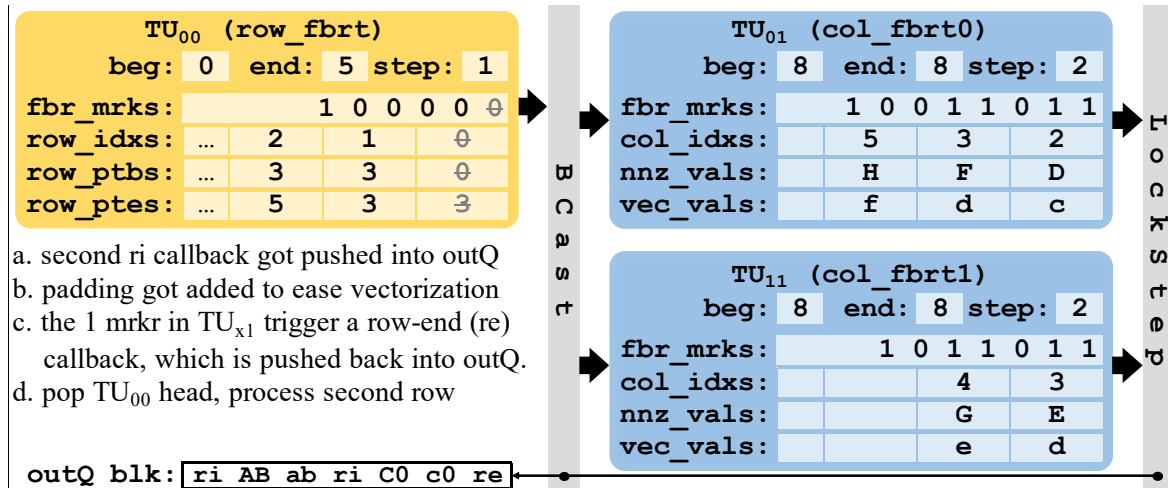
Figure 5.6: First part of the step-by-step TMU execution of SpMV with inner-loop vectorization (P1 in Table 5.4) over the CSR matrix in Figure 2.1. T1 loads the row-pointer fiber, T2 starts the compressed row traversal, and T3 has issued all requests for the first row while beginning the next row. This shows how the TMU exposes memory-level parallelism before operands are ready for computation.



(a) T4: starts to load dense-vector values (lookup operation)



(b) T5: some operands are ready to be queued into the outQ



(c) T6: all elements of the first row got queued into the outQ

Figure 5.7: Second part of the step-by-step TMU execution of SpMV with inner-loop vectorization (P1 in Table 5.4) over the CSR matrix in Figure 2.1. T4 issues dense-vector lookups, T5 begins marshaling ready operands into the outQ, and T6 has queued all operands for the first row. This shows how traversal results become ordered callback operands for the core.

the previous or next TG to push data into the outQ. When a TG is in a gbeg, gite, or gend state, it checks whether any callback is associated with that state and, if so, pushes its ID and operands into the outQ buffer. For performance reasons, both the outQ and outQ buffers are double-buffered.

5.2.4 Memory Arbiter

The TMU sends out memory requests at the cacheline granularity. Each cycle, the TMU memory arbiter selects at most one cacheline request by walking a fixed priority hierarchy. It first prioritizes requests from the leftmost layers, which correspond to outer loops, then selects among TUs within the chosen layer in round-robin order, then selects streams within the chosen TU in configuration order, and finally selects requests within the chosen queue in FIFO order. TUs within the same layer snoop their memory requests and upgrade the status of their queued elements if their address has been requested by another TU.

By default, memory requests within a stream are spawned *eagerly*, meaning a cacheline can be requested as soon as an address for that cacheline is computed (queued). To reduce memory traffic, certain layers can adopt a *lazy* loading strategy. Under lazy loading, a queued cacheline request is held while subsequent entries in the same queue continue to target the same cacheline; the request is issued only when the queue reaches an entry for a different cacheline or when the queue must make progress. This coalesces multiple entries that access the same cacheline into a single memory request, which is useful when a queue produces data more slowly than the memory latency.

5.2.5 TU Queue Sizing

Queue storage is provisioned from shared per-lane storage at configuration time. Although TUs expose queues of fixed width internally, different logical layers require different depths: outer layers mostly produce loop bounds, whereas inner layers issue most memory requests and merges. The TMU therefore allocates shorter queues to lightly used leftmost layers and deeper queues to heavily used rightmost layers, coalescing unused physical storage across TUs in the same lane. An analytical model sizes these queues using statically estimated fiber lengths and nonzero counts.

5.2.6 System Integration

Placement: As shown in Figure 5.1, each DAE core in our DAE multicore system features a TMU and a compute core. The TMU traverses fibers by loading data from the LLC and

marshals the data into the outQ, which is written into the compute core’s private L2 cache. Data from fiber traversals is unlikely to be reused in private caches; therefore, the TMU bypasses the private L1 and L2 caches for traversal reads and issues them directly to the shared LLC. This avoids consuming the small number of private-cache MSHRs and instead uses the larger, banked miss-handling capacity available at the LLC, enabling higher memory-level parallelism for sparse traversal. Each outQ is core-private; therefore, injecting it into the L2 cache enables faster compute throughput.

Memory subsystem integration: Like prior general-purpose processor engines, the TMU operates with virtual addresses and uses the host core’s address translation hardware (MMU) [161, 71, 5, 99, 163]. In particular, it queries the L2 TLB. A hit returns the physical address directly; a TLB miss is handled by the existing page-table walker and stalls only the corresponding TMU memory request, while other independent TMU requests can continue to make progress. We assume the TMU shares the host core’s L2 TLB, so TMU traversal accesses may compete for L2 TLB capacity similarly to ordinary core loads. If the translation results in a page fault, the TMU interrupts the core so the OS can handle the fault. Once the missing translation is available, the MMU signals the TMU to retry the memory access.

The TMU operates decoupled from the compute core by issuing coherent read-only traversal requests and writing the resulting outQ into the compute core’s private L2 cache. The outQ is a core-private producer-consumer buffer: the TMU writes it and the compute core reads it, but no other DAE core observes or modifies it. Thus, the TMU does not introduce new shared read-write data across DAE cores beyond ordinary coherent cache behavior.

Context switching and exceptions: TMU architectural state must be saved and restored when a thread is context-switched. When the OS deschedules a thread that uses the TMU, it quiesces the TMU, saves its context, and restores it when the thread is rescheduled. On exceptions or syscalls that return to the same thread without descheduling it, the OS does not need to save TMU state. If handling the event deschedules the thread, the normal context-switch path quiesces and saves the TMU before another thread uses it. Quiescing the TMU requires bringing its dataflow queues to a restartable state. The TMU stops issuing new memory requests and rolls each stream back to the last position at which all dependent queue entries are valid, which can be identified by walking the stream dependencies backwards. Any outstanding requests beyond that restart point are discarded or replayed after restore; once the queues contain only valid restartable entries, the TMU context can be saved. After quiescence, the minimum context state that needs to be saved is the initial TMU configuration (e.g., queue types and sizes, beg and end iteration boundaries), the head of each TU ite stream, and some control registers such as the base outQ address and current writing offset. The memory-mapped outQ is private per thread. Fortunately, support for efficiently

saving large state is already implemented in the Linux kernel via on-demand expansion of per-task context switch buffers. This is used by the new Intel Advanced Matrix eXtensions (AMX) [62, 101], which employ tile registers that already occupy 8KB in current implementations.

5.2.7 TMU Execution Example

Figure 5.6 and Figure 5.7 depict a step-by-step example for SpMV (Figure 5.2) where the TMU has been programmed using the code in Figure 5.5. The example loads the CSR matrix shown in Figure 2.1 and assumes a two-lane design.

The remainder of this chapter demonstrates the performance potential of the TMU for accelerating sparse tensor algebra kernels in scientific and machine learning models.

5.3 Scientific Sparse Tensor Algebra Evaluation

The scientific evaluation demonstrates that the TMU accelerates diverse sparse tensor algebra kernels by removing traversal and merging from the critical path of the CPU core. Across memory-intensive, compute-intensive, and merge-intensive workloads, the TMU improves performance by exposing memory-level parallelism, reducing control overhead, and feeding SIMD-friendly operands to the core. Section 5.3.1 first describes the methodology for this study. Sections 5.3.2 and 5.3.3 compare the performance of a multicore DAE processor composed of TMU–CPU cores against a traditional CPU baseline, and analyze how these speedups translate into system utilization. Sections 5.3.4 and 5.3.5 study sensitivity to TMU parameters and compare against representative prefetching and scalar traversal-engine designs. Section 5.3.6 closes the scientific evaluation by quantifying the area cost of the TMU; Section 5.4 then evaluates the same decoupling principle on machine learning embedding operations. Both evaluations use the same DAE processor organization, but adjust bandwidth per core to match the target processor class: HPC-class HBM systems for scientific sparse tensor algebra [126] and datacenter-class HBM systems for embedding operations [121]. Each configuration is detailed in its corresponding methodology section.

5.3.1 Scientific Evaluation Methodology

Simulation Infrastructure

We evaluate the resulting DAE processor using the gem5 v20.1.0.0 simulator [89]. The DAE multicore system models an HBM-based processor with bandwidth per core comparable to

Table 5.5: Specifications of the simulated DAE multicore processor used for the scientific sparse-tensor evaluation. The DAE system combines high-performance DAE cores each composed of a TMU and an Arm-like core, an HBM-backed memory hierarchy, and a mesh interconnect. The TMU and LLC capacities are balanced at the system level: the sum of outstanding-request slots across TMUs matches the sum of MSHRs across LLC slices.

Cores	8 Neoverse-N1-like out-of-order cores at 2.4GHz
SVE width	512 bits
Reorder buffer	224 entries
Load/Store queues	96 entries, 96 entries
Private L1 I&D	64 KiB/core, 4-way, 2 cycle data access, 32 MSHRs
L1I prefetcher	Tagged
L1D prefetcher	Stride, degree 2
Private L2	512 KiB/core, 8-way, 8 cycle data access, 64 MSHRs
L2 prefetcher	Best Offset Prefetcher
Shared L3 (LLC)	Mostly exclusive, 8 slices of 1MiB, 16 ways, 12 cycles data access, 128 MSHRs
Coherence protocol	MOESI-like AMBA 5 CHI specification
Network topology	4×4 2D mesh, 1 cycle routers, 1 cycle links
Memory	4 HBM2e channels, FR-FCFS, 37.5GB/s per channel
TMU	2KB per-lane storage, 8 lanes, 4 TGs with mergers 128 outstanding requests

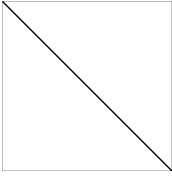
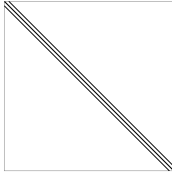
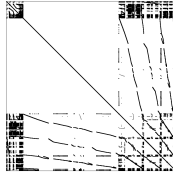
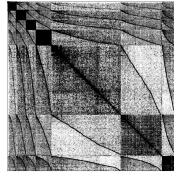
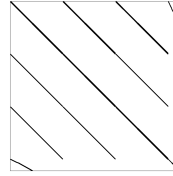
commercial HBM-based HPC processors [126]. Each core integrates one TMU to offload sparse tensor traversal and merging. The simulated system runs Ubuntu 20.04 with Linux kernel 5.4.65. Table 5.5 summarizes the architectural parameters.

Workloads

We evaluate our DAE proposal on several sparse linear and tensor algebra kernels and real-world applications with different characteristics. We employ high-performance multi-threaded software baselines that we either generated with the well-known TACO compiler [70] or extracted from existing reference libraries [118, 85]. We vectorized all baselines with Arm SVE and utilized vector gather and scatter instructions where appropriate. In particular, we evaluate the following kernels from Table 5.4:

- **SpMV CSR** [157]: Software baseline from TACO vectorized with Arm SVE. The TMU implements inner-loop vectorization (P1).

Table 5.6: Inputs for the scientific sparse tensor workloads. The table lists SuiteSparse matrices for sparse linear algebra kernels and FROSTT tensors for sparse tensor kernels, along with nonzero counts, dimensions, domains, and matrix-structure visualizations. These inputs span different sparsity patterns and domains, stressing traversal, lookup, and merging in different ways.

Id	Matrix [33]	nnzs	rows	nnzs/row	Domain
M1	af_0_k101	17.6M	504K	~35	structural
M2	atmosmodm	10.3M	1.5M	~7	fluid dynamic
M3	Freescall	17.1M	3.4M	~5	circuit simulation
M4	gb_osm	13.3M	7.7M	~2	graph
M5	halfb	12.4M	225K	~55	structural
M6	testl	9.4M	393K	~24	semiconductor
Id	Tensor [143]	nnzs	Dimensions	Domain	
T1	Chicago-crime	5M	6K x 24 x 77 x 32	count	
T2	LBNL-network	2M	2K x 4K x 2K x 4K x 1M	network	
T3	NIPS pubs	3M	3K x 3K x 14K x 17	text	
T4	Uber pickups	3M	183 x 24 x 1K x 2K	map	
M1	M2	M3	M4	M5	M6
					

- **SpMSpM CSR** [44]: Implements $Z = AA^T$. Software baseline from TACO vectorized with Arm SVE. The TMU implements inner-loop vectorization (P2).
- **SpKAdd DCSR** [59]: Software baseline from TACO ($k=8$) vectorized with Arm SVE. To preserve domain characteristics, input matrices (A^x with $x = 1, \dots, k$) are generated by cyclically distributing rows of the original matrix (A) to each input matrix (i.e., i -th row of input $A_i^x = A_{i*k+x}$).
- **MTTKRP COO** [118]: Performs a Matricized COO Tensor Times Khatri-Rao Product with the permutation optimization proposed by Phipps et al. [118]. The TMU implements parallelism at the mode (P1) and rank (P2) levels.
- **SpTC CSF** [85]: Performs a tensor contraction of two CSF tensors with the expressions evaluated by Liu et al. [85]. We evaluate the symbolic phase to limit simulation time.

Moreover, we evaluate the following real-world applications:

- **PR**: Performs the PageRank algorithm as implemented in the GAP Benchmark suite [16] (Jacobi-style method), optimized with Arm SVE.

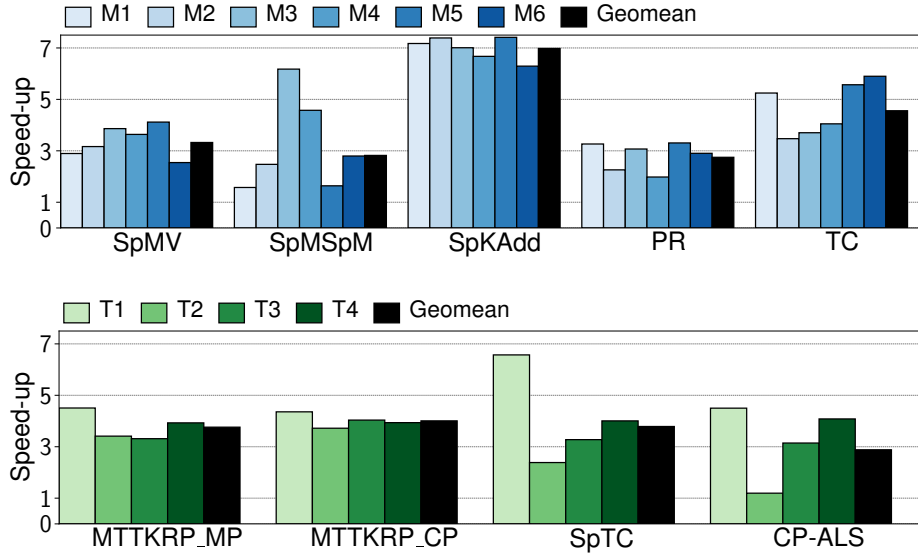


Figure 5.8: Speedup of the proposed DAE processor over the traditional CPU baseline for scientific sparse tensor algebra workloads. Both systems use the simulated multicore processor described in Table 5.5; the DAE processor augments each core with a TMU, while the baseline executes the optimized software implementations described in Section 5.3.1. The top panel reports sparse linear-algebra workloads, and the bottom panel reports sparse tensor workloads; each bar corresponds to one input matrix or tensor from Table 5.6. Overall, the DAE processor achieves 3.58 $\times$, 2.82 $\times$, and 4.94 $\times$ geomean speedups on memory-intensive, compute-intensive, and merge-intensive workloads, respectively, showing that decoupling traversal and merging is most effective when memory access or merge control dominates execution.

- **TC**: Counts the triangles in a graph as in the fused GraphBLAS version [23] based on masked SpMSpM [95].
- **CP-ALS** [118]: Performs a canonical polyadic decomposition of a COO tensor with an alternating least squares algorithm, from the GenTen framework [118].

We run these applications with the settings suggested by the authors. Table 5.6 describes the inputs we used for all linear and tensor algebra applications, from SuiteSparse [33] and FROSTT [143], respectively.

5.3.2 Scientific Performance Analysis

Figure 5.8 reports the performance speedups achieved by the proposed DAE processor against a traditional one over the software baselines detailed in Section 5.3.1. Overall, the proposed DAE system achieves geomean performance speedups of 3.58 $\times$, 2.82 $\times$, and 4.94 $\times$

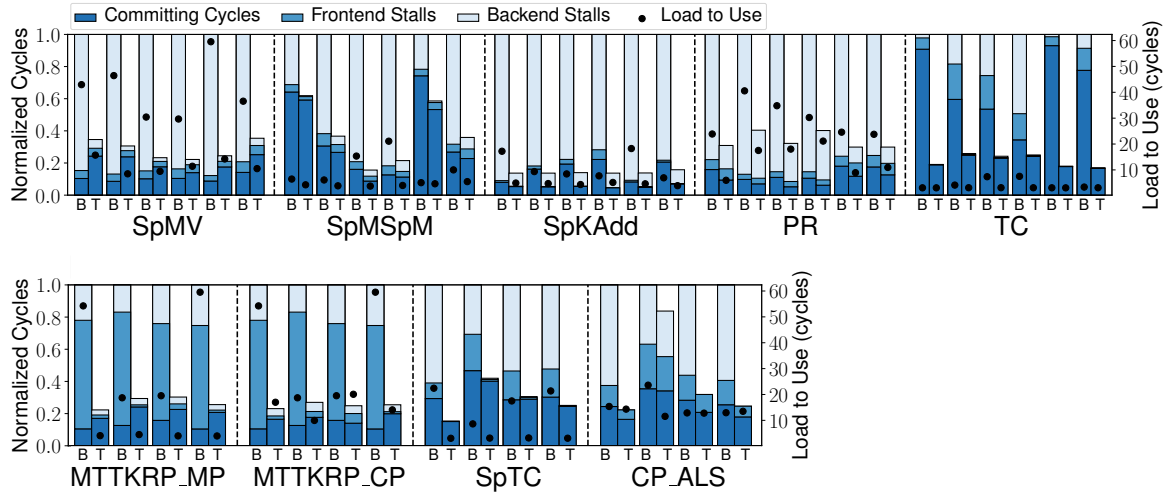


Figure 5.9: Normalized cycle breakdown of the traditional CPU baseline and the proposed DAE processor on the scientific workloads described in Section 5.3.1. Both systems use the simulated multicore configuration in Table 5.5; the DAE processor augments each core with a TMU. Groups of bars correspond to workloads, and each pair compares one input on the DAE processor (T, left) against the CPU baseline (B, right). Stacked bars report normalized cycles spent committing instructions, stalled in the frontend, or stalled in the backend on the left y-axis; dots report average load-to-use latency on the right y-axis. The breakdown shows that the TMU primarily improves performance by reducing backend stalls and load-to-use latency in traversal-heavy workloads, while also reducing frontend and commit overheads in merge-heavy workloads.

on memory-intensive (SpMV, PR, MTTKRP, CP-ALS), compute-intensive (SpMSpM), and merge-intensive (SpKAdd, TC, SpTC) workloads, respectively. Figure 5.9 shows, for each evaluated workload and input, the normalized cycles spent (left y-axis) (i) committing at least one instruction, (ii) on frontend (fetch) stalls, or (iii) on backend (commit) stalls. On the right y-axis, we show average load-to-use latency as seen by the cores.

SpMV and PR exhibit similar performance regimes, with geomean speedups of $3.32\times$ and $2.74\times$, respectively. PR attains slightly lower performance since the weight update is not accelerated by the TMU. Figure 5.9 shows that offloading tensor traversal and merging to the TMU drastically reduces backend stalls, the main bottleneck of these memory-intensive workloads. In addition, we observe a sharp reduction in load-to-use latency, e.g., from 67 to 23 cycles for M1 in SpMV. We observe similar behavior on the memory-intensive tensor workloads (MTTKRP_MP, MTTKRP_CP, and CP-ALS), which obtain geomean speedups of $3.76\times$, $4.01\times$, and $2.88\times$, respectively. Overall, the TMU effectively hides long-latency misses on memory-intensive workloads [38].

The proposed DAE system also achieves speedups from 1.58 $\times$ to 6.18 $\times$ on SpMSPM (2.82 $\times$ geomean). SpMSPM is a compute-intensive workload, as shown by the large portion of committing cycles in Figure 5.9. However, the time spent on computation is input-dependent, i.e., matrices with more nonzeros per row perform more computation. For input matrices with a larger portion of committing cycles (i.e., M1 and M5), offloading tensor traversal and lookups to the TMU does not provide large performance speedups due to Amdahl’s law, but it still reduces backend stalls to a minimum. In contrast, for matrices that have a lower portion of committing cycles (i.e., M3 and M4), it delivers 6.18 $\times$ and 4.58 $\times$ speedups, respectively.

On merge-intensive workloads such as SpKAdd, TC, and SpTC, the DAE processor achieves speedups of 6.98 $\times$, 4.56 $\times$, and 3.79 $\times$, respectively. SpKAdd presents significant speedups for two main reasons. First, the TMU enables parallel loading of all eight matrices in a dataflow manner, unlocking memory-level parallelism that significantly reduces backend stalls. Second, it offloads merging, almost eliminating frontend stalls and reducing committing cycles (4 $\times$ reduction on M4), since the compute core has less work to do. TC relies heavily on merging, experiencing a large portion of frontend stalls and committing cycles. In this case, offloading merging to the TMU again almost completely eliminates frontend stalls and drastically reduces the amount of merging work performed by the core, greatly reducing committing cycles. We observe similar behavior in SpTC. Overall, workloads that perform merging operations greatly benefit from the TMU’s ability to co-iterate multiple fibers, merge them in place, and produce a SIMD-friendly outQ for the core to process.

5.3.3 System Utilization

To explain the improvements the TMU brings in terms of system utilization, we present Roofline models [158] in Figure 5.10. Roofline models relate operational intensity (x-axis), measured by the ratio of total floating-point operations to total data movement, and performance in giga-flops (y-axis). The diagonal line draws the peak memory bandwidth, while the horizontal line is the processor’s peak performance.

Figure 5.10a shows the evaluated workloads using the geomean across inputs; TC (integer arithmetic) and SpTC (symbolic phase) are omitted because they do not perform floating-point computation. Baseline CPU-only implementations are unable to use a substantial amount of memory bandwidth, attaining poor system utilization and compute throughput. The DAE processor, instead, achieves performance close to the memory bandwidth roof, and SpMV almost saturates peak bandwidth, achieving significantly higher compute throughput. SpMSPM is unable to use as much bandwidth as the other workloads due to its compute-intensive nature.

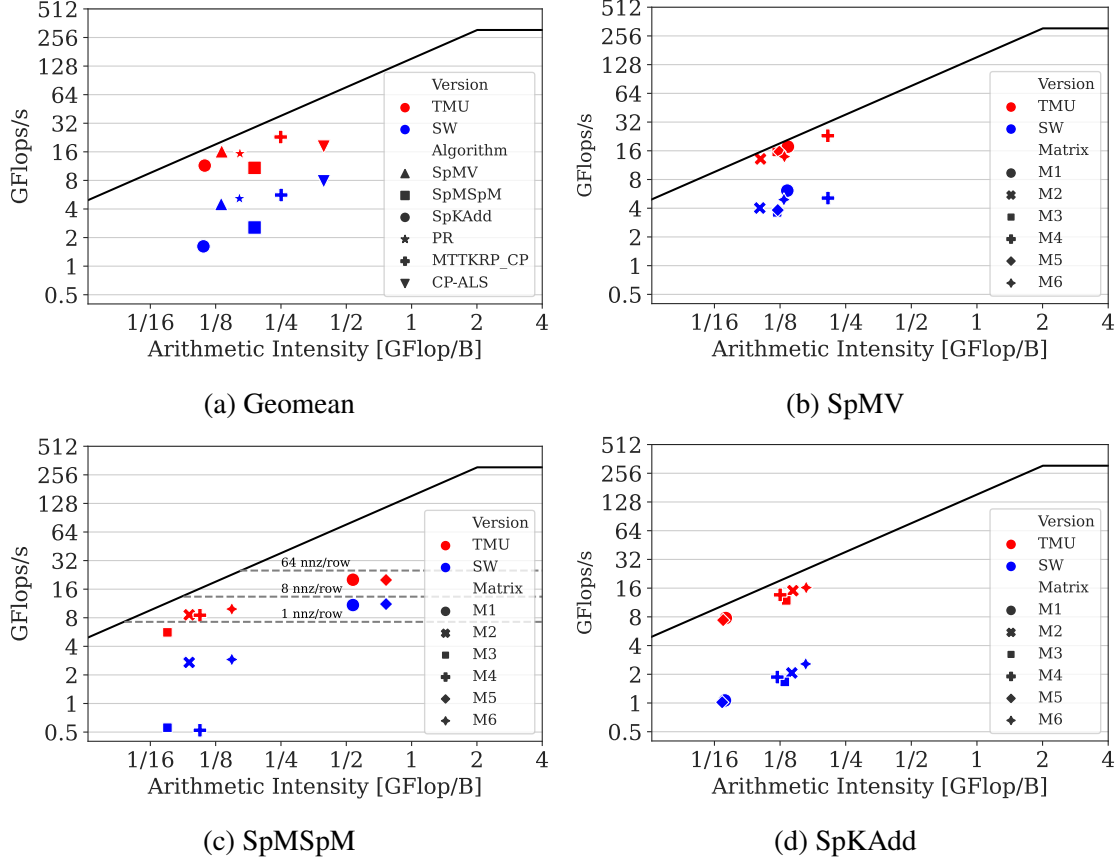


Figure 5.10: Roofline analysis of the traditional CPU baseline and the proposed DAE processor on scientific sparse kernels. Both systems use the simulated multicore configuration in Table 5.5; the DAE processor augments each core with a TMU, while the baseline executes the optimized software implementations described in Section 5.3.1. Each point reports one execution, with operational intensity on the x-axis and achieved floating-point throughput on the y-axis; diagonal and horizontal roofs denote the memory-bandwidth and peak-compute limits, respectively. Figure 5.10a reports geomeans across inputs for each workload, while Figures 5.10b to 5.10d report all inputs for SpMV, SpMSpM, and SpKAdd; the dashed ceilings in Figure 5.10c show idealized SpMSpM limits for fixed nonzeros per row. Overall, the TMU does not change operational intensity, but it moves executions closer to the relevant bandwidth or compute roof by improving memory-level parallelism and reducing traversal and merging overheads.

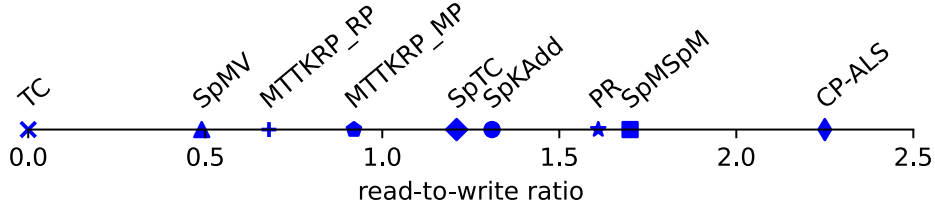


Figure 5.11: Read-to-write ratio of the proposed DAE processor on the scientific workloads described in Section 5.3.1. For each outQ block, the ratio compares the time the compute core spends consuming and processing the block against the time the TMU spends producing it, averaged across blocks. Values below one indicate that the core consumes blocks faster than the TMU produces them, so traversal or merging remains the bottleneck; values above one indicate that the TMU produces blocks faster than the core consumes them, so computation becomes the bottleneck. SpMV, MTTKRP, and TC are TMU-limited, SpKAdd and SpTC are close to balanced, and SpMSPM, PR, and CP-ALS are core-limited. Thus, the read-to-write ratio exposes whether each workload needs more access-side throughput, more compute-side throughput, or a balanced DAE pipeline.

The rest of the Roofline models focus on SpMV, SpMSPM, and SpKAdd, showing data for all inputs. For SpMV (Figure 5.10b), which is memory-intensive, the baseline CPU-only version cannot use the available memory bandwidth because data dependencies on long-latency memory accesses quickly fill out-of-order core resources and stall execution [38]. The dataflow design of the TMU unlocks additional memory-level parallelism for offloaded traversals, increasing memory bandwidth utilization by almost 4 $\times$ and leading to much better system-level performance. We observe similar behavior in the CPU-only baseline versions of SpKAdd (Figure 5.10d), where the additional pressure that merging puts on core resources leads to even lower performance. The TMU again enables near-peak bandwidth utilization, which, combined with its merging support, significantly boosts overall performance.

Finally, for SpMSPM we also plot three additional dashed ceilings (Figure 5.10c). We compute these ceilings by processing synthetic matrices, each storing a fixed number of nonzeros per row ($n = \{1, 8, 64\}$) located at column indices $\{0, \dots, n - 1\}$, to approximate performance under ideal spatio-temporal locality. The data points starting from the right correspond to inputs M5 and M1, respectively. These two matrices have higher nonzero counts per row (see Table 5.6) and perform better on the CPU-only baseline than other inputs due to higher operational intensity. DAE executions improve performance and bandwidth utilization, but performance is constrained by the inherent nonzeros-per-row compute ceilings.

The compute core and the TMU work in a decoupled pipelined manner. Therefore, we introduce the *read-to-write ratio*, defined as the ratio between the time it takes the compute core to process (read) an outQ block and the time it takes the TMU to produce (write) it, averaged over all blocks. Figure 5.11 shows this ratio for the evaluated workloads. A value

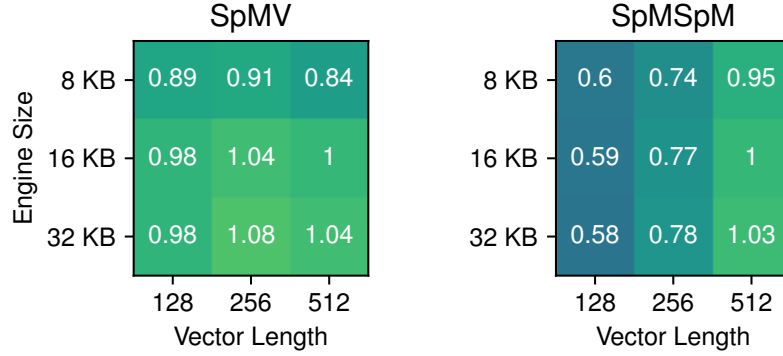


Figure 5.12: Sensitivity of scientific workloads to TMU storage capacity and SVE vector length. Each cell reports speedup normalized to the default DAE configuration used in the main evaluation, with 16KB of TMU storage and 512-bit SVE vectors. SVE length also determines the number of TMU lanes: 256-bit SVE corresponds to four lanes, while 512-bit SVE corresponds to eight lanes. SpMV benefits primarily from larger TMU storage, which enables more outstanding traversal requests and improves memory-latency tolerance, whereas SpMSPM benefits more from wider SVE vectors because its read-to-write ratio makes the compute core the bottleneck. Overall, the sensitivity study shows that TMU storage is most important for memory-limited traversal, while vector width matters most when computation dominates.

below one implies the compute core is faster than the engine. We observe this behavior for TC, since merging is offloaded to the TMU, and for SpMV and MTTKRP, where computation is very regular due to the SIMD-friendly data layout produced by the TMU. SpKAdd and SpTC have ratios close to one, which indicates balanced execution. SpMSPM, PR, and CP-ALS have higher ratios due to more compute and less regular computation, indicating that the bottleneck is on the compute core side.

Our evaluation demonstrates that the TMU successfully accelerates tensor traversals and merging operations, thereby improving overall system utilization. Moreover, performing computation on the core enables additional flexibility to seamlessly express and accelerate workloads that implement kernel fusion (TC) and that require evaluating partial results (CP-ALS).

5.3.4 TMU Sensitivity Analysis

Figure 5.12 shows a sensitivity analysis varying the total engine size and SVE vector length, which is tied to the number of TMU lanes. That is, a system with 512-bit SVE has an 8-lane TMU design, while a system with 256-bit SVE has a 4-lane TMU design. The heatmap shows normalized speedup with respect to the evaluated 16KB 512-bit SVE configuration.

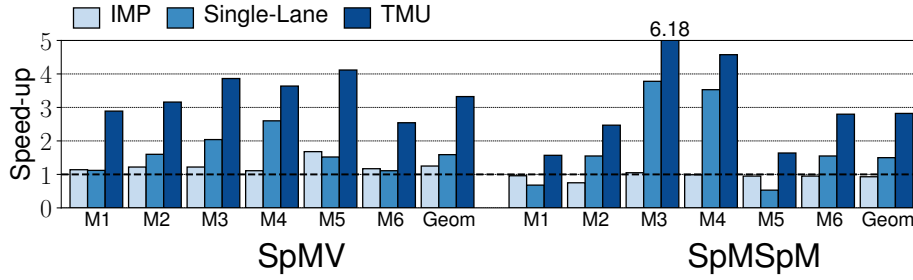


Figure 5.13: Comparison of the proposed TMU against representative prior access-side techniques on scientific sparse kernels. Bars report speedup over the same traditional CPU baseline and optimized software implementations used in Section 5.3.1. *IMP* augments the baseline with the Indirect Memory Prefetcher [163]; *Single-Lane* augments each core with a single-lane TMU with the same storage as the full design, proxying scalar traversal engines from prior work; and *TMU* uses the full multi-lane TMU configuration from Table 5.5. *Single-Lane* improves over the baseline but remains below the full TMU, which reaches $3.32\times$ and $2.82\times$ geomean speedups on SpMV and SpMSpM, respectively. Thus, decoupled traversal helps even in scalar form, but multi-lane parallel data loading is necessary to approach the full performance potential.

We observe that SpMV is more sensitive to engine storage, as larger storage enables more memory-level parallelism and hides memory latency, which is the main bottleneck in SpMV. SVE length does not have a large impact, since SpMV has a read-to-write ratio of 0.5. In SpMSpM, we observe the opposite behavior: wider SVE length has a large performance impact, since the bottleneck is on the core side (1.68 read-to-write ratio).

5.3.5 Comparison with Prior Sparse Architectures

The graph-oriented traversal engines we compare against are integrated into multicore systems and are primarily designed for graph analytics [161, 99]. They feature architectures similar to a single TMU lane, but with fewer primitives (i.e., stream types) and no support for parallel data loading or merging, limiting their applicability to simple linear algebra kernels. Two prominent examples are HATS [99] and SpZip [161]: the former exploits locality in graph communities, and the latter implements de(compression) techniques. Both optimizations are orthogonal to TMU operation. Therefore, we evaluate a *Single-Lane* TMU with the same storage as the multi-lane *TMU* employed throughout the evaluation. Figure 5.13 shows that *Single-Lane* achieves $1.59\times$ and $1.50\times$ geomean speedups on SpMV and SpMSpM, while *TMU* achieves $3.32\times$ and $2.82\times$, respectively. This indicates that graph-oriented traversal engines would also benefit from parallel data loading.

We also evaluate the Indirect Memory Prefetcher (IMP) [163], configured as recommended by the paper’s authors, including the use of virtual addresses to prefetch across memory page boundaries. IMP performs well on SpMV, with an average speedup of 1.25 $\times$, but fails to deliver performance improvements on SpMSPM due to thrashing of partial results. Overall, *Single-Lane* and *TMU* obtain better performance because they unlock more memory-level parallelism while always fetching useful data.

5.3.6 RTL Implementation and Area Analysis

We implement the TMU in System Verilog using the parameters listed in Table 5.5 and synthesize the design using the GlobalFoundries 22nm FD-SOI technology node. We have used the Cadence Genus tool v19.11 for the logical synthesis and the Cadence Innovus tool v19.11 for the place-and-route. The standard cell libraries use 8-track, low-Vt cells and a 104 CPP. We use high-performance dual-port SRAMs to implement the different data and metadata arrays inside the engine. The area cost of the TMU is 0.0704mm², where each lane occupies just 0.0080mm². The TMU requires 1.52% of the area of a Neoverse N1 core [116], the core modeled in our evaluation, when scaled to the same technology node [147]. Moreover, the entire sparse engine is 0.5 $\times$ smaller than a 32 KB, 8-way L1 data cache in 22nm technology.

5.4 Machine Learning Embedding Evaluation

The embedding evaluation shows that the same decoupling principle also addresses the dominant bottleneck in sparse machine learning workloads: irregular embedding lookup. By offloading lookup to the TMU while keeping accumulation on the core, the DAE processor increases memory-level parallelism, improves bandwidth utilization, and enables more energy-efficient organizations for embedding-heavy models. After Section 5.4.1 describes the methodology, Section 5.4.2 shows that decoupling lookup from computation improves memory-level parallelism and bandwidth utilization. Sections 5.4.3 and 5.4.4 then compare the resulting DAE processor against CPUs and GPUs, and Section 5.4.5 explores specialized DAE organizations that simplify or relocate the compute side for embedding-heavy workloads.

5.4.1 Embedding Evaluation Methodology

Figure 5.14 shows the DAE system we use to evaluate embedding operations. The system builds on the datacenter-class HBM multicore processor described in Section 4.2.1, whose

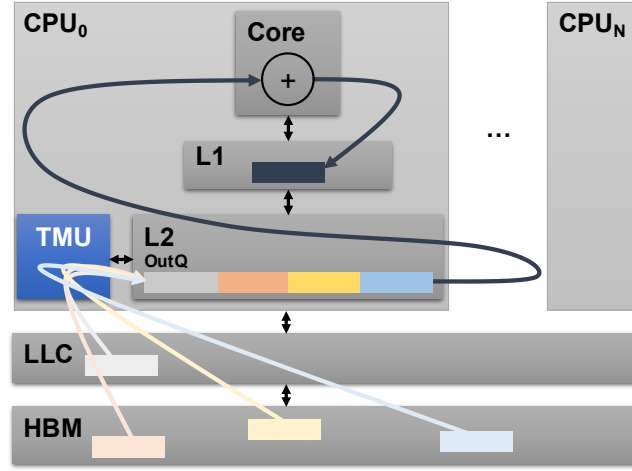


Figure 5.14: Embedding execution on the evaluated DAE processor, which augments the CPU baseline in Table 4.4 with one TMU per core. The TMU loads embedding vectors from the LLC or HBM and marshals them into the core’s private L2 cache, where the core accumulates them with SVE units and stores partial results. In this way, embedding lookup is offloaded to the TMU while reduction remains on the vector core.

HBM bandwidth is comparable to the Nvidia H100 GPU [107] and whose bandwidth per core and per-core performance are in the range of commercial HBM-based CPUs [121]. We augment each core with a TMU that issues and tracks embedding-vector lookups on behalf of the core. Because this datacenter-class DAE processor provides more bandwidth per core than the HPC-oriented DAE processor used in Section 5.3.1, we provision it with more TMU MSHRs. The actual number of TMU MSHRs is driven by the analytical study in Section 4.2.5, particularly the ideal request-tracking requirements reported in Figure 4.10. The GPU experimental methodology also follows Section 4.2.1, with GPU power measured using the `torch.cuda.power_draw` function.

5.4.2 Architectural Benefits of Decoupling

As with irregular memory accesses in scientific workloads, offloading embedding lookups to the TMU has several advantages. Because the TMU implements tensor traversal and loading with specialized dataflow hardware, it can issue memory requests with higher throughput and efficiency than the core. Moreover, because the TMU can run at a lower frequency, it can track $8\times$ more outstanding requests without timing issues and with low overhead (as discussed in Section 5.3.6). Figure 5.15 demonstrates that the TMU can track 82% of the outstanding memory requests required to achieve full throughput (one request per cycle), whereas the CPU can track only 16%.

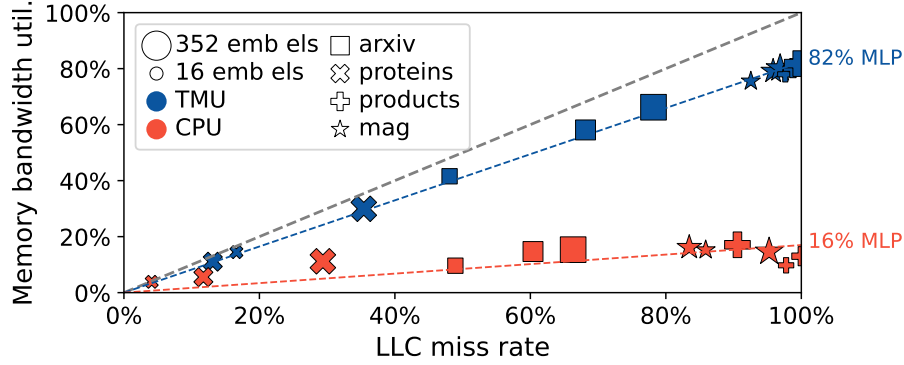


Figure 5.15: Memory-bandwidth utilization versus LLC miss rate for graph convolution layers on the CPU baseline in Table 4.4 and the TMU-augmented DAE system described in Section 5.4.1. Each point corresponds to one evaluated graph/layer configuration; marker shape identifies the input graph, marker size distinguishes the embedding-vector size, and color distinguishes CPU from TMU. The gray dashed line shows the ideal case in which every LLC miss can be serviced at peak memory throughput. The red and blue dashed lines are linear fits whose slopes estimate effective Memory-Level Parallelism (MLP) from Little’s rule [84]. The TMU reaches 82% of the ideal MLP, whereas the CPU reaches only 16%, showing that the TMU turns high miss rates into substantially higher memory-bandwidth utilization.

This additional memory-level parallelism translates directly into throughput and energy efficiency. As shown in Figure 5.16a and Figure 5.16b, the TMU achieves 5.7× higher requests/s and 5.6× higher requests/s/W than traditional cores. It also outperforms a core with doubled outstanding-request resources by 5.2× in requests/s and 6.3× in requests/s/W (Figure 4.11).

Finally, these per-core improvements scale to system-level bandwidth utilization. As shown in Figure 5.16c, the TMU uses 4×—8× more memory bandwidth than a traditional core. Consequently, Figure 5.17 shows that DAE architectures (1) achieve higher performance with the same core count, and (2) require fewer cores to achieve the same performance, improving efficiency.

5.4.3 TMU-Enabled DAE Processor vs. CPU

From the core’s perspective, offloading embedding lookup allows the core to fully utilize its resources for compute operations such as embedding reductions. Overall, as shown in Figure 5.18, decoupling embedding lookup from computation on multicore processors achieves a 2.9—8.2× speedup on GNN embedding operations, with speedups mostly proportional to their spatio-temporal locality. Moreover, as shown in Figure 5.19, this also improves performance by an average 5.8× across all embedding operations in Section 4.2.2.

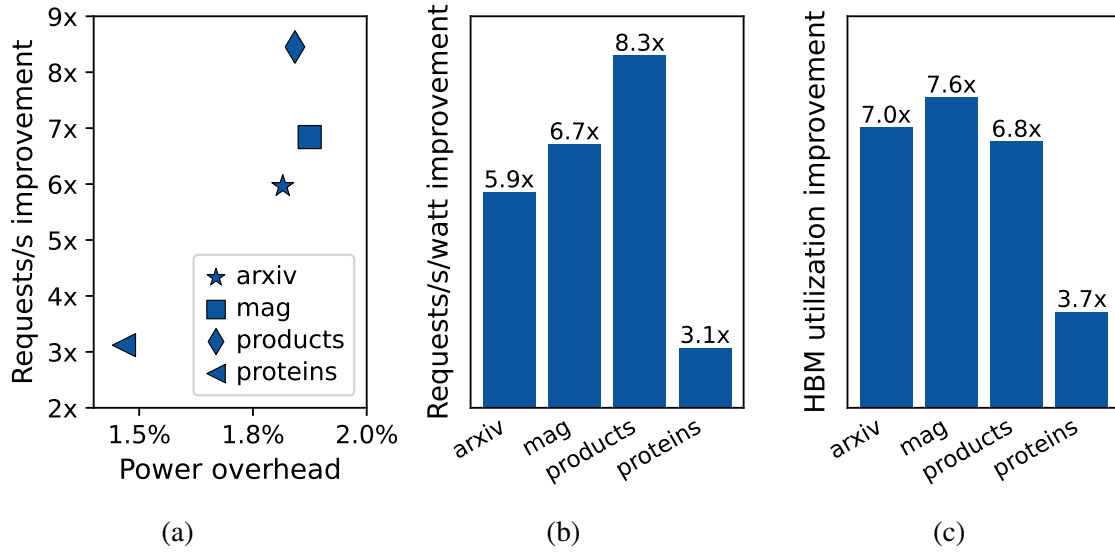


Figure 5.16: Per-core benefits of offloading graph-convolution embedding lookups from the CPU baseline in Table 4.4 to the TMU-augmented DAE system described in Section 5.4.1. Subfigure (a) plots throughput improvement in requests/s against TMU power overhead for each input graph; marker shape identifies the graph. The TMU adds less than 2% power overhead while improving request throughput by roughly 3.1x–8.5x. Subfigure (b) reports the corresponding requests/s/W improvement, which ranges from 3.1x on proteins to 8.3x on products. Subfigure (c) reports HBM-utilization improvement, ranging from 3.7x on proteins to 7.6x on mag. Overall, the TMU increases memory-request throughput with little power overhead, translating into higher per-watt efficiency and substantially higher HBM utilization.

5.4.4 TMU-Enabled DAE Processor vs. GPU

Figure 5.20a shows the inference latency breakdown of GNN models on the DAE processor and an NVIDIA H100 GPU [107]. As described in Section 5.4.1, the DAE system is dimensioned to match the H100’s peak HBM bandwidth. Despite this matched memory bandwidth, the DAE system utilizes 4.6x higher bandwidth on embedding operations, executing them 1.8x faster than the GPU. Hence, the DAE processor achieves 2.9x higher end-to-end performance than the GPU. As the DAE processor saturates memory bandwidth with only 8–16 cores per HBM stack, it consumes less power than the GPU, and achieves 4.1x higher performance/watt, as shown in Figure 5.20b.

5.4.5 Specialized TMU-Enabled DAE Architectures

Once embedding lookup is decoupled from computation, the compute side no longer needs to be a large out-of-order core for every workload. Building on the embedding evaluation in

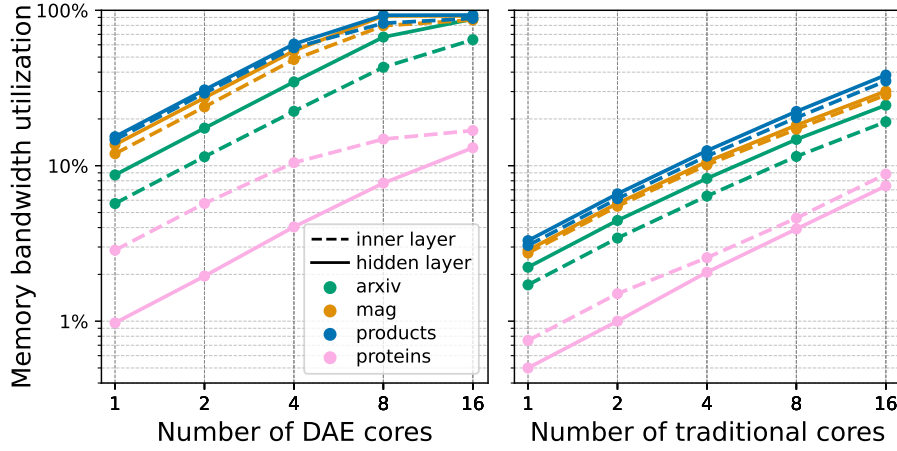


Figure 5.17: Memory-bandwidth utilization of a single HBM stack when scaling the number of DAE cores (left) and traditional CPU cores (right) for graph-convolution embedding lookups on the processor baseline in Table 4.4. The x-axis reports the number of cores per HBM stack, and the y-axis reports HBM bandwidth utilization on a logarithmic scale. Colors identify input graphs; dashed lines denote inner layers with smaller embedding vectors, and solid lines denote hidden layers with larger embedding vectors. DAE cores reach much higher bandwidth utilization with the same core count: high-miss-rate inputs approach HBM saturation with 8–16 DAE cores, whereas traditional cores remain far below saturation even at 16 cores. Overall, TMU-enabled DAE cores require fewer cores to reach a given bandwidth level and achieve substantially higher utilization at the same core count.

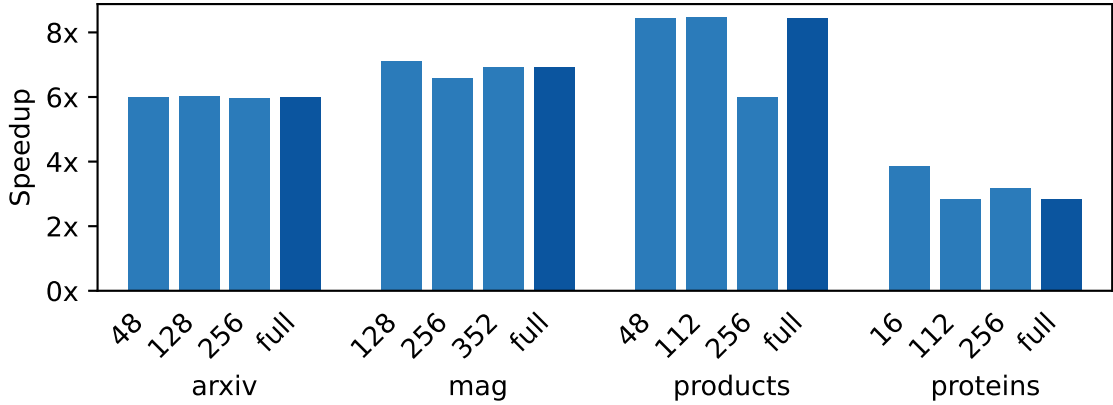


Figure 5.18: Speedup of graph-convolution embedding operations on the TMU-augmented DAE system described in Section 5.4.1 vs. the CPU baseline in Table 4.4. Groups correspond to the input graphs in Table 4.6. Within each group, bars labeled by embedding-vector size report the speedup of individual graph-convolution layers, while the full bar reports the aggregate speedup across all layers of that input. Speedups range from roughly 2.9 $\times$ on proteins, which has higher reuse and therefore less lookup pressure, to 8.2 $\times$ on low-reuse products. Overall, offloading embedding lookup to the TMU improves graph-convolution performance most when irregular memory accesses dominate execution.

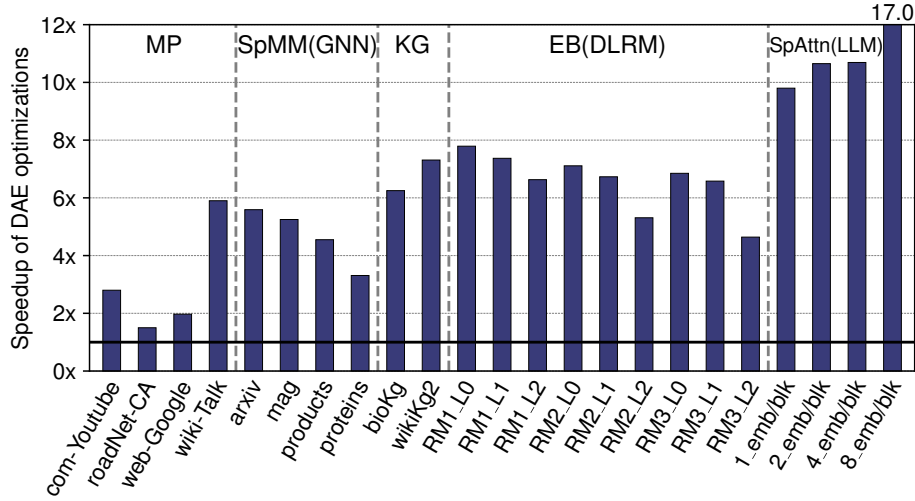


Figure 5.19: Speedup of embedding operations on the TMU-augmented DAE system described in Section 5.4.1 relative to the CPU baseline in Table 4.4. Bars are grouped by workload family: message passing (MP), SpMM-style graph convolution (SpMM/GNN), knowledge graphs (KG), DLRM EmbeddingBag (EB), and sparse-attention LLM embedding blocks. The horizontal black line marks the CPU baseline at 1 $\times$. For graph-learning models, the inputs and feature sizes are reported in Table 4.6; for DLRMs, we evaluate the three configurations in Table 4.5 under low (L0), medium (L1), and high (L2) locality [50]; and for LLMs, we use the BigBird setting [164] while varying the number of embedding blocks. Speedups range from roughly 1.5 $\times$ on roadNet-CA to 17.0 $\times$ on the largest sparse-attention LLM case, with an average of 5.8 $\times$. Overall, offloading lookup to the TMU improves embedding-operation performance across diverse sparse machine learning workloads.

Section 5.4, this section explores how the same decoupling principle enables more specialized architectures tailored for embedding operations.

Motivation

The DAE processor described in Section 5.4.1 is based on a conventional multicore design, augmented with a TMU per core to offload embedding lookups. While this approach improves performance and efficiency with minimal architectural changes, it does not fully exploit the specialization opportunities enabled by the TMU.

As shown in Figure 5.21, more than 75% of the processor power is consumed by compute cores and associative caches. However, these components are overprovisioned for embedding workloads, where computation is limited to simple vector accumulation. Furthermore, as discussed in Section 4.2.2, cache effectiveness depends strongly on input reuse. Consequently, both compute and cache resources can be significantly underutilized, as illustrated in Figure 5.22.

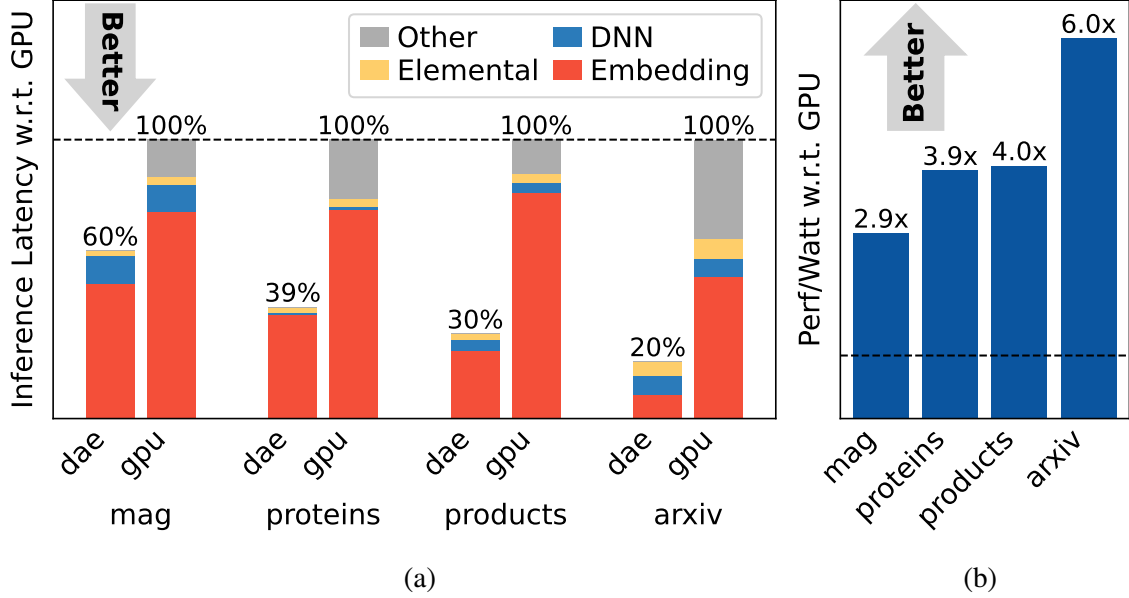


Figure 5.20: End-to-end performance and efficiency of the TMU-augmented DAE system described in Section 5.4.1 relative to an NVIDIA H100 SXM GPU with HBM3 memory [107] on GNN inference. Subfigure (a) reports inference latency normalized to the GPU for each input; within each group, the left bar is the DAE processor and the right bar is the GPU baseline at 100%. Stacked segments separate embedding operations, DNN layers, elementwise operations, and other work, and all operations are executed entirely on the corresponding platform. The DAE processor reduces latency to 60%, 39%, 30%, and 20% of GPU latency on mag, proteins, products, and arxiv, respectively. Subfigure (b) reports performance per watt normalized to the GPU, where the DAE processor improves efficiency by 2.9 $\times$ –6.0 $\times$. Overall, the DAE processor outperforms the GPU on both end-to-end latency and energy efficiency for GNN inference.

Motivated by these observations, in this section we explore two complementary optimizations: (i) reducing the complexity of DAE compute cores to improve energy efficiency, and (ii) moving these simplified cores closer to memory to bypass the cache hierarchy.

Experimental Setting

We name configurations based on whether embedding operations execute on big (B) or little (L) DAE cores, located either in the host processor (H) or near memory (M). Each DAE core consists of a TMU paired with either a big or a little compute core, as described in Table 5.7.

Our baseline is the datacenter-class DAE multicore processor from Section 5.4.1, denoted dae-bh. The dae-lh configuration replaces big cores with little cores while preserving core count, TMU capacity, and memory hierarchy. The dae-lm configuration extends dae-lh by placing little DAE cores in the base die of each Processing Near HBM (PN-HBM)

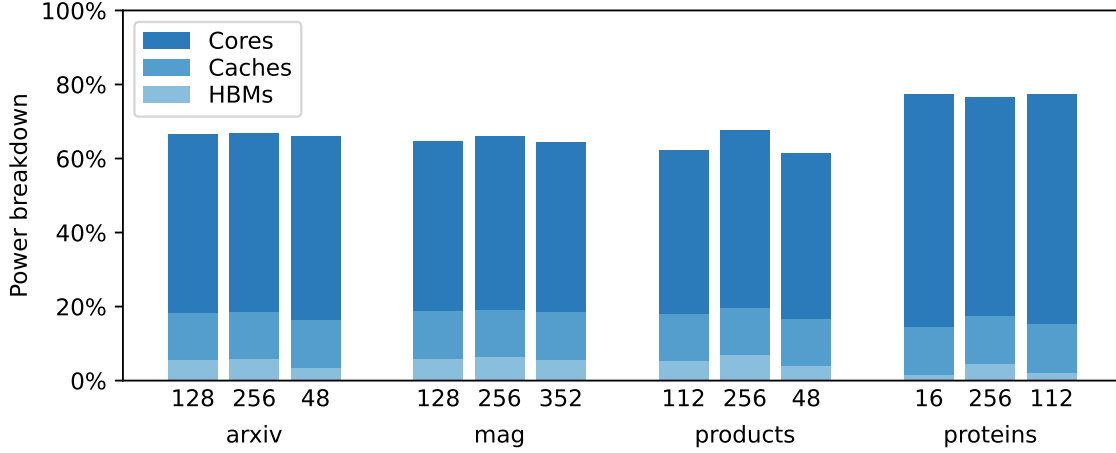


Figure 5.21: Power breakdown of the TMU-augmented DAE processor on graph-convolution embedding operations, normalized to total processor power. Groups correspond to input graphs, and bars within each group correspond to embedding-vector sizes. Stacked segments separate compute cores, caches, and HBM; TMU power is excluded because it contributes less than 2% of total power, as shown in Figure 5.16a. Compute cores dominate power consumption across inputs, while caches and HBM account for the remaining fraction. Overall, more than 75% of power is spent in compute cores and caches, motivating simpler DAE compute cores and cache organizations for embedding-heavy workloads.

Table 5.7: Specifications of the big and little DAE cores used in the specialized-architecture study. The TMU configuration, especially its Memory-Level Parallelism (MLP), is held constant across both designs; only the compute core frequency and vector width change. Overall, the table isolates the effect of simplifying the execute side while preserving access-unit capability.

	Big	Little
Type	Arm N1-like	Arm E1-like
Frequency	2.4 GHz	1.2 GHz
Vector FLOPs	$2 \times 64\text{B}$	$2 \times 16\text{B}$
TMU MLP	256 requests	256 requests

stack. This configuration uses 8 cores per HBM stack, following the bandwidth-scaling result in Figure 5.17, which shows that 8 DAE cores per stack are sufficient to saturate HBM bandwidth for low-locality layers. Table 5.8 summarizes these configurations.

The *dae-lm* system includes six PN-HBM devices, each treated as an accelerator with a private address space. The remaining processor components (cache hierarchy and host DAE cores) form the host CPU. We distribute work across PN-HBM devices using row parallelism [98], a widely used technique for partitioning large embedding tables in distributed recommendation systems. Under this scheme, embedding-table rows are mapped across

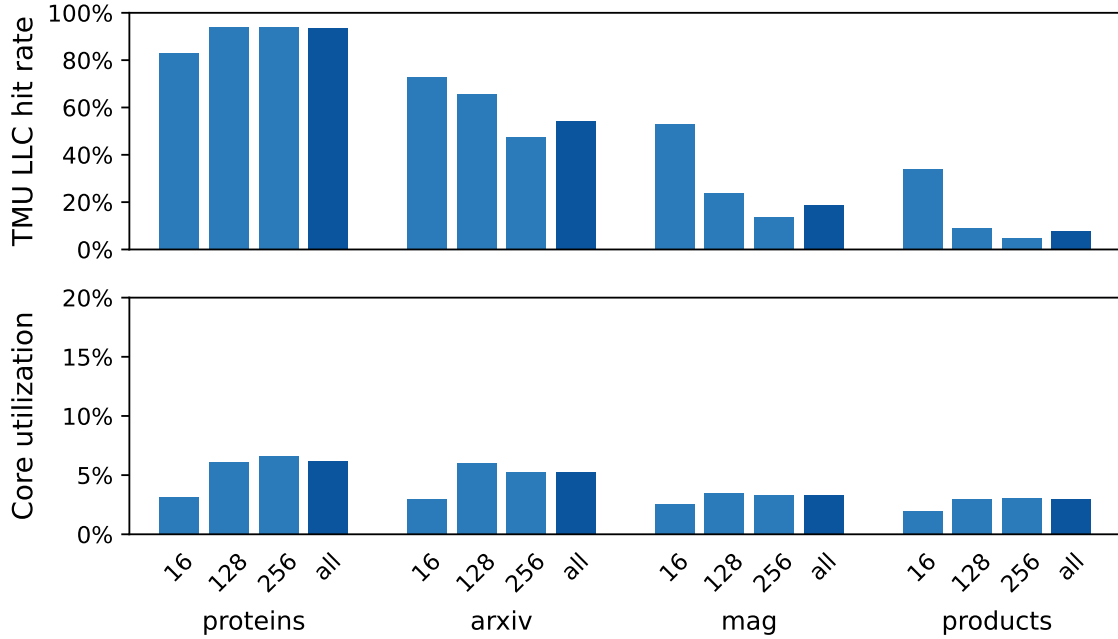


Figure 5.22: Cache and compute utilization of the TMU-augmented DAE processor on graph-convolution embedding operations. The top panel reports TMU-side LLC hit rate, and the bottom panel reports compute-core utilization as achieved floating-point throughput normalized to peak. Input graphs are ordered from higher reuse (proteins) to lower reuse (products), and bars within each group correspond to embedding-vector sizes plus the aggregate all case. LLC hit rate varies widely with reuse and vector size, from above 90% on proteins to below 10% on low-reuse products. In contrast, compute-core utilization remains below 7% across all cases. Overall, the DAE processor is still dominated by access behavior rather than vector computation, motivating simpler compute cores and reuse-aware cache placement.

Table 5.8: Specialized DAE configurations evaluated in this section. BH uses big DAE cores in the host processor, LH replaces them with little DAE cores in the host processor, and LM adds little DAE cores near memory for embedding operations while host little cores perform reduce-scatter. Overall, the configurations separate the effects of simplifying compute from the effects of moving execution closer to memory.

	DAE host cores	DAE mem cores
BH	TMU+Big compute	None
LH	TMU+Little compute	None
LM	TMU+Little compute (for reduce-scatter)	TMU+Little compute (for embedding op)

memory stacks, each device processes its partition independently, and partial results are combined via a reduce-scatter operation performed on the host CPU while PN-HBM devices execute embedding operations.

As before, we evaluate performance and power using gem5 [89] and McPAT [82] for the host CPU, and Ramulator2.0 [90] for PN-HBM modeling.

Results

Figure 5.23 summarizes the performance and efficiency impact of the proposed optimizations. Overall, reducing compute core complexity (dae-lh) substantially improves energy efficiency with minimal performance degradation, while near-memory execution (dae-lm) provides additional gains for low-reuse workloads.

The dae-lh configuration achieves 94%–99% of the performance of dae-bh, while improving performance per watt by $2.66\times$ – $4.12\times$. This is because, as demonstrated earlier, embedding operations underutilize large out-of-order cores, and simpler cores can deliver comparable throughput at only 10%–12% of the energy. Workloads with higher node degree achieve better performance due to increased computation per memory access, while low-reuse workloads exhibit lower efficiency due to dominant memory and cache overheads.

In contrast, dae-lm provides the highest benefits for low-reuse datasets, where caches provide limited filtering. By moving DAE cores closer to memory, this configuration eliminates unnecessary cache accesses, reducing both latency and energy. For example, on products, dae-lm achieves $1.85\times$ higher performance and $4.39\times$ higher efficiency than dae-lh, and $1.79\times$ higher performance and $7.86\times$ higher efficiency than dae-bh. Similar trends are observed for mag, where dae-lm improves performance by 9% and efficiency by 83% over dae-lh.

These gains stem from eliminating cache overheads, which account for up to 50% of total energy in low-reuse workloads. However, for higher-reuse workloads, caches remain beneficial. For example, on arxiv, dae-lm is slightly slower (1%) but $1.46\times$ more efficient than dae-lh. On proteins, where most accesses are cacheable, dae-lm is both slower and less efficient due to increased memory traffic. In this case, dae-lh achieves 21% higher performance and slightly better efficiency.

Overall, dae-lh is preferable for high-reuse workloads, while dae-lm is better suited for low-reuse workloads.

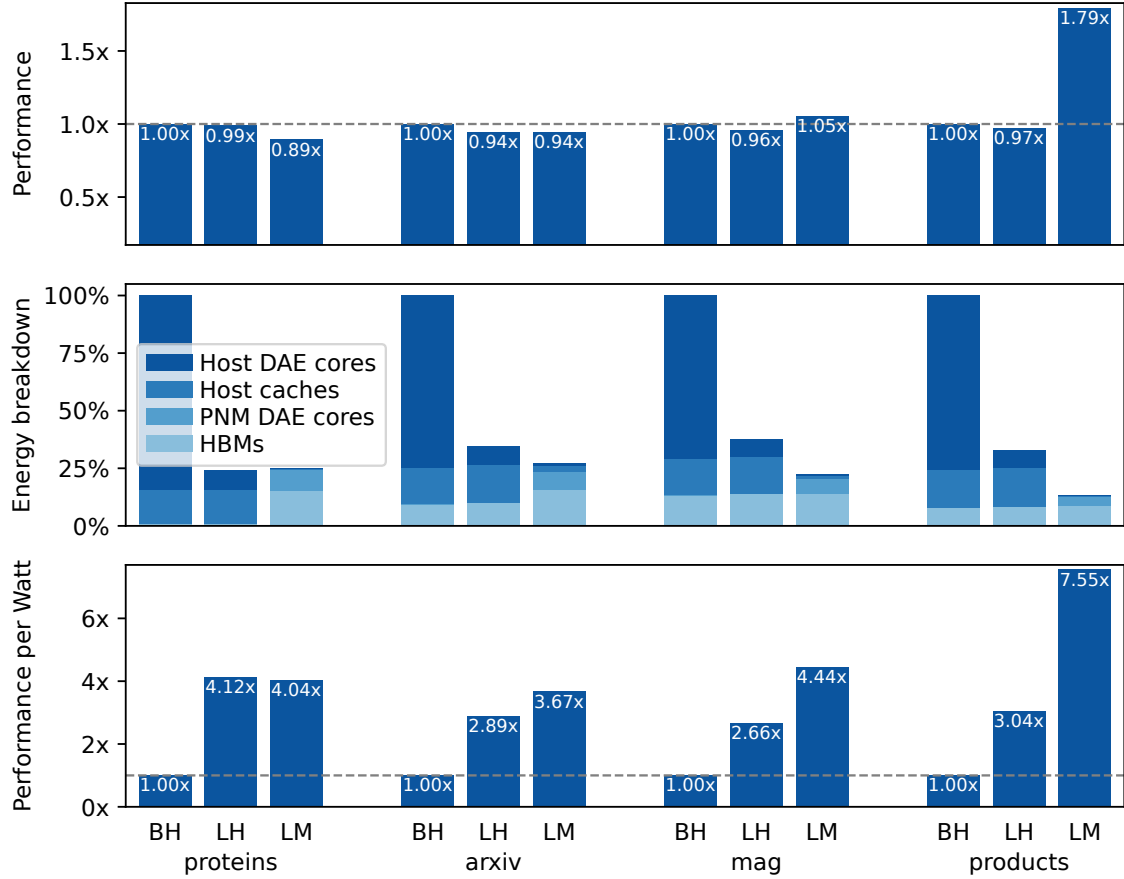


Figure 5.23: Performance, energy, and performance-per-watt of the specialized DAE configurations in Table 5.8, using the big-core host configuration (BH) as the baseline for each input. Inputs are sorted from higher reuse (proteins) to lower reuse (products). The top panel reports performance normalized to BH; the middle panel reports energy normalized to BH and broken down into host DAE cores, host caches, near-memory DAE cores, and HBM; and the bottom panel reports performance per watt normalized to BH. Replacing big cores with little host cores (LH) preserves 94%–99% of performance while improving efficiency by 2.66 $\times$ –4.12 $\times$. Moving little DAE cores near memory (LM) helps low-reuse inputs most, reaching 1.79 $\times$ performance and 7.55 $\times$ performance per watt on products, but is less attractive for high-reuse proteins, where cache locality remains useful. Overall, simpler host cores favor high-reuse workloads, while near-memory DAE execution is best for low-reuse workloads.

5.5 Summary

This chapter introduced the TMU, a near-core access unit that maps sparse tensor traversal, coordinate merging, and data marshaling to specialized hardware while preserving CPU vector execution for arithmetic. Across scientific sparse tensor algebra workloads, the TMU improves performance by exposing memory-level parallelism, reducing merge overheads, and producing SIMD-friendly operand streams at low area cost. Across machine learning embedding operations, the TMU improves bandwidth utilization and performance per watt, and the same decoupling principle enables more specialized DAE organizations with simpler or near-memory compute cores. The cost of this flexibility is a more complex programming model: users or compilers must describe tensor traversal, merging, callbacks, and outQ construction. Chapter 6 introduces Ember, a compiler stack that automates this mapping.

Chapter 6

The Ember Compiler for DAE Compilation

Despite its potential, decoupling memory access from computation introduces substantial challenges for programmability. In scientific computing, this is less problematic because applications are typically built on libraries of standardized tensor algebra routines—such as BLAS [77]—that can be manually replaced with hand-optimized DAE implementations. In contrast, machine learning workloads are automatically compiled from high-level model descriptions, and their computational kernels are frequently fused. This makes manual DAE programming impractical and necessitates an automated approach.

Section 6.1 first introduces a programming model for expressing sparse machine learning operations on DAE architectures, using embedding operations and our TMU–CPU system as concrete examples. Section 6.2 then presents Ember, a compiler that lowers sparse machine learning operations to optimized DAE code. Ember is implemented in MLIR so that it can integrate with existing machine-learning compiler stacks, including PyTorch through `torch-mlir` and TensorFlow through its native MLIR compiler stack. The key idea behind Ember is to optimize DAE code at an intermediate level that exposes the access/execute split without losing the control-flow and data-flow information needed for global optimization. Finally, Section 6.3 shows that Ember generates code whose performance matches that of hand-optimized implementations, unlocking the full potential of DAE architectures without imposing additional programmability burdens.

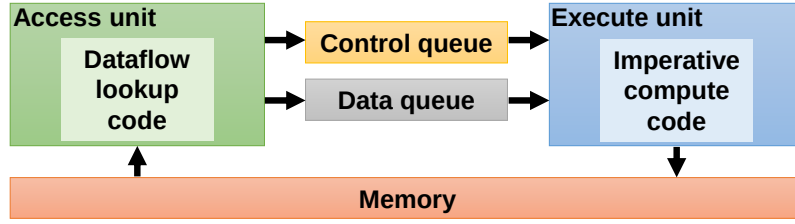


Figure 6.1: Logical DAE abstraction used by Ember for embedding operations. Lookup code runs on an access unit, compute callbacks run on an execute unit, and control/data queues carry callback tokens and operands between them. The goal of this abstraction is to hide TMU-specific details such as TUs, lanes, layers, merge modes, and outQ construction while preserving the access/execute split needed for DAE optimization.

6.1 A DAE Programming Abstraction for Embedding Operations

Automatic DAE compilation first requires an abstraction that hides architecture-specific implementation details while preserving the access/execute split that makes DAE architectures efficient. For embedding operations, this abstraction separates lookup code, compute callbacks, and the queues that carry operands and control tokens between them. This section formalizes that programming model for embedding operations and, more generally, sparse–dense tensor operations on DAE architectures.

Figure 6.1 shows the logical DAE abstraction used by Ember for embedding operations on TMU–CPU systems and related DAE architectures [161, 110, 14]. Unlike the low-level TMU programming model introduced in Section 5.1, this abstraction does not expose hardware-specific details such as TUs, layers, lanes, merge modes, queue allocation, or outQ construction. Instead, it represents only the logical separation between lookup code, compute callbacks, and the queues that carry operands and control tokens between them. We call this low-level compiler abstraction the Decoupled Lookup–Compute (DLC) IR, which is the lowest-level IR of Ember, as explained in Section 6.2.1.

The embedding operations targeted by Ember are sparse–dense tensor operations in which sparse metadata selects rows or blocks from dense tensors and the execute side performs elementwise reductions over the selected values [120]. For this class of programs, the lookup phase can be represented as a structured scan-and-lookup traversal hierarchy (Section 2.1.5), and computation can be attached to the beginning, iteration, or end of each traversal loop [70]. Unlike general sparse tensor algebra, these operations do not require sparse coordinate merging: the access unit traverses sparse metadata and performs indirect lookups, but it does not need to co-iterate multiple sparse fibers. Ember targets Einsum-style oper-

ators whose input and output tensors are distinct, so in-place updates are not part of the operator representation. Lookup operands are therefore read-only during a kernel, and output updates are handled by execute-side callbacks; consequently, the access unit does not depend on values produced by the execute unit during the same traversal.

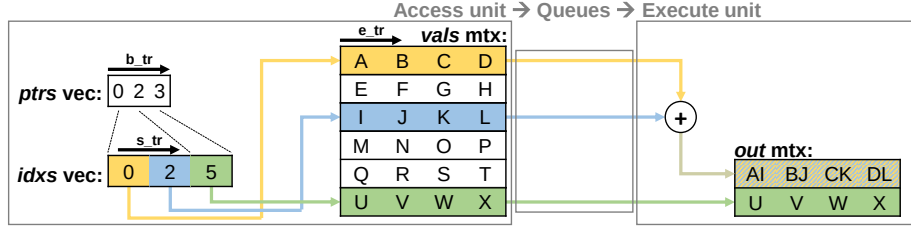
Because of these properties, embedding operations can be expressed as a dataflow chain of access and compute blocks [57]. We can map lookup blocks to the access unit and compute blocks to the execute unit, and stream data and commands through logical queues. The DLC IR represents lookup blocks with streaming dataflow code, as it maps well to spatial architectures for memory-intensive workloads [57, 161, 103, 113, 134, 36, 136], including the TMU. Compute blocks, instead, are represented with imperative code, which can implement all the compute variants introduced above and other general-purpose computation. Figures 6.2c and 6.2e show the DLC IR for the EB function, which we discuss in the remainder of this section.

The DLC lookup code loads embedding elements through memory streams. The IR uses index streams to maintain the iteration space for these memory streams, which are generated by traversal operators and can be transformed with integer ALU streams. Formally:

- `loop_tr(lb,ub,stride)`: creates a logical traversal over the iteration space defined as $i=lb; i < ub; i += stride$, where `lb` and `ub` are streams or immediate values and `stride` is an immediate value. The traversal handle exposes an `ite` stream containing the induction variable.
- `mem_str(base,idx)`: loads into a stream the values of the base memory locations indexed by the `idx` stream.
- `alu_str(op,op1,op2)`: computes an integer binary operation $op \in \{+, -, \times, \div\}$ on operands `op1` and `op2`, which can either be streams or immediate values.

Concretely, we show these streams in the lookup code for the EB example in Figure 6.2c. Lines 1–12 in Figure 6.2c represent lines 5–9 in Figure 6.2b, which are compiled by Ember.

Compute code consists of callbacks that the access unit triggers while traversing and loading tensor operands. In Figure 6.2b, we can wrap the compute operation in line 10 into a callback that the access unit can trigger at each iteration of its parent loop. To trigger these callbacks, the access unit enqueues the corresponding token (the inner-loop iteration token (e_i)) in the control queue and its operands (result coordinates b and e and value v) in the data queue. Lines 14–17 in Figure 6.2c and lines 2–4 in Figure 6.2e show how these tokens and data are enqueued and dequeued from the queues, and the exact queue content is shown in Figure 6.2d. Figure 6.6d shows a more complex example with multiple callbacks.



(a) Functional example of the EB function.

```

1 void eb(idxs, ptrs, vals, out){
2   for(int b = 0; b < num_batches; b++){ // Batch traversal (b_tr)
3     for(int p = ptrs[b]; p < ptrs[b+1]; p++){ // Segment traversal (s_tr)
4       idx = idxs[p]; // Load embedding index
5       for(int e = 0; e < emb_len; e++){ // Embedding vec traversal (e_tr)
6         val = vals[p,e]; // Load embedding element
7         out[b,e] += val; }}} // Reduce embedding elements

```

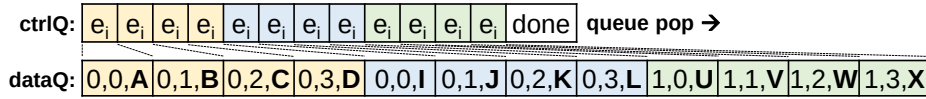
(b) Traditional imperative code, which also acts as input to Ember.

```

1 tr b_tr=loop_tr(0, num_batches, 1) // iterate thru batch
2 str beg_ptr=b_tr.mem_str(ptrs, b_tr.ite) // load segment beg ptr
3 str end_pos=b_tr.alu_str('+', b_tr.ite, 1) // compute next ptr pos
4 str end_ptr=b_tr.mem_str(ptrs, end_pos) // load segment end ptr
5
6 tr s_tr=loop_tr(beg_ptr, end_ptr, 1) // iterate thru segment
7 str emb_idx=s_tr.mem_str(s_tr, idxs, s_tr.ite) // load emb idx
8 str emb_beg=s_tr.alu_str(s_tr, '*', emb_idx, emb_len) // emb addr
9
10 tr e_tr=loop_tr(0, emb_len, 1) // iterate thru emb vec
11 str emb_pos=alu_str('+', emb_beg, e_tr.ite) // element addr
12 str emb_val=mem_str(vals, emb_pos) // load emb element
13
14 enq_op(b_tr.ite, e_tr, ite) // push batch position
15 enq_op(e_tr.ite, e_tr, ite) // push emb el position
16 enq_op(emb_val, e_tr, ite) // push emb el value
17 callback(e_tr, ite) //trigger call on every emb iter

```

(c) DLC dataflow lookup code.



(d) DLC queue content for the EB example.

```

1 while(ctrlQ.pop() == e_i){ // while some element
2   b = dataQ.pop<1 x i32>(); // read embedding position
3   e = dataQ.pop<1 x i32>(); // read element position
4   v = dataQ.pop<1 x f32>(); // read element value
5   fma(&out[b,e],v); // compute and store

```

(e) DLC imperative compute code.

Figure 6.2: DLC representation of an EB operation. The functional example and imperative code define the source computation, the lookup code traverses batches, segments, and embedding elements with streams, the queues serialize callback tokens and operands, and the compute code dequeues those values to update the output. Overall, DLC separates embedding lookup from accumulation while preserving the callback order of the original loop nest.

Formally, we can program the access unit to marshal control tokens and operands at three traversal locations—at each iteration, beginning, and end of a loop—through the following operations:

- `enq_op(s_id, tr_id, event)`: enqueues into the data queue (dataQ) the content of the `s_id` stream every time a traversal event $\in \{\text{beg}, \text{ite}, \text{end}\}$ occurs in `tr_id`.
- `callback(tr_id, event)`: enqueues into the control queue (ctrlQ) the corresponding control token every time a traversal event $\in \{\text{beg}, \text{ite}, \text{end}\}$ occurs in `tr_id`.

whereas the execute unit reads tokens and operands with:

- `deq()`: dequeues a control token from the control queue.
- `deq<ty>()`: dequeues a value with `ty` type from dataQ.

We designed our DLC IR to express general embedding operations on various DAE architectures from the literature beyond the TMU, including:

- **SpZip** [161] integrates near-core specialized units in multicore processors to accelerate traversal and (de)compression operations in graph analytics. Similar to the TMU and other architectures for sparse tensor algebra [57], SpZip’s access unit can be programmed with a streaming language to traverse sparse data structures and send control tokens and data to the core, which can be programmed to process them with imperative languages. Hence, both the access and execute units can be abstracted with the DLC IR.
- **MAPLE** [110] offloads general indirect accesses to specialized units placed in the network-on-chip of a multicore processor. Access/compute units communicate through push/pull operations to/from queues, respectively. Unlike the TMU and SpZip, MAPLE access units are general-purpose cores. Hence, both access and execute code are imperative. The authors note that such code can be generated from sparse tensor compilers like TACO [70]. Since the Sparse Abstract Machine (SAM) [57] implements all hardware primitives of TACO and since the DLC IR can represent all SAM operations necessary for sparse-dense tensor operations, the DLC IR can abstract the MAPLE programming model.
- **Planar** [14] accelerates scientific workloads by offloading strided and irregular memory accesses from CPUs to tiny near-memory cores. Since Planar has a programming model similar to MAPLE, we can also abstract it with the DLC IR.

Ultimately, although separating embedding lookups from computation substantially improves performance and efficiency, it also introduces significant complexity into the DAE programming model, hindering the widespread adoption of DAE architectures. In Sections 6.2 and 6.3, we demonstrate how the Ember compiler addresses this challenge by automatically targeting the DLC IR from standard imperative code and compiling down to DAE hardware code.

6.2 Ember Compiler Design

Ember is built around the observation that DAE code must be optimized before it is fully serialized into queues. The MLIR Structured Control Flow (SCF) IR preserves the loop structure and data flow needed for global optimization, but does not expose the access/execute split. The Decoupled Lookup–Compute (DLC) IR presented earlier exposes that split, but is too low-level for many global optimizations. Ember therefore introduces the Structured Lookup–Compute (SLC) IR as an intermediate representation that exposes DAE structure while preserving enough program information to optimize it. This section begins with the compiler pipeline in Section 6.2.1, then explains how SCF is lowered through SLC to DLC in Section 6.2.2, and finally describes the optimizations enabled by this representation in Section 6.2.3.

6.2.1 Compiler Overview

Figure 6.3 shows our MLIR [76] implementation of Ember. Overall, Ember compiles structured representations of sparse machine learning operations into optimized DLC code, which is then used for DAE code generation. The Structured Control Flow (SCF) Dialect is perhaps the most popular structured IR in MLIR, and resembles the imperative code in Figure 6.2b. Ember operates on structured MLIR, which enables integration with existing machine-learning compiler stacks. For example, PyTorch programs can enter Ember through `torch-mlir` [87], while TensorFlow programs can enter through TensorFlow’s native MLIR compiler stack. Ember can also generate SCF from general tensor algebra expressions with tools such as the MLIR sparsifier [47].

The compiler pipeline uses four levels of representation. SCF is the structured-loop input. Structured Lookup–Compute (SLC) is the optimization IR: it marks access streams and compute callbacks while preserving structured control flow and SSA-style data flow. DLC is the low-level DAE IR: it makes control and data queues explicit after global optimizations

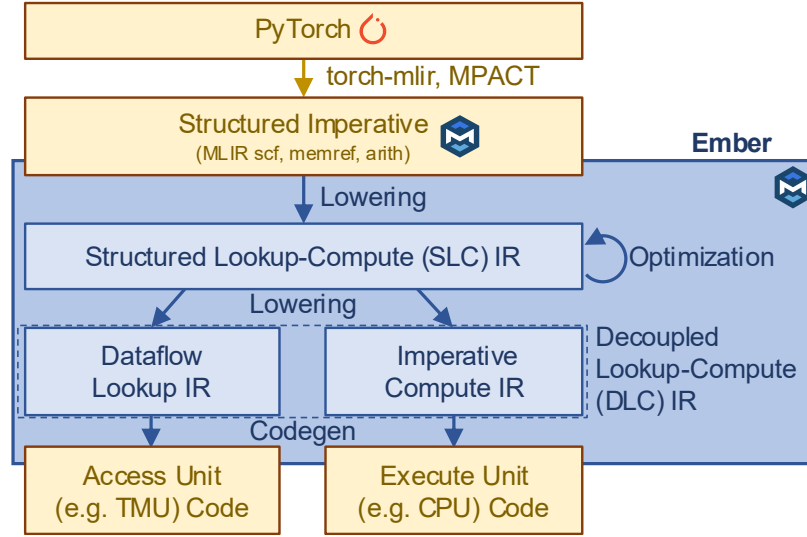


Figure 6.3: Overview of the Ember compiler. Yellow boxes represent compiler inputs and outputs, and blue boxes represent the main compiler components introduced in this thesis. Ember starts from structured MLIR, introduces the Structured Lookup–Compute (SLC) IR to preserve optimization context across access and execute code, lowers optimized programs to the Decoupled Lookup–Compute (DLC) IR, and finally emits target-specific DAE code. In this way, Ember compiles sparse machine learning operations into optimized DAE code without requiring users to write DAE programs manually.

have been selected. Finally, target-specific dialects such as `tmu` and `llvm` generate code for the access and execute units.

To generate optimized DAE code from SCF, Ember needs to decouple memory accesses from computation and generate optimized code for the access unit and execute unit. However, the main challenge of this process is that DAE code cannot be optimized either in the DLC IR or in SCF alone. Global optimizations across access and execute code need the control flow and data flow of the entire input program, which are already lost and decoupled in the DLC IR. For instance, it would be impractical to identify what stream generates the `v` value in the DLC IR in Figure 6.6d, which is an essential analysis for several optimizations discussed in Section 6.2.3. Instead, while SCF has knowledge of both control flow and data flow, it does not differentiate between access and execute code, nor does it identify how data is (de)serialized between them. Therefore, global optimizations would need to be integrated into the code separation and/or generation logic, which is impractical for a large set of complex optimizations, such as those in Section 6.2.3, that should be executed iteratively [76].

For this reason, we designed the Structured Lookup–Compute (SLC) IR. The SLC IR is a natural extension of the SCF IR for DAE code. The key feature of the SLC IR is to represent

<i>Any CPU Statement</i>	<i>STMT</i>		<i>Memory Reference</i>	<i>mref</i>
<i>CPU variable</i>	<i>var</i>		<i>Stream Variable</i>	<i>s</i>
<i>Unsigned Integer</i>	<i>uint</i>			
<i>For-Loop</i>	<i>FOR</i>	$::=$	for (<i>HEAD</i>) { <i>SDEC</i> * <i>BODY</i> }	
<i>For Header</i>	<i>HEAD</i>	$::=$	<i>s</i> = <i>r</i> to <i>r</i> step uint	
<i>Stream Declaration</i>	<i>SDEC</i>	$::=$	<i>s</i> = <i>se</i>	
<i>Loop Body</i>	<i>BODY</i>	$::=$	<i>CALL</i> <i>CALL?</i> <i>FOR</i> <i>CALL?</i>	
<i>Callback</i>	<i>CALL</i>	$::=$	callback() { <i>TVAL</i> * <i>STMT</i> ⁺ }	
<i>Value Conversion</i>	<i>TVAL</i>	$::=$	<i>var</i> = to_val(<i>s</i>)	
<i>Loop Range</i>	<i>r</i>	$::=$	<i>s</i> <i>var</i> uint	
<i>Stream Expression</i>	<i>se</i>	$::=$	<i>ls</i> <i>as</i> <i>cs</i> <i>bs</i>	
<i>Load Stream</i>	<i>ls</i>	$::=$	load_str(<i>mref</i> [<i>indices</i> *], <i>uint</i>)	
<i>Indices</i>	<i>indices</i>	$::=$	<i>s</i> <i>var</i> uint	
<i>ALU Stream</i>	<i>as</i>	$::=$	alu_str(<i>sop</i> , <i>s</i> ₁ , <i>s</i> ₂)	
<i>Stream Ops</i>	<i>sop</i>	$::=$	+ - * / % ...	

Figure 6.4: Grammar of Ember’s Structured Lookup–Compute (SLC) IR. The SLC IR represents structured loops, stream declarations, callbacks, and stream-to-value conversions in a single structured representation. In this way, SLC preserves the control flow and data flow needed for global DAE optimizations before the program is serialized into explicit DLC queues.

DAE code while preserving the control flow and data flow of the input program, enabling iterative DAE optimizations across access and execute code. In this way, Ember decides what SCF operations to map to the access and execute units and generates the SLC IR. Then, it performs global optimizations on the SLC IR and lowers it to the DLC IR for separate optimizations of access and execute code. Access and execute code are then lowered to target-specific IRs such as the tmu and llvm dialects, which are then used for code generation. Sections 6.2.2 and 6.2.3 present these steps in order.

6.2.2 From SCF to DLC

This section introduces the SLC IR and the two lowering steps around it. First, we define SLC as the intermediate representation that preserves structured loops while exposing the lookup–compute split. Second, we describe how Ember lowers SCF code into SLC by identifying access-unit traversals and execute-unit callbacks. Third, we describe how optimized SLC code is serialized into DLC queues and callbacks. Together, these steps identify loop traversals that can run on the access unit, preserve computation that must remain on the

execute unit, and record stream-to-value uses so later optimizations can reason across the access/execute boundary.

Structured Lookup–Compute IR

Figure 6.4 shows the SLC grammar, and Figure 6.5b shows the SLC IR of the EB function example. The main goal of the SLC IR is to abstract the queue (de)serialization logic in the DLC IR, enabling global DAE optimizations.

Similar to the DLC IR, the SLC IR defines access code (i.e. memory access and index arithmetic) through streams and compute code through callbacks. However, unlike the DLC IR, the SLC IR places these streams and callbacks within structured SLC imperative for-loops.

In this way, the value of the streams can be directly accessed from the callbacks through stream-to-value conversion operations, preserving the data flow of the input code. Semantically, an SLC stream produces one logical value per iteration of its defining loop. A callback observes stream values only through `to_val` operations, which materialize the value associated with the callback event. The lexical position of callbacks in the loop body defines the callback order, and lowering to DLC preserves that order when emitting control tokens and operands into queues.

The SLC IR allows Ember to simultaneously optimize lookup and compute code through global analyses and transformations, including dominance analysis, vectorization, and code motion across different compute regions as well as across access and execute code. We later describe these analyses and optimizations that the SLC IR enables in more detail in Section 6.2.3.

Lowering SCF to SLC

Figure 6.5 shows a lowering example from SCF to SLC. To generate the SLC IR, Ember recursively traverses the loop hierarchy of the SCF code looking for loops to offload to the access unit. We define an *offloading candidate* as a loop that (1) has iteration bounds that are either static or computed by another offloading candidate and (2) loads from at least one read-only memory location that has not been read from a parent loop. Condition (1) is necessary because access units cannot generally read data from the execute units. Condition (2) excludes workspace loops [69, 166] (i.e., loops that work only on partial results). As partial results are likely cached, workspace loops do not benefit from memory acceleration. All loops in Figure 6.5a are offloading candidates. For a more complex example, only the first two loops from the MP pseudocode in Section 4.2.2 are offloading candidates. The

```

1 void eb(idxs: mref<? x idx>,
2         ptrs: mref<? x idx>,
3         vals: mref<? x f32>,
4         out: mref<? x ? x f32>){
5   for(idx b=0; b<n_batches; b++){
6     idx beg=ptrs[b];
7     idx end=ptrs[b+1];
8     for(idx p=beg; p<end; p++){
9       idx i=idxs[p];
10      for(idx e=0; e<emb_len; e++){
11        f32 val=vals[i,e];
12        f32 acc=out[b,e];
13        out[b,e]=acc+val; }}}}
```

(a) SCF IR

```

1 void eb(idxs: mref<? x idx>,
2         ptrs: mref<? x idx>,
3         vals: mref<? x f32>,
4         out: mref<? x ? x f32>){
5   slc.for(str b from 0 to n_batches){
6     str beg=slc.mem_str(ptrs[b]);
7     str end=slc.mem_str(ptrs[b+1]);
8     slc.for(str p from beg to end){
9       str i=slc.mem_str(idxs[p]);
10      slc.for(str e from 0 to emb_len){
11        str val=slc.mem_str(vals[i,e]);
12        slc.callback{
13          idx bv=slc.to_val(beg);
14          idx ev=slc.to_val(end);
15          f32 vv=slc.to_val(val);
16          f32 acc=out[bv,ev];
17          out[bv,ev]=acc+vv; }}}}}
```

(b) SLC IR

Figure 6.5: Lowering from SCF to Ember’s SLC IR. Ember converts SCF **for-loops** into SLC for-loops, converts read-only **memory accesses** into SLC streams, and wraps **computation** into callbacks that access streams through stream-to-value operations. In this way, the lowering moves lookup work toward the access unit while preserving reductions and output updates on the execute side.

last two lines (loops) in MP are workspace loops, as they just multiply and accumulate the temporary vector with the vertex vector, which has already been read.

The legality of this decoupling transformation follows from Ember’s Einsum-style operator representation. Input tensors and the output tensor are distinct: input tensors are read-only, the output tensor is write-only, and the representation does not allow tensor overlap or in-place updates. As a result, moving input loads and index arithmetic to the access unit cannot introduce read-write conflicts or aliasing with output updates. Ember therefore offloads only operations whose operands are constants, loop induction variables, input-tensor loads, or values produced by other access-unit streams. Operations that write the output tensor, update partial results, or depend on execute-side values remain in callbacks. In this way, reductions and output stores stay on the execute unit, while read-only traversal and index computation move to the access unit.

As embedding operations are variants of sparse-dense tensor multiplications, their loop hierarchy can have at most one offloading candidate per level and at most one workspace loop per level [70, 69]. The decoupling algorithm, therefore, recursively traverses and selects one

offloading candidate per level, leaving the other loops for software execution. The SCF offloading candidates are lowered to SLC for-loops.

Then, Ember places callbacks in the SLC loops. Operations to be offloaded (i.e., read-only load operations and index arithmetic) are transformed into streams and moved before their corresponding callback. Operations that should not be offloaded to the access unit are moved inside their corresponding callback, meaning they will be executed in software. For instance, Ember transforms and moves the read-only load operation in line 11 in Figure 6.5a before the callback, whereas Ember moves the accumulation operations in lines 12 and 13 inside the callback. Finally, stream-to-value operations are added in a callback for each operation reading from a stream.

Why Ember Starts from Structured IR

To effectively optimize DAE code, Ember needs the control flow and data flow of the input program. In structured IRs like the SCF IR in MLIR, control flow is expressed as structured loop hierarchies, whereas data flow is expressed with use-def chains in SSA form. Lower-level IRs like the LLVM IR, instead, are generally unstructured and represent control flow with conditional branches. This substantially complicates code analysis and transformation, making unstructured IRs unsuitable.

We believe that lowering from higher-level tensor IRs like MLIR Linalg is not ideal either. First, lowering from tensor IRs would not allow us to leverage fundamental algorithmic optimizations, such as operator fusion, which are generally applied (1) at the tensor level or (2) while lowering to structured IRs. Moreover, current tensor IRs like Linalg in MLIR can only express tensor operators between two operands, but embedding operations in message-passing models (e.g. SDDMM) and knowledge graphs require more operators. Finally, tensor IRs can only represent tensor algebra operations, but embedding operations can be more complex. For instance, EB-like functions might identify the boundaries of the indices of active categories in a feature through segment lengths rather than pointers. While this is not a standard sparse tensor format, it can still be represented with for-loops and the DLC IR, but requires a more flexible solution than sparse tensor formats alone.

The Sparse Abstract Machine (SAM) IR [57] defines primitives to represent sparse tensor algebra expressions as dataflow programs. However, SAM cannot express DAE code separation nor data (de)serialization. Hence, while the SAM IR can be used as another Ember frontend IR, it cannot be used for DAE code optimization. Moreover, there is currently no lowering from higher-level IRs (e.g. Linalg) to SAM, preventing automatic PyTorch compilation.

Therefore, we believe that structured IRs are the most suitable input for Ember.

Lowering SLC to DLC

After performing global optimizations, the SLC IR is lowered to the low-level DLC IR. The DLC IR is generated by traversing the SLC IR from the outer to the inner SLC for-loop. SLC for-loops and streams are lowered to DLC traversal operators and streams. Callbacks, instead, are moved into a while-loop in the compute code (Figure 6.1). Multiple callbacks are chained into an if-then-else construct that dispatches on the token IDs from the control queue (e.g. Figure 6.6d). Ember generates push instructions for control tokens based on the location of the callback in the SLC IR and queueing instructions for operands according to the SLC stream-to-value operations.

Target Code Generation

After lowering to the DLC IR, Ember performs separate target-specific optimizations of access and execute code and generates machine code. Ember lowers DLC access code to TMU code by mapping logical streams to physical streams, whereas it lowers DLC compute code for the CPU by lowering queue pop instructions to load operations and queue handling code.

6.2.3 SLC-Level Optimizations

This section first describes three key DAE optimizations enabled by the SLC IR: vectorization, embedding-vector buffering, and queue alignment. Figures 6.6 and 6.7 show how these optimizations transform the SLC IR, compute code, and queues. We then discuss model-specific optimizations and Ember–TMU HW–SW co-design. The common theme is that these optimizations require visibility across the access/execute boundary before queue serialization has been fixed, which is precisely the information preserved by SLC.

Vectorization

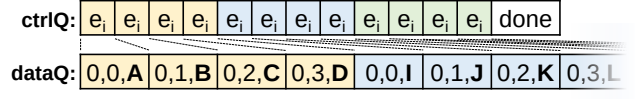
Vectorization is one of the most impactful DAE optimizations (Sections 5.3 and 5.4). As shown in Figure 6.6b, for each token, vectorization pops a vector of *vector length* (*vlen*) elements, improving marshaling and compute efficiency. While auto-vectorizing general code is challenging [142, 68], the SLC IR provides an effective representation to vectorize loading, marshaling, and computing of embedding operations.

As shown in Figure 6.7b, Ember vectorizes code by converting SLC operations into their vectorized SLCV duals. Unlike the SLC for-loop, the SLCV for-loop (1) adds a *vector length* attribute, (2) instantiates vectorized induction variables, and (3) introduces the concept of

```

1 while(ctrlQ.pop() == ei){
2   b = dataQ.pop<1 x idx>();
3   e = dataQ.pop<1 x idx>();
4   v = dataQ.pop<1 x f32>();
5   acc(&out[b,e],v); }

```

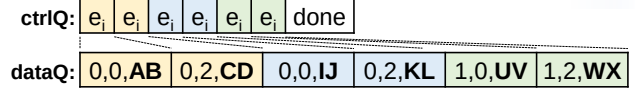


(a) Unoptimized code.

```

1 while(ctrlQ.pop() == ei){
2   b = dataQ.pop<1 x idx>();
3   e = dataQ.pop<1 x idx>();
4   v = dataQ.pop<vlen x f32>();
5   v_acc(&out[b,e], v);

```

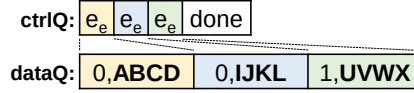


(b) Vectorized code.

```

1 while(ctrlQ.pop() == ee){
2   b = dataQ.pop<1 x idx>()
3   for(e=0; e<emb_len; e+=vlen){
4     v = dataQ.pop<vlen x f32>()
5     v_acc(&out[b,e],v) }

```

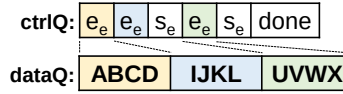


(c) Buffered code.

```

1 out_ptr = out
2 while(tkn = ctrlQ.pop() != done){
3   if(tkn == ee){
4     for(e=0; e<emb_len; e+=vlen){
5       v = dataQ.pop<vlen x f32>()
6       v_acc(out_ptr+e, v) }}
7   else if(tkn == se){
8     out_ptr += emb_len }

```



(d) Queue aligned code.

Figure 6.6: Impact of SLC optimizations on DLC EB compute code and output queues. Vectorization widens queued values, buffering groups an entire embedding vector behind one token, and queue alignment removes scalar operands that would otherwise misalign vector data in the queue. Overall, these optimizations reduce queue traffic and improve execute-side vector efficiency.

mask to handle loop boundaries that are not multiples of the vector length. Masks are used by SLCV streams to perform vectorized index computation and data loading.

We define a *vectorization scheme* as the set of for-loops in a loop-nest interval, from a selected parent loop to a selected inner loop, inclusive, that we intend to vectorize. A for-loop can be vectorized if and only if all of its callbacks can be vectorized. A vectorization scheme is *legal* if and only if all of its for-loops can be vectorized.

Vector extensions such as Arm SVE [146] provide instructions to vectorize most callbacks in embedding operations, making the space of legal vectorization schemes large. How-

```

1 void eb(idxs: mref<? x idx>, offs: mref<? x idx>,
2        vals: mref<? x f32>, out: mref<? x ? x f32>){
3   slc.for(str s_b from 0 to num_batches){
4     str s_beg = slc.mem_str(offs[s_b]);
5     str s_end = slc.mem_str(offs[s_b+1]);
6     slc.for(str s_ptr from s_beg to s_end){
7       str s_idx = slc.mem_str(idxs[s_ptr]);
8       slc.for(str s_e from 0 to emb_len){
9         str s_val = slc.mem_str(vals[s_idx,s_e]);
10        slc.callback{
11          idx b = slc.to_val(s_b);
12          idx e = slc.to_val(s_e);
13          f32 val = slc.to_val(s_val);
14          f32 acc = out[b,e];
15          out[b,e] = acc + val; }}}}

```

(a) Unoptimized code.

```

7   str s_idx = slc.mem_str(idxs[s_ptr]);
8   slcv.for<vlen>((str s_e, str msk) from 0 to emb_len){
9     str s_val = slcv.mem_str<vlen>(vals[s_idx,s_e], msk);
10    slcv.callback{
11      idx b = slc.to_val(s_b);
12      idx e = slcv.to_val(s_e)[0];
13      vec<vlen x f32> val = slcv.to_val<vlen>(s_val);
14      vec<vlen x f32> acc = vload<vlen>(out[b,e]);
15      vstore<vlen>(acc + val, out[b,e], acc); }}}}

```

(b) Vectorized code.

```

7   str s_idx = slc.mem_str(idxs[s_ptr]);
8   str<vec<vlen x f32>> buf = slcv.buf_str(); // Buffer str
9   slcv.for<vlen>((str s_e, str msk) from 0 to emb_len){
10    str s_val = slcv.mem_str<vlen>(vals[s_idx,s_e], msk);
11    slc.push(buf, s_val); } // Push into the buffer
12   slcv.callback{ // Callback moved at the end of inner loop
13     idx b = slc.to_val(s_b);
14     vec<vlen x f32> buf_vec = slc.to_val(buf); // Get buffer
15     for(idx e = 0; e < emb_len; e++){ // Iterate buffer
16       vec<vlen x f32> val = buf_vec[e]; // Get buffered element
17       vec<vlen x f32> acc = vload<vlen>(out[b,e]);
18       vstore<vlen>(acc + val, out[b,e], acc); }}}}

```

(c) Buffered code.

```

7   str s_idx = slc.mem_str(idxs[s_ptr]);
8   str<vec<vlen x f32>> buf = slcv.buf_str();
9   slcv.for<vlen>((str s_e, str msk) from 0 to emb_len)(idx i=0){
10    str s_val = slcv.mem_str<vlen>(vals[s_idx,s_e], msk);
11    slc.push(buf, s_val); }
12   slcv.callback{
13     vec<vlen x f32> buf_vec = slc.to_val(buf);
14     for(idx e = 0; e < emb_len; e++){
15       vec<vlen x f32> val = buf_vec[e];
16       vec<vlen x f32> acc = vload<vlen>(out[i,e]);
17       vstore<vlen>(acc + val, out[i,e], acc); }}}}
18   slc.callback{ i++; } } // end of indices in segment loop

```

(d) Queue aligned code.

Figure 6.7: Progressive optimization of the EB SLC IR. The figure shows the unoptimized SLC program, the vectorized SLCV program, the buffered version that materializes embedding vectors as compound stream values, and the queue-aligned version that regenerates simple indices on the execute side. Gray code is unchanged from the previous version.

ever, Sections 5.1.3 and 5.3.1 motivate using inner-loop vectorization for sparse-dense tensor multiplication, assuming the dense tensor is in row-major order and has rows that are larger than the vector length. Embedding operations generally satisfy these assumptions. Hence, like the MLIR sparsifier [21], Ember only attempts inner-loop vectorization.

If the inner for-loop is legal, Ember first vectorizes its access code, then its execute code. As a first step, Ember vectorizes the inner for-loop and its streams. As the stream-to-value operations in the callbacks expect stream types, the algorithm adds a temporary cast operation for source materialization. Then, during callback vectorization, Ember recursively vectorizes the uses of these cast operations, producing full SLCV code.

We decided to implement our custom vectorization pass on SLC code rather than vectorizing in SCF before lowering because scalar SCF code is simpler to decouple than vector SCF code that contains complex instructions like mask manipulation, which hinder code analysis. Moreover, SLC exposes both the producing streams and the consuming callbacks, allowing Ember to choose how vector operands should be marshaled through the queue. For instance, an access unit like the TMU can handle out-of-bounds loads in two ways, where the benefit depends on context. The first option sends a mask to the CPU along with the vector operand to indicate the valid vector elements. Although the mask can be directly fed into Arm SVE instructions [146], mask overhead might impact the performance of tight loops. As a second option, the TMU can send zero-padded vector operands, which is the best choice when embedding vectors are multiples of the architectural vector length.

Embedding Vector Buffering

The vectorized code in Figure 6.6b, in addition to (de)serializing vector operands, also needs to (de)serialize their positions within the embedding vector (variable e). However, since the size of the embedding vector is fixed, we can generate these positions in the compute unit to reduce queue utilization.

Ember implements this optimization by (de)serializing embedding vectors as compound types. As shown in Figure 6.6c, the access unit pushes into the control queue one e_e (embedding-vector end) token for each embedding vector and, in the data queue, the position of the output embedding vector and all of its values. As the length of the embedding vector (`emb_len`) is constant, once the core reads an e_e token, it pops `emb_len` elements with a vectorized for-loop. This optimization greatly improves marshaling and compute efficiency.

The SLC IR greatly simplifies this optimization. With the SLC IR, Ember identifies optimization opportunities by examining iteration callbacks for stream-to-value operations whose input is an SLC(V) loop with constant bounds. Then, as shown in Figure 6.7c, Ember initializes a *buffer* stream of vector type (line 8) before the inner SLCV for-loop, where that

loop can push the loaded embedding elements (line 11). Finally, Ember moves the inner-loop body callback (lines 10–15 in Figure 6.7b) immediately after the loop (lines 12–18 in Figure 6.7c), adds a stream-to-value operation for the buffer (line 14 in Figure 6.7c), and adds a loop to iterate through it (lines 15–18 in Figure 6.7c). This transformation needs SLC because Ember must still see the loop bounds, the stream that produces each embedding value, and the callback that consumes those values.

Implementing the same optimization in the DLC IR would require complex analyses to reconstruct the dependency chain across the queue that Ember needs to search for optimization opportunities. Implementing the same optimization in the SCF IR is complex since the code is not in DAE form, and Ember would need to integrate such optimization in the decoupling logic, substantially complicating the compiler design. Moreover, since the SCF IR cannot represent DAE code, it would need to lower directly to the DLC IR.

Queue Alignment

Vector loads are more efficient when aligned to cache lines [37, 40]. However, as shown in Figure 6.6c, the access unit pushes both scalar and vector operands into the data queue. Hence, the vector loads that the execute unit performs from the queue are misaligned by scalar operands. Ember implements two optimizations to align queue accesses according to code characteristics.

For simpler functions like the EB in Figure 6.7c, where all segment IDs are just loop induction variables, Ember stores a reference of these indices in the core and increments them after the loop. This increment is triggered by a segment-end token, s_e in Figure 6.6d.

As shown in Figure 6.7d, Ember implements this optimization by examining iteration callbacks for stream-to-value operations that only read the induction variable of their own SLC(V) loop (e.g. line 13 in Figure 6.7c). Then Ember adds a new variable in the SLC loop (i variable in line 9 in Figure 6.7d), replaces all uses of the stream-to-value operations with that variable (line 16–17 in Figure 6.7d), and increments it in the end callback of its child loop (line 18 in Figure 6.7d).

However, for more complex models like MP, certain scalars, such as rescaling values, cannot be simplified. In this case, Ember preserves alignment by padding scalars to vectors while generating the DLC IR. As a further optimization, instead of sending segment IDs, Ember offloads full index calculation of partial and output results to the access unit and directly sends addresses to the core, reducing pressure on the core’s ALUs.

Similar to embedding buffering, implementing queue alignment in SCF is not possible, and implementing it in the DLC IR requires complex analyses to reconstruct the data flow, control flow, and queue (de)serialization of the input program. Queue alignment requires

control flow information to understand where to add the increment callback. It also requires queue (de)serialization to understand which alignment strategy to select. SLC provides both views at once: it exposes which values will cross the queue while preserving the loop structure needed to move or regenerate those values safely.

Model-Specific Optimizations and HW–SW Co-Design

While the optimizations described so far target general embedding operations, we also extended Ember to automatically implement model-specific optimizations. These extensions all follow the same pattern: when a model or format exposes additional structure, Ember represents that structure in SLC/DLC and uses it to specialize the access unit, the queue protocol, or the callback placement. This demonstrates the generality of the SLC IR and our compilation framework, as well as how Ember lends itself to HW–SW co-design.

Cache Utilization For instance, block-sparse attention mechanisms exhibit (1) large structured reuse within each block, (2) low reuse throughout blocks, and (3) no computation. Therefore, we extended the SLC IR, DLC IR, and TMU ISA with *store* streams to write directly into memory without passing through the core, and we extended load streams to (1) select what cache level to read from and (2) whether to issue (non-)temporal requests. As demonstrated in Section 6.3.3, this greatly improves core and cache utilization. Since the SLC IR preserves code structure, Ember can automatically identify callbacks with no computation, move the store operations into the access unit, and simplify the callback.

Efficient Compression Formats As shown in Figure 6.2, the EB function encodes all indices of all segments in a single vector, while storing the begin and end pointers of each segment in a second vector. This representation closely resembles the CSR (Compressed Sparse Row) format used in sparse linear algebra. However, modern EB implementations [114] typically avoid storing explicit begin and end pointers; instead, they store only the segment lengths and compute boundaries via prefix accumulation. To support this more efficient representation, we extend both the SLC IR and DLC IR with *accumulation streams*, which track EB segment boundaries by accumulating lengths rather than relying on explicit offsets [98]. This reduces memory overhead and matches contemporary runtime practices.

COO-Like Compression Formats Another extension targets COO-like formats, which differ fundamentally from CSR-style encodings. In these cases, callbacks cannot be naturally triggered at fiber boundaries, since segment structure is not encoded explicitly. To address this, we introduce *toggle callbacks*, a mechanism that enables computation to be triggered

Table 6.1: Evaluated code versions and reference implementations. Ember’s input is structured MLIR generated from sparse machine learning operations, and its output is TMU–CPU machine code. The emb-opt* variants progressively add SLC-level optimizations, while ref-dae is hand-optimized TMU–CPU code. In this evaluation, PyTorch programs enter Ember through torch-mlir [87].

Name	MLIR dialects	Description
emb-opt0	slc, scf	unoptimized Ember DAE code
emb-opt1	slcv, scf, vector	emb-opt0+vectorization
emb-opt2	slcv, scf, vector	emb-opt1+bufferization
emb-opt3	slcv, scf, vector	emb-opt2+queue alignment
ref-dae	tmu, scf, vector	hand-optimized TMU-CPU code

directly on sparse coordinate formats [118]. This enhancement broadens the applicability of the IR to workloads that rely on COO representations and ensures that compiler-driven optimizations remain effective across diverse sparse tensor formats.

6.3 Ember Evaluation

The evaluation demonstrates that Ember unlocks DAE performance without requiring hand-written DAE programs. SLC-level optimizations improve unoptimized DAE code by 6.6×–21×, model-specific optimizations extend the same framework to sparse LLM embedding patterns, and automatically generated Ember code reaches 99% of hand-optimized TMU–CPU performance. Section 6.3.2 first quantifies the impact of vectorization, bufferization, and queue alignment. Section 6.3.3 then measures the benefit of model-specific optimizations for LLMs, and Section 6.3.4 compares Ember-generated code against hand-optimized implementations.

6.3.1 Evaluation Methodology

To the best of our knowledge, this is the first work that explores DAE optimizations for embedding operations. As discussed at the beginning of this chapter, we focus the compiler evaluation on machine learning embedding operations because scientific sparse applications are typically exposed through standardized library kernels that can be replaced with hand-optimized DAE implementations, whereas machine learning kernels are generated from high-level models and frequently fused, making manual DAE programming impractical. As such, the handwritten code we compare Ember against is SCF+VEC MLIR code that we manually

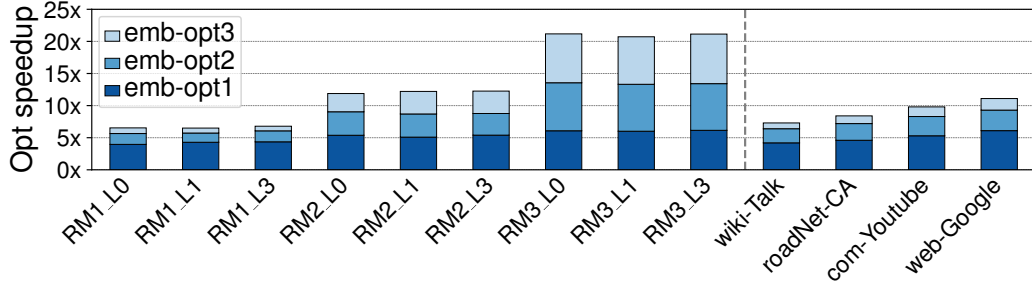


Figure 6.8: Speedup of SLC-level Ember optimizations over unoptimized Ember DAE code (emb-opt0). Bars report MP models and EB operations for the DLRM configurations in Table 4.5 under different input-locality settings. Stacked segments show the incremental gains from vectorization (emb-opt1), bufferization (emb-opt2), and queue alignment (emb-opt3), as defined in Table 6.1. Vectorization provides the largest single improvement across workloads, while bufferization and queue alignment add larger gains on longer EB loops, especially RM3. Overall, all optimizations combined improve performance by 6.6 $\times$ –21 $\times$ on EB and up to 11 $\times$ on MP models.

decoupled and iteratively optimized through performance analyses with gem5. The experimental setting follows the DAE embedding study in Section 5.4.1, using a datacenter-class HBM multicore processor with one TMU per core to offload embedding lookups. Table 6.1 summarizes the code used in the experiments. This evaluation focuses on the TMU because, among the DAE designs considered in this thesis, it exposes both vectorization and multiple callbacks. We believe similar considerations apply to the other DAE designs the DLC IR can abstract [161, 110, 14].

6.3.2 Impact of SLC-Level Optimizations

Figure 6.8 shows the performance impact of general optimizations such as vectorization (emb-opt1), bufferization (emb-opt2), and queue alignment (emb-opt3) over unoptimized Ember-generated code (emb-opt0) for the EB function and more compute-intensive MP models.

For the EB function, we evaluated the three DLRMs in Table 4.5, each running representative synthetic inputs [50] with low (L0), medium (L1), and high (L2) locality. Overall, all optimizations combined (emb-opt3) achieve 6.6 $\times$, 12.1 $\times$, and 21 $\times$ better performance over unoptimized code (emb-opt0) for RM1, RM2, and RM3, respectively. Vectorization is consistently the most impactful optimization with a 5.13 $\times$ speedup and only 17% deviation, whereas other optimizations deliver widely different performance improvements.

To better understand these results, Figure 6.9 shows how these optimizations impact the throughput at which the compute unit reads and the access unit writes into the L2 queue.

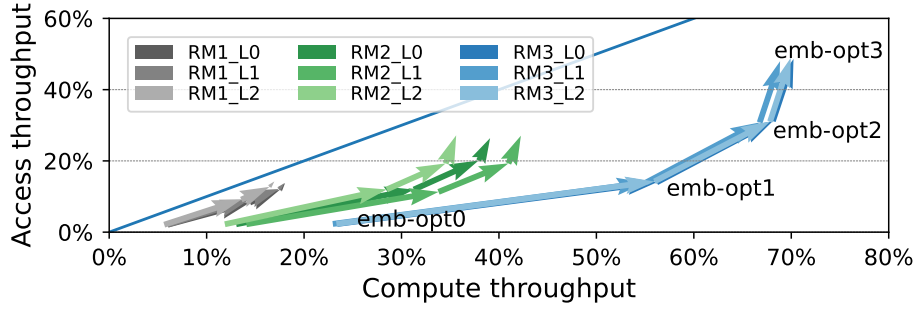


Figure 6.9: Impact of Ember optimizations on compute-side and access-side throughput for EB operations. The x-axis reports compute throughput, i.e., how quickly the CPU consumes queue entries, and the y-axis reports access throughput, i.e., how quickly the TMU produces them. Each arrow follows one DLRM/locality case through the optimization sequence from emb-opt0 to emb-opt3; colors identify DLRM configurations, and shades identify locality settings. The blue diagonal marks the balanced point where access throughput equals compute throughput. Vectorization (emb-opt1) primarily increases compute throughput, while bufferization and queue alignment (emb-opt2, emb-opt3) also increase access throughput, moving points closer to the balanced high-throughput region. Overall, Ember improves performance by optimizing both sides of the DAE pipeline rather than only the compute call-back.

Compute optimizations move upward, whereas memory optimizations move rightward. The blue line indicates where compute-unit throughput equals access-unit throughput. Optimizing only compute code cannot move above the blue line, since the access unit would not be able to marshal enough data to process. This would improve throughput only up to $8\times$ (RM3, emb-opt0) before the access unit becomes the bottleneck (blue line). However, because of the SLC IR, Ember can perform global optimizations on both access and compute code and move both rightward and upward in the plot, improving performance by up to $21\times$ (RM3, emb-opt3). For RM1, the most control-intensive model (shorter loops), vectorization already saturates throughput, leaving other optimizations little room for improvement. For RM2 and RM3, by reducing coordinate overhead, bufferization helps move closer to the blue line. For RM3, the model with the largest loops, queue alignment removes index overhead, pushing performance closer to the limit.

As shown in Figure 6.8, MP models have similar trends, and the optimization impact depends on their compute intensity.

6.3.3 Impact of Model-Specific Optimizations

As discussed in Section 6.2.3, Ember lends itself to model-specific optimizations. Figure 6.10 shows that, in block-sparse attention mechanisms, loading highly reused embedding

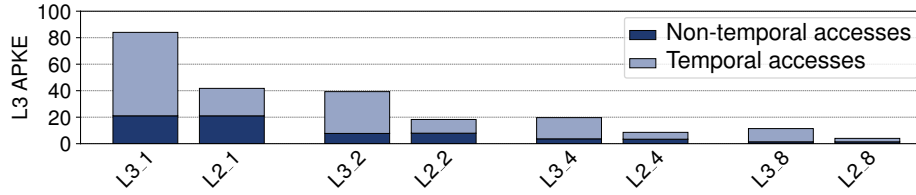


Figure 6.10: L3 accesses per kilo element (APKE) for the BigBird gather function under different block sizes and TMU load configurations. For each block size (1, 2, 4, and 8), L3_* loads embedding vectors from the LLC, whereas L2_* reuses embedding vectors from L2 when possible. Stacked bars separate non-temporal accesses, which load sparse indices, from temporal accesses, which load embedding vectors. Larger blocks increase temporal reuse and reduce APKE in both configurations. Reading temporal embedding data from L2 further filters LLC traffic, reducing total APKE by roughly 50%–74% across block sizes. Overall, model-specific cache placement lets Ember exploit block reuse in sparse attention and substantially reduce LLC pressure.

blocks from L2 rather than LLC filters 67%–74% of the embedding reads and 50%–65% of the overall accesses, which include non-temporal loads for indices, with larger reductions on larger block sizes with more intrinsic reuse. By directly writing data with the store streams instead of going through the core, Ember enables efficient gather operations with low resource utilization.

6.3.4 Comparison with Hand-Optimized Code

Figure 6.11 shows the performance of DAE code automatically generated and optimized by Ember (emb-opt3) compared to hand-optimized code (ref-dae) for all model classes in Section 4.2.2. In addition to the optimizations discussed in Section 6.2.3, hand-optimized code also includes low-level, CPU-specific optimizations to improve callback invocations. These CPU-specific optimizations include, for instance, (1) reordering the if-cases of multi-callback code (e.g. Figure 6.6d) according to their taken frequency or (2) setting the values of control tokens so they can be directly used in compute code (e.g. to increment variables).

Overall, these CPU-specific optimizations primarily affect multi-callback code such as MP, EB, and SpMM, yielding performance improvements of up to 5%, with an average geometric mean improvement of 1%. The limited impact of these low-level optimizations arises because, as discussed in Section 6.3.2, the optimizations introduced in Section 6.2.3 already push the architecture close to its limits, leaving little room for further gains. Nevertheless, because these low-level optimizations are highly CPU-specific, we chose not to integrate them into Ember, which is designed to provide a more general solution for a larger class of architectures.

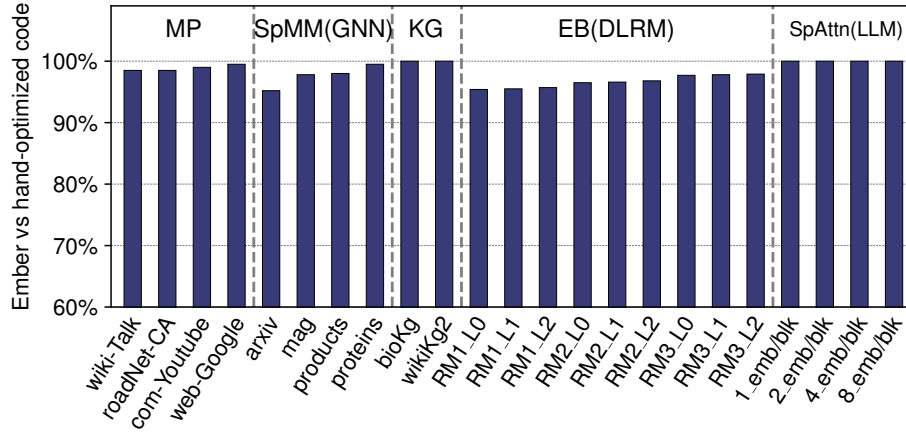


Figure 6.11: Performance of DAE code automatically generated and optimized by Ember (emb-opt3) relative to hand-optimized TMU-CPU code (ref-dae). Bars are grouped by workload family from Section 4.2.2: MP, SpMM-style graph convolution (SpMM/GNN), knowledge graphs (KG), DLRM EB, and sparse-attention LLMs. The y-axis is normalized to hand-optimized code, so 100% indicates equal performance. Ember reaches 95%–100% of hand-optimized performance across all cases and a 99% geometric mean, showing that the SLC-level optimizations recover nearly all of the performance of manually tuned DAE implementations.

Ultimately, we believe that the optimizations presented in Section 6.2.3 are sufficient to fully unlock the potential of the DAE architectures and embedding operations studied in this thesis. Ember targets structured sparse–dense and embedding-style loops; it does not attempt to compile arbitrary pointer-heavy programs or general control flow to DAE hardware.

6.4 Summary

This chapter introduced Ember, a compiler stack for generating optimized DAE code from structured sparse machine learning operations. The central challenge is that SCF retains the input program structure but does not expose queue-based decoupling, whereas DLC exposes decoupling but loses the global control-flow and data-flow information needed for optimization. Ember addresses this with the SLC IR, which represents access streams, compute callbacks, and stream-to-value uses in a structured form. This enables vectorization, embedding buffering, queue alignment, and model-specific optimizations before lowering to explicit DLC queues and target-specific TMU-CPU code. The evaluation shows that these optimizations improve generated code by 6.6×–21× over unoptimized DAE code and achieve 99% of hand-optimized performance, removing the need for users to manually write DAE programs.

Chapter 7

Conclusions

This thesis demonstrates that DAE architectures can substantially accelerate irregular sparse workloads in scientific computing and machine learning without sacrificing programmability. The results lead to three main lessons about how future systems should support sparse tensor algebra.

7.1 Lessons Learned

The main lessons learned from this thesis are:

- Traditional compute cores, which tightly couple memory access, control flow, and computation, cannot be scaled efficiently to support the irregular operations that dominate sparse tensor algebra. The two primary bottlenecks are *sparse tensor traversal* and *sparse tensor merging*. Sparse tensor traversal generates irregular memory accesses that require substantially more memory-level parallelism than conventional cores can efficiently sustain. Sparse tensor merging introduces data-dependent control flow that defeats speculative mechanisms tuned for dense and regular workloads. Scaling conventional structures such as reorder buffers, load-store queues, and miss-status handling registers yields diminishing performance returns while incurring steep area and power costs. Therefore, sustaining full-core throughput is fundamentally impractical when memory access, control flow, and computation remain tightly coupled within the same compute pipeline.
- The right architectural boundary is to specialize sparse traversal, coordinate merging, and data marshaling while preserving general-purpose vector cores for arithmetic. General-purpose processors already provide many features that sparse tensor operations need: large memory capacity, flexible arithmetic, virtual memory, mature soft-

ware support, and integration with real systems. However, they are inefficient at irregular sparse traversal and merging. DAE architectures expose a better split by separating memory access from computation into queue-connected units that can be optimized independently. By mapping traversal and merging to a specialized vector access unit such as the TMU, and computation to a general-purpose core, the system gains much of the efficiency of specialization without giving up flexibility, memory capacity, or software integration. The results also show that spatio-temporal locality is a key factor in determining the amount of memory-level parallelism, cache filtering, and specialization required by each workload.

- Efficient DAE architectures require automatic compilation, and that compilation is only practical with the right intermediate abstractions. The access unit and execute unit communicate through queues, so programmers must describe how data and control tokens are serialized, deserialized, and consumed across the access/execute boundary. This is difficult to write manually and also difficult to optimize automatically once the code has been fully decoupled, because explicit queue code loses much of the original control-flow and data-flow structure. Ember shows that the right compiler abstraction can overcome this challenge. With its SLC IR, Ember represents DAE code in a structured form that preserves control flow and data flow while exposing access streams, compute callbacks, and stream-to-value uses. This enables iterative DAE optimizations and allows automatically generated code to match the performance of hand-optimized implementations.

Together, these results show that sparse tensor algebra can be accelerated by specializing the irregular parts of the computation without replacing the whole processor or imposing a low-level DAE programming model on users.

7.2 Research Outputs, Awards, and Follow-On Work

7.2.1 Publications and Patents

The contributions of this thesis led to the following research outputs:

- The architectural analysis of sparse tensor algebra for scientific computation presented in Section 4.1 formed the basis of SpChar, published in the *Journal of Parallel and Distributed Computing* [135].
- The architectural analysis of sparse tensor algebra for machine learning presented in Sections 4.2 and 5.4 is currently under submission.

- The TMU design described in Chapter 5 was published at the *56th IEEE/ACM International Symposium on Microarchitecture (MICRO)* [140].
- The TMU design presented in Chapter 5 has also been patented [139].
- The Ember compiler presented in Chapter 6, developed in collaboration with researchers at Stanford University, appeared at the *24th IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* [138].

7.2.2 Awards

This work was recognized through the following awards:

- 2025 Machine Learning and Systems Rising Star Award by MLCommons [97].
- A scholarship from the Government of Catalonia (AGAUR FI grant) [3].

7.2.3 Follow-On Research Projects

Beyond the thesis contributions themselves, this work catalyzed several follow-on projects:

- Francisco (Xicu) Marí Prats’s master’s thesis, *Integrating the Tensor Marshaling Unit for Sparse Tensor Algebra with a RISC-V Processor*, presented the first RTL implementation and SoC integration of the Tensor Marshaling Unit (TMU) [93]. The prototype integrated the TMU with a CVA6 RISC-V core in an OpenPiton-based system, demonstrated the feasibility of the TMU in a real SoC, and established the first milestone of a larger effort toward enabling the TMU in production systems.
- Diego Gorfini’s master’s thesis builds on Xicu Marí Prats’s first TMU SoC prototype by improving the communication path between the core and the TMU [94]. This work integrates the decoupled SpMV access unit with Cohort’s coherent shared-memory queues, avoiding memory-mapped I/O and ISA extensions. It also introduces a hybrid tile organization that co-locates the core, Cohort, and the TMU, allowing the accelerator to deliver operands directly into the tile’s private L2 cache through a custom arbiter. RTL simulations show that this organization reduces inter-tile communication overhead and improves performance over the original multi-tile integration.
- A collaboration between the Barcelona Supercomputing Center and UC Santa Barbara to improve OpenPiton as a research platform for accelerator integration, with a particular focus on supporting future TMU prototypes in many-core RISC-V systems.

7.2.4 Research Appointments and Professional Contributions

During his PhD, Marco spent substantial time at the Barcelona Supercomputing Center, Arm, and Stanford University. During his research appointment at the Barcelona Supercomputing Center, he co-authored Hades, a network-on-chip accelerator for data movement primitives. During his visiting appointments at Stanford University, he co-authored FuseFlow, a fusion-centric compilation framework for sparse deep learning [74].

In the final year of his PhD, Marco joined Samsung Semiconductor USA as a Staff Engineer and Tech Lead for AI supercomputer design. He is currently a Principal AI Performance Engineer at Arm USA.

Chapter 8

Future Work

The TMU and Ember demonstrate that DAE execution can make sparse tensor workloads both more efficient and more programmable. They also expose several directions for extending this approach from a research prototype into a broader computing substrate:

8.1 Broader Workload and System Characterization

- A natural extension of the characterization and modeling effort in Chapter 4 is to broaden the set of evaluated workloads, input datasets, and target architectures. This includes additional scientific kernels, emerging machine learning models, training workloads, distributed executions, and systems with different memory hierarchies. Such a study would help distinguish which bottlenecks are fundamental consequences of sparsity from those that are specific to the workloads and machines evaluated in this thesis, and would provide stronger guidance for future DAE designs.

8.2 TMU Implementation and System Integration

- A near-term direction is to continue extending and optimizing the RTL implementation of the TMU. Xicu Mari Prats’s master’s thesis implemented the first scalar TMU prototype, consisting of one Traversal Unit (TU), one layer, and one lane [93]. Since then, we have extended the design with multiple layers, enabling nested-loop traversal, and multiple lanes, enabling sparse coordinate merging. The next step is to use the multi-lane design for vectorization and to integrate the TMU with a vectorized core. This path would progressively close the gap between the initial scalar prototype and the multi-layer, multi-lane TMU evaluated in Chapter 5.

- A complementary direction is to continue improving the integration of the TMU into real systems-on-chip. This line of work follows Diego Gorfini’s master’s thesis, which builds on the first TMU SoC prototype by improving the interface between the TMU, the core, Cohort, and OpenPiton [94]. The current design uses coherent shared-memory queues for core-TMU communication, avoiding memory-mapped I/O and ISA extensions, and explores a hybrid tile organization that co-locates the TMU, Cohort, and the core to reduce inter-tile communication overhead. Continuing this direction requires refining the core-TMU interface, the Cohort queue protocol, and the OpenPiton integration, exposing practical questions about cache arbitration, queue management, batching, coherence, and software/runtime support.

8.3 Extending the TMU Architecture

- Another architectural direction is to optimize the TMU ISA and microarchitecture for dense tensor data movement. The TMU already supports dense tensor traversal, as described in Section 5.1.2, but its design is primarily motivated by sparse traversal and merging. Dense tensor workloads do not require coordinate merging, but they can still benefit from efficient tiling, prefetching, and data marshaling between memory and compute units, similar in spirit to NVIDIA’s Tensor Memory Accelerators (TMAs) [106]. Making dense traversal more efficient would turn the TMU into a unified access unit for both sparse and dense tensor movement, reducing hardware fragmentation and providing a common programming target for mixed sparse–dense workloads.
- A further architectural avenue is to extend TMU-like access units across node boundaries. Modern distributed machine learning systems spend substantial time moving data through remote memory accesses and collective operations such as allreduce. A distributed TMU could orchestrate remote fetches, reductions, and collective data movement, exposing communication-level parallelism in the same way that the TMU exposes memory-level parallelism within a node. The key challenges are preserving ordering, protection, programmability, and integration with existing network and runtime systems.

8.4 Extending Ember

- From the compiler perspective, an important direction is to extend Ember beyond sparse–dense tensor multiplications to full sparse tensor algebra. The primary challenge is representing sparse coordinate merging in a structured form. Unlike sparse–dense embedding operations, general sparse tensor algebra often requires dynamically advancing multiple sparse fibers, which naturally introduces while loops and if–then–else constructs that do not map cleanly onto the current SLC IR described in Section 6.2.2. Addressing this limitation may require introducing a custom structured merge operation, extending SLC with richer control constructs, or starting the lowering from a higher-level tensor IR such as linalg.

Together, these directions point toward a broader vision: programmable access units and compiler support that make irregular data movement a first-class part of future processors, rather than an inefficiency hidden inside conventional cores.

Funding and Project Acknowledgments

This work is supported by the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF; by the Ministry for Digital Transformation and Public Service through the Càtedra Chip UPC project [grant number TSI-069100-2023-0015] and within the framework of the Recovery, Transformation and Resilience Plan - NextGenerationEU [REGAGE22e00058408992]; by the Generalitat of Catalunya through contract [2021-SGR-00763]; by DARPA under the Machine learning and Optimization-guided Compilers for Heterogeneous Architectures (MOCHA) program [HR00112520038]; by the PRISM center, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA; by the Stanford Portal Center; and by the Arm-BSC Center of Excellence. M. Siracusa has been partially supported through an FI fellowship [2022FI_B 00969]. O. Hsu has been partially supported through a National Science Foundation Graduate Research Fellowship (GRFP). A. Armejach is a Serra Hunter Fellow. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the aforementioned funding agencies.

Bibliography

- [1] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software. <https://www.tensorflow.org/>. Software available from tensorflow.org. Accessed: 2026-07-10.
- [2] Acun, B., Murphy, M., Wang, X., Nie, J., Wu, C.-J., and Hazelwood, K. (2021). Understanding training efficiency of deep learning recommendation models at scale. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 802–814.
- [3] Agency for Management of University and Research Grants (2026). Grants for PhD studies. <https://agaur.gencat.cat/en/beques-i-ajuts/Research-grants/ajuts-predotorals/index.html>. Accessed: 2026-05-30.
- [4] Ahrens, W., Donenfeld, D., Kjolstad, F., and Amarasinghe, S. (2023). Looplets: A language for structured coiteration. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*, CGO 2023, pages 41–54, New York, NY, USA. Association for Computing Machinery.
- [5] Ainsworth, S. and Jones, T. M. (2016). Graph prefetching using data structure knowledge. In Ozturk, O., Ebcioğlu, K., Kandemir, M. T., and Mutlu, O., editors, *Proceedings of the 2016 International Conference on Supercomputing, ICS 2016, Istanbul, Turkey, June 1-3, 2016*, pages 1–11. ACM. Article 39, 11 pages.
- [6] Aktulga, H. M., Buluç, A., Williams, S., and Yang, C. (2014). Optimizing sparse matrix-multiple vectors multiplication for nuclear configuration interaction calculations. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*, pages 1213–1222.
- [7] Alappat, C., Meyer, N., Laukemann, J., Gruber, T., Hager, G., Wellein, G., and Wettig, T. (2022). Execution-cache-memory modeling and performance tuning of sparse matrix-vector multiplication and lattice quantum chromodynamics on A64FX. *Concurrency and Computation: Practice and Experience*, 34(20):e6512.
- [8] Amazon Web Services (2022). Announcing New Amazon EC2 C7g Instances Powered by AWS Graviton3 Processors. <https://aws.amazon.com/about-aws/whats-new/2022/05/amazon-ec2-c7g-instances-powered-aws-graviton3-processors/>. Accessed: 2026-06-01.

- [9] Amazon Web Services (2026). AWS Graviton Technical Guide. <https://aws.github.io/graviton/>. Accessed: 2026-06-01.
- [10] Armejach, A., Casas, M., and Moretó, M. (2019). Design trade-offs for emerging HPC processors based on mobile market technology. *J. Supercomput.*, 75(9):5717–5740.
- [11] Asgari, B., Hadidi, R., Dierberger, J., Steinichen, C., Marfatia, A., and Kim, H. (2021). Copernicus: Characterizing the performance implications of compression formats used in sparse workloads. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*, pages 1–12.
- [12] Asgari, B., Hadidi, R., Krishna, T., Kim, H., and Yalamanchili, S. (2020). Alrescha: A lightweight reconfigurable sparse-computation accelerator. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 249–260.
- [13] Bansal, M., Hsu, O., Olukotun, K., and Kjolstad, F. (2023). Mosaic: An interoperable compiler for tensor algebra. *Proc. ACM Program. Lang.*, 7(PLDI):394–419. Article 122, 26 pages.
- [14] Barredo, A., Armejach, A., Beard, J., and Moreto, M. (2021). Planar: a programmable accelerator for near-memory data rearrangement. In *Proceedings of the 35th ACM International Conference on Supercomputing, ICS '21*, pages 164–176, New York, NY, USA. Association for Computing Machinery.
- [15] Barrett, R., Berry, M., Chan, T. F., Demmel, J., Donato, J., Dongarra, J., Eijkhout, V., Pozo, R., Romine, C., and van der Vorst, H. (1994). *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics.
- [16] Beamer, S., Asanović, K., and Patterson, D. (2015). The GAP Benchmark Suite. arXiv:1508.03619. Accessed: 2026-07-10.
- [17] Bean, A., Kapre, N., and Cheung, P. (2015). G-dma: improving memory access performance for hardware accelerated sparse graph computation. In *2015 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pages 1–6.
- [18] Bell, N. and Garland, M. (2009). Implementing sparse matrix-vector multiplication on throughput-oriented processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, pages 1–11.
- [19] Berners-Lee, T., Hendler, J., and Lassila, O. (2001). The Semantic Web: A new form of web content that is meaningful to computers will unleash a revolution of new possibilities. *Scientific American*, 284(5):34–43.
- [20] Bharadwaj, V., Buluc, A., and Demmel, J. (2022). Distributed-memory sparse kernels for machine learning. In *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 47–58, Los Alamitos, CA, USA. IEEE Computer Society.
- [21] Bik, A., Koanantakool, P., Shpeisman, T., Vasilache, N., Zheng, B., and Kjolstad, F. (2022). Compiler support for sparse tensor computations in MLIR. *ACM Trans. Archit. Code Optim.*, 19(4):1–25. Article 50, 25 pages.

- [22] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs. Software, version 0.3.13. <http://github.com/google/jax>. Accessed: 2026-07-10.
- [23] Brock, B., McMillan, S., Buluç, A., Mattson, T., and Moreira, J. (2022). GraphBLAS C++ specification. <https://github.com/GraphBLAS/graphblas-api-cpp>. Draft Version 0.1a. Accessed: 2026-07-10.
- [24] Buluc, A. and Gilbert, J. R. (2008). On the representation and multiplication of hyparsparse matrices. In *2008 IEEE International Symposium on Parallel and Distributed Processing*, pages 1–11.
- [25] Cabezas, V. C. and Püschel, M. (2014). Extending the roofline model: Bottleneck analysis with microarchitectural constraints. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*, pages 222–231.
- [26] Carter Edwards, H., Trott, C. R., and Sunderland, D. (2014). Kokkos: Enabling many-core performance portability through polymorphic memory access patterns. *J. Parallel Distrib. Comput.*, 74(12):3202–3216.
- [27] Chen, C., Wang, Y., Yang, J., Liu, Y., Lu, M., Zheng, Z., He, B., Wong, W.-F., You, L., Sun, P., Zhao, Y., Hu, F., and Rudoff, A. (2023). OpenEmbedding: A distributed parameter server for deep learning recommendation models using persistent memory. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pages 2976–2987.
- [28] Chen, P.-H., Cheng, B. W., Oduoza, M., Xie, Z., Koul, K., Ravipati, S. G., Mei, Y., Lu, R., Carsello, A., Horowitz, M., and Raina, P. (2025). Opal: A 16nm coarse-grained reconfigurable array for full sparse ML applications. In *2025 IEEE Custom Integrated Circuits Conference (CICC)*, pages 1–3.
- [29] Chou, S., Kjolstad, F., and Amarasinghe, S. (2018). Format abstraction for sparse tensor algebra compilers. *Proc. ACM Program. Lang.*, 2(OOPSLA):1–30. Article 123, 30 pages.
- [30] Cong, J., Liu, B., Neuendorffer, S., Noguera, J., Vissers, K., and Zhang, Z. (2011). High-level synthesis for FPGAs: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 30(4):473–491.
- [31] Criteo AI Lab (2024). Criteo 1TB Click Logs Dataset. <https://ailab.criteo.com/download-criteo-1tb-click-logs-dataset/>. Accessed: 2026-07-10.
- [32] Dadu, V., Weng, J., Liu, S., and Nowatzki, T. (2019). Towards general purpose acceleration by exploiting common data-dependence forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 924–939.
- [33] Davis, T. A. and Hu, Y. (2011). The university of florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1):1–25. Article 1, 25 pages.

- [34] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T., editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- [35] Ding, C. and Zhong, Y. (2003). Predicting whole-program locality through reuse distance analysis. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation*, PLDI '03, pages 245–257, New York, NY, USA. Association for Computing Machinery.
- [36] Doerfler, D., Fatollahi-Fard, F., MacLean, C., Nguyen, T., Williams, S., Wright, N., and Siracusa, M. (2021). Experiences porting the SU3_Bench microbenchmark to the Intel Arria 10 and Xilinx Alveo U280 FPGAs. In *Proceedings of the 9th International Workshop on OpenCL, IWOCL '21*, pages 1–9, New York, NY, USA. Association for Computing Machinery. Article 1, 9 pages.
- [37] Eichenberger, A. E., Wu, P., and O'Brien, K. (2004). Vectorization for SIMD architectures with alignment constraints. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*, PLDI '04, pages 82–93, New York, NY, USA. Association for Computing Machinery.
- [38] Esmaili-Dokht, P., Sgherzi, F., Girelli, V. S., Boixaderas, I., Carmin, M., Monemi, A., Armejach, A., Mercadal, E., Llorc, G., Radojković, P., Moreto, M., Giménez, J., Martorell, X., Ayguadé, E., Labarta, J., Confalonieri, E., Dubey, R., and Adlard, J. (2024). A mess of memory system benchmarking, simulation and application profiling. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 136–152.
- [39] Fey, M. and Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- [40] Fireman, L., Petrank, E., and Zaks, A. (2007). New algorithms for SIMD alignment. In *Proceedings of the 16th International Conference on Compiler Construction*, CC'07, pages 1–15, Berlin, Heidelberg. Springer-Verlag.
- [41] Firoozshahian, A., Coburn, J., Levenstein, R., Nattoji, R., Kamath, A., Wu, O., Grewal, G., Aepala, H., Jakka, B., Dreyer, B., Hutchin, A., Diril, U., Nair, K., Aredestani, E. K., Schatz, M., Hao, Y., Komuravelli, R., Ho, K., Abu Asal, S., Shajrawi, J., Quinn, K., Sreedhara, N., Kansal, P., Wei, W., Jayaraman, D., Cheng, L., Chopda, P., Wang, E., Bikumandla, A., Karthik Sengottuvel, A., Thottempudi, K., Narasimha, A., Dodds, B., Gao, C., Zhang, J., Al-Sanabani, M., Zehtabioskuie, A., Fix, J., Yu, H., Li, R., Gondkar, K., Montgomery, J., Tsai, M., Dwarakapuram, S., Desai, S., Avidan, N., Ramani, P., Narayanan, K., Mathews, A., Gopal, S., Naumov, M., Rao, V., Noru, K., Reddy, H., Venkatapuram, P., and Bjorlin, A. (2023). MTIA: First generation silicon targeting Meta's recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, pages 1–13, New York, NY, USA. Association for Computing Machinery. Article 80, 13 pages.
- [42] Fowers, J., Ovtcharov, K., Strauss, K., Chung, E. S., and Stitt, G. (2014). A high memory bandwidth FPGA accelerator for sparse matrix-vector multiplication. In *2014*

- IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines*, pages 36–43.
- [43] Gale, T., Zaharia, M., Young, C., and Elsen, E. (2020). Sparse GPU kernels for deep learning. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14.
- [44] Gao, J., Ji, W., Chang, F., Han, S., Wei, B., Liu, Z., and Wang, Y. (2023). A systematic survey of general sparse matrix-matrix multiplication. *ACM Comput. Surv.*, 55(12):1–36. Article 244, 36 pages.
- [45] Giannoula, C., Fernandez, I., Gómez-Luna, J., Koziris, N., Goumas, G., and Mutlu, O. (2022). SparseP: Efficient sparse matrix vector multiplication on real processing-in-memory architectures. In *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 288–291.
- [46] Ginart, A., Naumov, M., Mudigere, D., Yang, J., and Zou, J. (2021). Mixed dimension embeddings with application to memory-efficient recommendation systems. In *2021 IEEE International Symposium on Information Theory (ISIT)*, pages 2786–2791. IEEE Press.
- [47] Google (2021). MLIR Sparsifier. <https://developers.google.com/mlir-sparsifier>. Accessed: 2026-07-10.
- [48] Grigoras, P., Burovskiy, P., Hung, E., and Luk, W. (2015). Accelerating SpMV on FPGAs by compressing nonzero values. In *2015 IEEE 23rd Annual International Symposium on Field-Programmable Custom Computing Machines*, pages 64–67.
- [49] Gupta, U., Hsia, S., Saraph, V., Wang, X., Reagen, B., Wei, G.-Y., Lee, H.-H. S., Brooks, D., and Wu, C.-J. (2020a). DeepRecSys: A system for optimizing end-to-end at-scale neural recommendation inference. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 982–995.
- [50] Gupta, U., Wu, C., Wang, X., Naumov, M., Reagen, B., Brooks, D., Cotel, B., Hazelwood, K., Hempstead, M., Jia, B., Lee, H. S., Malevich, A., Mudigere, D., Smelyanskiy, M., Xiong, L., and Zhang, X. (2020b). The architectural implications of Facebook’s DNN-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 488–501, Los Alamitos, CA, USA. IEEE Computer Society.
- [51] Ham, T. J., Aragón, J. L., and Martonosi, M. (2015). DeSC: Decoupled supply-compute communication management for heterogeneous architectures. In *Proceedings of the 48th International Symposium on Microarchitecture, MICRO-48*, pages 191–203, New York, NY, USA. Association for Computing Machinery.
- [52] Hamilton, W. L., Ying, R., and Leskovec, J. (2017). Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, pages 1025–1035, Red Hook, NY, USA. Curran Associates Inc.

- [53] Hegde, K., Asghari-Moghaddam, H., Pellauer, M., Crago, N., Jaleel, A., Solomonik, E., Emer, J., and Fletcher, C. W. (2019). ExTensor: An accelerator for sparse tensor algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO '52*, pages 319–333, New York, NY, USA. Association for Computing Machinery.
- [54] Heroux, M., Dongarra, J., and Luszczek, P. (2013). HPCG benchmark technical specification. Technical Report SAND2013-8752, Sandia National Laboratories.
- [55] Heroux, M. A. and Dongarra, J. (2013). Toward a new metric for ranking high performance computing systems. Technical Report SAND2013-4744, Sandia National Laboratories.
- [56] Hestenes, M. R. and Stiefel, E. (1952). Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436.
- [57] Hsu, O., Strange, M., Sharma, R., Won, J., Olukotun, K., Emer, J. S., Horowitz, M. A., and Kjølstad, F. (2023). The sparse abstract machine. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2023*, pages 710–726, New York, NY, USA. Association for Computing Machinery.
- [58] Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. (2020). Open graph benchmark: datasets for machine learning on graphs. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA. Curran Associates Inc. Article 1855, 16 pages.
- [59] Hussain, M., Abhishek, G., Buluc, A., and Azad, A. (2022). Parallel algorithms for adding a collection of sparse matrices. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 285–294, Los Alamitos, CA, USA. IEEE Computer Society.
- [60] Hwang, R., Kim, T., Kwon, Y., and Rhu, M. (2020). Centaur: A chiplet-based, hybrid sparse-dense accelerator for personalized recommendations. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 968–981.
- [61] Ibrahim, M. A., Kayiran, O., and Aga, S. (2021). Efficient cache utilization via model-aware data placement for recommendation models. In *Proceedings of the International Symposium on Memory Systems, MEMSYS '21*, pages 1–11, New York, NY, USA. Association for Computing Machinery. Article 2, 11 pages.
- [62] Intel Corporation (2023). Intel architecture instruction set extensions and future features. <https://software.intel.com/content/dam/develop/external/us/en/documents-tps/architecture-instruction-set-extensions-programming-reference.pdf>. Document 319433-048. Accessed: 2026-07-10.
- [63] Jimborean, A., Koukos, K., Spiliopoulos, V., Black-Schaffer, D., and Kaxiras, S. (2014). Fix the code. don't tweak the hardware: A new compiler approach to voltage-frequency scaling. In *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization, CGO '14*, pages 262–272, New York, NY, USA. Association for Computing Machinery.

- [64] Joglekar, M. R., Li, C., Chen, M., Xu, T., Wang, X., Adams, J. K., Khaitan, P., Liu, J., and Le, Q. V. (2020). Neural input search for large scale recommendation models. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20, pages 2387–2397, New York, NY, USA. Association for Computing Machinery.
- [65] Jouppi, N., Kurian, G., Li, S., Ma, P., Nagarajan, R., Nai, L., Patil, N., Subramanian, S., Swing, A., Towles, B., Young, C., Zhou, X., Zhou, Z., and Patterson, D. A. (2023). TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, pages 1–14, New York, NY, USA. Association for Computing Machinery. Article 82, 14 pages.
- [66] Kanellopoulos, K., Vijaykumar, N., Giannoula, C., Azizi, R., Koppula, S., Ghiasi, N. M., Shahroodi, T., Luna, J. G., and Mutlu, O. (2019). SMASH: Co-designing software compression and hardware-accelerated indexing for efficient sparse matrix operations. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '52, pages 600–614, New York, NY, USA. Association for Computing Machinery.
- [67] Ke, L., Gupta, U., Cho, B. Y., Brooks, D., Chandra, V., Diril, U., Firoozshahian, A., Hazelwood, K., Jia, B., Lee, H.-H. S., Li, M., Maher, B., Mudigere, D., Naumov, M., Schatz, M., Smelyanskiy, M., Wang, X., Reagen, B., Wu, C.-J., Hempstead, M., and Zhang, X. (2020). RecNMP: Accelerating personalized recommendation with near-memory processing. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, ISCA '20, pages 790–803. IEEE Press.
- [68] Kennedy, K. and Allen, J. R. (2001). *Optimizing compilers for modern architectures: a dependence-based approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [69] Kjolstad, F., Ahrens, W., Kamil, S., and Amarasinghe, S. (2019). Tensor algebra compilation with workspaces. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO 2019, pages 180–192. IEEE Press.
- [70] Kjolstad, F., Kamil, S., Chou, S., Lugato, D., and Amarasinghe, S. (2017). The tensor algebra compiler. *Proc. ACM Program. Lang.*, 1(OOPSLA):1–29. Article 77, 29 pages.
- [71] Kocberber, O., Grot, B., Picorel, J., Falsafi, B., Lim, K., and Ranganathan, P. (2013). Meet the walkers accelerating index traversals for in-memory databases. In *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 468–479.
- [72] Koul, K., Strange, M., Melchert, J., Carsello, A., Mei, Y., Hsu, O., Kong, T., Chen, P.-H., Ke, H., Zhang, K., Liu, Q., Nyengele, G., Balasingam, A., Adivarahan, J., Sharma, R., Xie, Z., Torng, C., Emer, J., Kjolstad, F., Horowitz, M., and Raina, P. (2024). Onyx: A 12nm 756 GOPS/W coarse-grained reconfigurable array for accelerating dense and sparse applications. In *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, pages 1–2.

- [73] Kovach, S., Kolichala, P., Gu, T., and Kjolstad, F. (2023). Indexed streams: A formal intermediate representation for fused contraction programs. *Proc. ACM Program. Lang.*, 7(PLDI):1169–1193. Article 154, 25 pages.
- [74] Lacouture, R., Zhang, N., Sharma, R., Siracusa, M., Kjolstad, F., Olukotun, K., and Hsu, O. (2026). FuseFlow: A fusion-centric compilation framework for sparse deep learning on streaming dataflow. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '26*, pages 798–820, New York, NY, USA. Association for Computing Machinery. DOI: <https://doi.org/10.1145/3779212.3790165>.
- [75] Lattner, C. and Adve, V. (2004). Llvm: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, CGO '04, pages 75–86, USA. IEEE Computer Society.
- [76] Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. (2021). MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '21, pages 2–14. IEEE Press.
- [77] Lawson, C. L., Hanson, R. J., Kincaid, D. R., and Krogh, F. T. (1979). Basic linear algebra subprograms for fortran usage. *ACM Trans. Math. Softw.*, 5(3):308–323.
- [78] Leskovec, J. and Krevl, A. (2014). SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>. Accessed: 2026-07-10.
- [79] LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, Cambridge, United Kingdom.
- [80] LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA.
- [81] Li, J., Louri, A., Karanth, A., and Bunescu, R. (2021). GCNAX: A flexible and energy-efficient accelerator for graph convolutional neural networks. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 775–788.
- [82] Li, S., Ahn, J. H., Strong, R. D., Brockman, J. B., Tullsen, D. M., and Jouppi, N. P. (2009). McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 469–480.
- [83] Lin, C. Y., Zhang, Z., Wong, N., and So, H. K.-H. (2010). Design space exploration for sparse matrix-matrix multiplication on FPGAs. In *2010 International Conference on Field-Programmable Technology*, pages 369–372.
- [84] Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Oper. Res.*, 9(3):383–387.

- [85] Liu, J., Ren, J., Gioiosa, R., Li, D., and Li, J. (2021). Sparta: High-performance, element-wise sparse tensor contraction on heterogeneous memory. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '21, pages 318–333, New York, NY, USA. Association for Computing Machinery.
- [86] Liu, J., Zhao, Z., Ding, Z., Brock, B., Rong, H., and Zhang, Z. (2024). UniSparse: An intermediate language for general sparse format customization. *Proc. ACM Program. Lang.*, 8(OOPSLA1):137–165. Article 99, 29 pages.
- [87] LLVM Project (2026). torch-mlir. <https://github.com/llvm/torch-mlir>. Accessed: 2026-05-30.
- [88] Lopoukhine, A., Ficarelli, F., Vasiladiotis, C., Lydike, A., Van Delm, J., Dutilleul, A., Benini, L., Verhelst, M., and Grosser, T. (2025). A multi-level compiler backend for accelerated micro-kernels targeting RISC-V ISA extensions. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '25, pages 163–178, New York, NY, USA. Association for Computing Machinery.
- [89] Lowe-Power, J., Ahmad, A. M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., Black, G., Bloom, G., Bruce, B. R., Carvalho, D. R., Castrillon, J., Chen, L., Derumigny, N., Diestelhorst, S., Elsasser, W., Escuin, C., Fariborz, M., Farmahini-Farahani, A., Fotouhi, P., Gambord, R., Gandhi, J., Gope, D., Grass, T., Gutierrez, A., Hanindhito, B., Hansson, A., Haria, S., Harris, A., Hayes, T., Herrera, A., Horsnell, M., Jafri, S. A. R., Jagtap, R., Jang, H., Jeyapaul, R., Jones, T. M., Jung, M., Kanno, S., Khaleghzadeh, H., Kodama, Y., Krishna, T., Marinelli, T., Menard, C., Mondelli, A., Moreto, M., Mück, T., Naji, O., Nathella, K., Nguyen, H., Nikoleris, N., Olson, L. E., Orr, M., Pham, B., Prieto, P., Reddy, T., Roelke, A., Samani, M., Sandberg, A., Setoain, J., Shingarov, B., Sinclair, M. D., Ta, T., Thakur, R., Travaglini, G., Upton, M., Vaish, N., Vougioukas, I., Wang, W., Wang, Z., Wehn, N., Weis, C., Wood, D. A., Yoon, H., and Zulian, E. F. (2020). The gem5 Simulator: Version 20.0+. arXiv:2007.03152. Accessed: 2026-07-10.
- [90] Luo, H., Tugrul, Y. C., Bostanci, F. N., Olgun, A., Yaglikci, A. G., and Mutlu, O. (2024). Ramulator 2.0: A modern, modular, and extensible DRAM simulator. *IEEE Computer Architecture Letters*, 23(1):112–116.
- [91] Manoni, S., Scheffler, P., Zanatta, L., Acquaviva, A., Benini, L., and Bartolini, A. (2025). SpikeStream: Accelerating spiking neural network inference on RISC-V clusters with sparse computation extensions. In *2025 Design, Automation and Test in Europe Conference (DATE)*, pages 1–7.
- [92] Mantovani, F., Garcia-Gasulla, M., Gracia, J., Stafford, E., Banchelli, F., Josep-Fabrego, M., Criado-Ledesma, J., and Nachtmann, M. (2020). Performance and energy consumption of HPC workloads on a cluster based on Arm ThunderX2 CPU. *Future Generation Computer Systems*, 112:800–818.
- [93] Marí Prats, F. (2025). Integrating the Tensor Marshaling Unit for sparse tensor algebra with a RISC-V processor. Master's thesis, Universitat Politècnica de Catalunya. Available at <https://hdl.handle.net/2117/452568>. Accessed: 2026-07-10.

- [94] Marí Prats, X., Gorfini, D., and López-Paradís, G. (2026). Improvements on queue-based communication for accelerating sparse workloads in hardware with cohort. In *13th International BSC Severo Ochoa Doctoral Symposium 2026: Book of Abstracts*, pages 93–94, Barcelona, Spain. Available at https://www.bsc.es/sites/default/files/public/13th_bsc_so_doctoral_symposium_book_of_abstracts_2026v3.pdf. Accessed: 2026-07-10.
- [95] Milaković, S., Selvitopi, O., Nisa, I., Budimlić, Z., and Buluç, A. (2022). Parallel algorithms for masked sparse matrix-matrix products. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP '22, pages 453–454, New York, NY, USA. Association for Computing Machinery.
- [96] Mishra, A. K., Nurvitadhi, E., Venkatesh, G., Pearce, J., and Marr, D. (2017). Fine-grained accelerators for sparse machine learning workloads. In *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 635–640.
- [97] MLCommons (2025). Introducing the 2025 MLCommons rising stars. <https://mlcommons.org/2025/06/2025-mlc-rising-stars/>. Accessed: 2026-05-30.
- [98] Mudigere, D., Hao, Y., Huang, J., Jia, Z., Tulloch, A., Sridharan, S., Liu, X., Ozdal, M., Nie, J., Park, J., Luo, L., Yang, J. A., Gao, L., Ivchenko, D., Basant, A., Hu, Y., Yang, J., Ardestani, E. K., Wang, X., Komuravelli, R., Chu, C.-H., Yilmaz, S., Li, H., Qian, J., Feng, Z., Ma, Y., Yang, J., Wen, E., Li, H., Yang, L., Sun, C., Zhao, W., Melts, D., Dhulipala, K., Kishore, K., Graf, T., Eisenman, A., Matam, K. K., Gangidi, A., Chen, G. J., Krishnan, M., Nayak, A., Nair, K., Muthiah, B., khorashadi, M., Bhattacharya, P., Lapukhov, P., Naumov, M., Mathews, A., Qiao, L., Smelyanskiy, M., Jia, B., and Rao, V. (2022). Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, pages 993–1011, New York, NY, USA. Association for Computing Machinery.
- [99] Mukkara, A., Beckmann, N., Abeydeera, M., Ma, X., and Sanchez, D. (2018). Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–14.
- [100] Nagasaka, Y., Matsuoka, S., Azad, A., and Buluç, A. (2018). High-performance sparse matrix-matrix products on Intel KNL and multicore architectures. In *Workshop Proceedings of the 47th International Conference on Parallel Processing*, ICPP Workshops '18, pages 1–10, New York, NY, USA. Association for Computing Machinery. Article 34, 10 pages.
- [101] Nassif, N., Munch, A. O., Molnar, C. L., Pasdast, G., Lyer, S. V., Yang, Z., Mendoza, O., Huddart, M., Venkataraman, S., Kandula, S., Marom, R., Kern, A. M., Bowhill, B., Mulvihill, D. R., Nimmagadda, S., Kalidindi, V., Krause, J., Haq, M. M., Sharma, R., and Duda, K. (2022). Sapphire Rapids: The next-generation Intel Xeon scalable processor. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 44–46.
- [102] Naumov, M., Mudigere, D., Shi, H. M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C., Azzolini, A. G., Dzhulgakov, D., Mallevich, A., Cherniavskii, I.,

- Lu, Y., Krishnamoorthi, R., Yu, A., Kondratenko, V., Pereira, S., Chen, X., Chen, W., Rao, V., Jia, B., Xiong, L., and Smelyanskiy, M. (2019). Deep learning recommendation model for personalization and recommendation systems. *CoRR*, abs/1906.00091.
- [103] Nguyen, T., MacLean, C., Siracusa, M., Doerfler, D., Wright, N. J., and Williams, S. (2022). FPGA-based HPC accelerators: An evaluation on performance and energy efficiency. *Concurrency and Computation: Practice and Experience*, 34(20):e6570.
- [104] Nurvitadhi, E., Mishra, A., and Marr, D. (2015). A sparse matrix vector multiply accelerator for support vector machine. In *2015 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, pages 109–116.
- [105] Nurvitadhi, E., Mishra, A., Wang, Y., Venkatesh, G., and Marr, D. (2016). Hardware accelerator for analytics of sparse data. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1616–1621.
- [106] NVIDIA Corporation (2022). NVIDIA Hopper Architecture In-Depth. <https://www.nvidia.com/en-us/data-center/technologies/hopper-architecture/>. Whitepaper. Accessed: 2026-07-10.
- [107] NVIDIA Corporation (2024a). NVIDIA H100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/h100/>. Accessed: 2026-07-10.
- [108] NVIDIA Corporation (2024b). NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>. Accessed: 2026-07-10.
- [109] Oliveira, G. F., Gómez-Luna, J., Orosa, L., Ghose, S., Vijaykumar, N., Fernandez, I., Sadrosadati, M., and Mutlu, O. (2021). Damov: A new methodology and benchmark suite for evaluating data movement bottlenecks. *IEEE Access*, 9:134457–134502.
- [110] Orenes-Vera, M., Manocha, A., Balkind, J., Gao, F., Aragón, J. L., Wentzlaff, D., and Martonosi, M. (2022). Tiny but mighty: Designing and realizing scalable latency tolerance for manycore SoCs. In *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA '22*, pages 817–830, New York, NY, USA. Association for Computing Machinery.
- [111] Pal, S., Beaumont, J., Park, D.-H., Amarnath, A., Feng, S., Chakrabarti, C., Kim, H.-S., Blaauw, D., Mudge, T., and Dreslinski, R. (2018). OuterSPACE: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 724–736. IEEE.
- [112] Parashar, A., Rhu, M., Mukkara, A., Puglielli, A., Venkatesan, R., Khailany, B., Emer, J., Keckler, S. W., and Dally, W. J. (2017). SCNN: An accelerator for compressed-sparse convolutional neural networks. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 27–40, Los Alamitos, CA, USA. IEEE Computer Society.
- [113] Parravicini, A., Cellamare, L. G., Siracusa, M., and Santambrogio, M. D. (2021). Scaling up HBM efficiency of top-K SpMV for approximate embedding similarity on FPGAs. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 799–804. IEEE.

- [114] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, NeurIPS '19, pages 8024–8035, Red Hook, NY, USA. Curran Associates Inc.
- [115] Pellauer, M., Clemons, J., Balaji, V., Crago, N., Jaleel, A., Lee, D., O'Connor, M., Parashar, A., Treichler, S., Tsai, P.-A., Keckler, S. W., and Emer, J. S. (2023). Symphony: Orchestrating sparse and dense tensors with hierarchical heterogeneous processing. *ACM Trans. Comput. Syst.*, 41(1-4):1–30. Article 4, 30 pages.
- [116] Pellegrini, A., Stephens, N., Bruce, M., Ishii, Y., Pusdesris, J., Raja, A., Abernathy, C., Koppanalil, J., Ringe, T., Tummala, A., Jalal, J., Werkheiser, M., and Kona, A. (2020). The Arm Neoverse N1 platform: Building blocks for the next-gen cloud-to-edge infrastructure SoC. *IEEE Micro*, 40(2):53–62.
- [117] Pennington, J., Socher, R., and Manning, C. (2014). GloVe: Global vectors for word representation. In Moschitti, A., Pang, B., and Daelemans, W., editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.
- [118] Phipps, E. T. and Kolda, T. G. (2019). Software for sparse tensor decomposition on emerging computing architectures. *SIAM Journal on Scientific Computing*, 41(3):C269–C290.
- [119] Qin, E., Samajdar, A., Kwon, H., Nadella, V., Srinivasan, S., Das, D., Kaul, B., and Krishna, T. (2020). SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 58–70.
- [120] Rahman, M. K., Sujon, M. H., and Azad, A. (2021). FusedMM: A unified SDDMM-SpMM kernel for graph embedding and graph neural networks. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 256–266.
- [121] Reguly, I. Z. (2023). Comparative evaluation of bandwidth-bound applications on the Intel Xeon CPU MAX series. arXiv:2309.09084. Accessed: 2026-07-12.
- [122] Rotem, N., Fix, J., Abdulrasool, S., Deng, S., Dzhabarov, R., Hegeman, J., Levenstein, R., Maher, B., Satish, N., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *CoRR*, abs/1805.00907.
- [123] Rucker, A., Vilim, M., Zhao, T., Zhang, Y., Prabhakar, R., and Olukotun, K. (2021). Capstan: A vector rda for sparsity. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, pages 1022–1035, New York, NY, USA. Association for Computing Machinery.
- [124] Saad, Y. and Schultz, M. H. (1986). GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869.

- [125] Sadi, F., Sweeney, J., Low, T. M., Hoe, J. C., Pileggi, L., and Franchetti, F. (2019). Efficient SpMV operation for large and highly sparse matrices using scalable multi-way merge parallelization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-52, pages 347–358, New York, NY, USA. Association for Computing Machinery.
- [126] Sato, M., Ishikawa, Y., Tomita, H., Kodama, Y., Odajima, T., Tsuji, M., Yashiro, H., Aoki, M., Shida, N., Miyoshi, I., Hirai, K., Furuya, A., Asato, A., Morita, K., and Shimizu, T. (2020). Co-design for A64FX manycore processor and Fugaku. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '20, pages 1–15. IEEE Press. Article 47, 15 pages.
- [127] Scheffler, P., Benz, T., Potocnik, V., Fischer, T., Colagrande, L., Wistoff, N., Zhang, Y., Bertaccini, L., Ottavi, G., Eggimann, M., Cavalcante, M., Paulin, G., Gürkaynak, F. K., Rossi, D., and Benini, L. (2025). Occamy: A 432-core dual-chiplet dual-HBM2E 768-DP-GFLOP/s RISC-V system for 8-to-64-bit dense and sparse computing in 12-nm FinFET. *IEEE Journal of Solid-State Circuits*, 60(4):1324–1338.
- [128] Scheffler, P., Colagrande, L., and Benini, L. (2024). SARIS: Accelerating stencil computations on energy-efficient RISC-V compute clusters with indirect stream registers. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, DAC '24, pages 1–6, New York, NY, USA. Association for Computing Machinery. Article 251, 6 pages.
- [129] Scheffler, P., Zaruba, F., Schuiki, F., Hoefler, T., and Benini, L. (2021). Indirection stream semantic register architecture for efficient sparse-dense linear algebra. In *2021 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pages 1787–1792.
- [130] Scheffler, P., Zaruba, F., Schuiki, F., Hoefler, T., and Benini, L. (2023). Sparse stream semantic registers: A lightweight isa extension accelerating general sparse linear algebra. *IEEE Transactions on Parallel and Distributed Systems*, 34(12):3147–3161.
- [131] Selvitopi, O., Brock, B., Nisa, I., Tripathy, A., Yelick, K., and Buluç, A. (2021). Distributed-memory parallel algorithms for sparse times tall-skinny-dense matrix multiplication. In *Proceedings of the 35th ACM International Conference on Supercomputing*, ICS '21, pages 431–442, New York, NY, USA. Association for Computing Machinery.
- [132] Senanayake, R., Hong, C., Wang, Z., Wilson, A., Chou, S., Kamil, S., Amarasinghe, S., and Kjolstad, F. (2020). A sparse iteration space transformation framework for sparse tensor algebra. *Proc. ACM Program. Lang.*, 4(OOPSLA):1–30. Article 158, 30 pages.
- [133] Sethi, G., Acun, B., Agarwal, N., Kozyrakis, C., Trippel, C., and Wu, C.-J. (2022). Recshard: statistical feature-based memory optimization for industry-scale neural recommendation. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, pages 344–358, New York, NY, USA. Association for Computing Machinery.
- [134] Sgherzi, F., Parravicini, A., Siracusa, M., and Santambrogio, M. D. (2021). Solving large top-K graph eigenproblems with a memory and compute-optimized FPGA design. In *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 78–87.

- [135] Sgherzi, F., Siracusa, M., Fernandez, I., Armejach, A., and Moretó, M. (2024). SpChar: Characterizing the sparse puzzle via decision trees. *Journal of Parallel and Distributed Computing*, 192:104941.
- [136] Siracusa, M., Del Sozzo, E., Rabozzi, M., Di Tucci, L., Williams, S., Sciuto, D., and Santambrogio, M. D. (2022). A comprehensive methodology to optimize FPGA designs via the roofline model. *IEEE Transactions on Computers*, 71(8):1903–1915.
- [137] Siracusa, M. and Ferrandi, F. (2020). Tensor optimization for high-level synthesis design flows. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(11):4217–4228.
- [138] Siracusa, M., Hsu, O., Soria-Pardos, V., Randall, J., Grasset, A., Biscondi, E., Joseph, D. J., Allen, R., Kjolstad, F., Moretó Planas, M., and Armejach, A. (2026). Ember: A compiler for embedding operations on decoupled access-execute architectures. In *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, CGO '26, pages 150–163. IEEE. DOI: <https://doi.org/10.1109/CGO68049.2026.11395192>.
- [139] Siracusa, M., Randall, J., Joseph, D. J., Moretó Planas, M., and Armejach Sanosa, A. (2025). Data structure marshalling unit. US Patent Application US20250077499A1, Arm Ltd., published March 6, 2025. <https://patents.google.com/patent/US20250077499A1/en>. Accessed: 2026-07-10.
- [140] Siracusa, M., Soria-Pardos, V., Sgherzi, F., Randall, J., Joseph, D. J., Moretó Planas, M., and Armejach, A. (2023). A tensor marshaling unit for sparse tensor algebra on general-purpose processors. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO '23*, pages 1332–1346, New York, NY, USA. Association for Computing Machinery.
- [141] Smith, J. E. (1982). Decoupled access/execute computer architectures. In *Proceedings of the 9th Annual Symposium on Computer Architecture, ISCA '82*, pages 112–119, Washington, DC, USA. IEEE Computer Society Press.
- [142] Smith, L. L. (1991). Vectorizing c compilers: how good are they? In *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing, Supercomputing '91*, pages 544–553, New York, NY, USA. Association for Computing Machinery.
- [143] Smith, S., Choi, J. W., Li, J., Vuduc, R., Park, J., Liu, X., and Karypis, G. (2017). FROSTT: The formidable repository of open sparse tensors and tools. <http://frostd.io/>. Accessed: 2026-07-10.
- [144] Smith, S. and Karypis, G. (2015). Tensor-matrix products with a compressed sparse tensor. In *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms, IA³ '15*, pages 1–7, New York, NY, USA. Association for Computing Machinery. Article 5, 7 pages.
- [145] Srivastava, N., Jin, H., Liu, J., Albonesi, D., and Zhang, Z. (2020). MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 766–780. IEEE.

- [146] Stephens, N., Biles, S., Boettcher, M., Eapen, J., Eyole, M., Gabrielli, G., Horsnell, M., Magklis, G., Martinez, A., Premillieu, N., Reid, A., Rico, A., and Walker, P. (2017). The arm scalable vector extension. *IEEE Micro*, 37(02):26–39.
- [147] Stillmaker, A. and Baas, B. (2017). Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration*, 58:74–81.
- [148] Talati, N., May, K., Behroozi, A., Yang, Y., Kaszyk, K., Vasiladiotis, C., Verma, T., Li, L., Nguyen, B., Sun, J., Morton, J. M., Ahmadi, A., Austin, T., O’Boyle, M., Mahlke, S., Mudge, T., and Dreslinski, R. (2021). Prodigy: Improving the memory latency of data-indirect irregular workloads using hardware-software co-design. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 654–667.
- [149] Talati, N., Ye, H., Yang, Y., Belayneh, L., Chen, K.-Y., Blaauw, D., Mudge, T., and Dreslinski, R. (2022). NDMiner: Accelerating graph pattern mining using near data processing. In *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA ’22*, pages 146–159, New York, NY, USA. Association for Computing Machinery.
- [150] The IREE Authors (2019). IREE. Software. <https://github.com/iree-org/iree>. Accessed: 2026-07-10.
- [151] Umuroglu, Y. and Jahre, M. (2014). An energy efficient column-major backend for FPGA SpMV accelerators. In *2014 IEEE 32nd International Conference on Computer Design (ICCD)*, pages 432–439.
- [152] van der Vorst, H. A. (1992). Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 13(2):631–644.
- [153] Wang, M., Zheng, D., Ye, Z., Gan, Q., Li, M., Song, X., Zhou, J., Ma, C., Yu, L., Gai, Y., Xiao, T., He, T., Karypis, G., Li, J., and Zhang, Z. (2019). Deep graph library: A graph-centric, highly-performant package for graph neural networks. *CoRR*, abs/1909.01315.
- [154] Wang, Z., Wang, Y., Feng, B., Mudigere, D., Muthiah, B., and Ding, Y. (2022). EL-Rec: Efficient large-scale recommendation model training via tensor-train embedding table. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14.
- [155] Weng, J., Liu, S., Dadu, V., Wang, Z., Shah, P., and Nowatzki, T. (2020). DSAGEN: Synthesizing programmable spatial accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 268–281.
- [156] Wilkinson, F. and McIntosh-Smith, S. (2022). An initial evaluation of arm’s scalable matrix extension. In *2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 135–140.
- [157] Williams, S., Oliker, L., Vuduc, R., Shalf, J., Yelick, K., and Demmel, J. (2007). Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC ’07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, pages 1–12.

- [158] Williams, S., Waterman, A., and Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76.
- [159] Wulf, W. A. and McKee, S. A. (1995). Hitting the memory wall: implications of the obvious. *SIGARCH Comput. Archit. News*, 23(1):20–24.
- [160] Yadav, R., Aiken, A., and Kjolstad, F. (2022). SpDISTAL: Compiling distributed sparse tensor computations. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, SC '22, pages 1–15. IEEE Press. Article 59, 15 pages.
- [161] Yang, Y., Emer, J. S., and Sanchez, D. (2021). SpZip: Architectural support for effective data compression in irregular applications. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1069–1082.
- [162] Ye, Z., Lai, R., Shao, J., Chen, T., and Ceze, L. (2023). SparseTIR: Composable abstractions for sparse compilation in deep learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, pages 660–678, New York, NY, USA. Association for Computing Machinery.
- [163] Yu, X., Hughes, C. J., Satish, N., and Devadas, S. (2015). IMP: Indirect memory prefetcher. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 178–190.
- [164] Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. (2020). Big bird: Transformers for longer sequences. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 17283–17297. Curran Associates, Inc.
- [165] Zhang, G., Attaluri, N., Emer, J. S., and Sanchez, D. (2021). GAMMA: Leveraging gustavson’s algorithm to accelerate sparse matrix multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 687–701.
- [166] Zhang, G., Hsu, O., and Kjolstad, F. (2024). Compilation of modular and general sparse workspaces. *Proc. ACM Program. Lang.*, 8(PLDI):1213–1238. Article 196, 26 pages.
- [167] Zhang, S., Du, Z., Zhang, L., Lan, H., Liu, S., Li, L., Guo, Q., Chen, T., and Chen, Y. (2016). Cambricon-X: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–12.
- [168] Zhang, Z., Wang, H., Han, S., and Dally, W. J. (2020). SpArch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 261–274. IEEE.
- [169] Zhou, X., Du, Z., Guo, Q., Liu, S., Liu, C., Wang, C., Zhou, X., Li, L., Chen, T., and Chen, Y. (2018). Cambricon-S: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 15–28.

-
- [170] Zienkiewicz, O. C., Taylor, R. L., and Zhu, J. Z. (2013). *The Finite Element Method: Its Basis and Fundamentals*. Butterworth-Heinemann, Oxford, United Kingdom, 7 edition.

